

1. Semantic HTML

Q: What is Semantic HTML and why is it important?

Semantic HTML uses elements according to their meaning rather than using generic `<div>` elements for everything.

Examples:

`<header>`

`<nav>`

`<main>`

`<section>`

`<article>`

`<footer>`

`<button>`

Benefits:

- Better accessibility
- Better SEO
- Clearer code structure
- Better browser/assistive-technology support
- Easier maintenance

Interview answer:

Semantic HTML means using HTML elements according to their intended meaning. It improves accessibility, SEO, maintainability, and allows browsers and assistive technologies to understand the structure of the page.

2. aria-label vs aria-labelledby vs aria-describedby

aria-label provides an accessible name directly.

`<button aria-label="Close menu">X</button>`

aria-labelledby uses another element's text as the accessible name.

`<h2 id="title">Delete account</h2>`

`<div role="dialog" aria-labelledby="title">`

aria-describedby provides additional descriptive information.

`<p id="help">Password must contain 8 characters.</p>`

`<input aria-describedby="help">`

Easy way to remember:

- `aria-label` → **name directly**
 - `aria-labelledby` → **name from another element**
 - `aria-describedby` → **additional description**
-

3. tabindex

`tabindex` controls whether an element can receive keyboard focus.

`tabindex="0"`

Adds the element to the normal keyboard tab order.

`<div tabindex="0">Focusable</div>`

`tabindex="-1"`

The element can receive programmatic focus but isn't reached using normal Tab navigation.

`element.focus();`

Avoid positive values like:

`tabindex="5"`

because they create a custom and difficult-to-maintain focus order.

Best practice: Prefer native interactive elements such as `<button>`, `<a>`, and `<input>` instead of adding `tabindex` to non-interactive elements.

4. Focus Management in SPAs

In a traditional page navigation, the browser naturally moves focus to the new document.

In an SPA, React changes the content without a full page navigation, so focus may remain on the old navigation element.

Good focus management means intentionally moving focus to meaningful content after navigation.

For example:

`headingRef.current?.focus();`

The target should usually have:

`<h1 tabindex="-1">Dashboard</h1>`

Interview answer:

In an SPA, route changes don't automatically create a new document, so I manage focus manually. After navigation, I generally move focus to the new page's main heading or another appropriate landmark.

5. Focus Trapping and Modal Focus

Focus trapping means keeping keyboard focus inside a modal while it is open.

For example:

Modal

↓

Input

↓

Cancel

↓

Submit

↓

Input

Pressing Tab should not move focus behind the modal.

Use it for:

- Modal dialogs
- Dialog-like overlays
- Some full-screen menus

Don't use it unnecessarily for ordinary components.

6. inert

`inert` makes an element and its descendants essentially unavailable for user interaction and focus.

`<main inert>`

This prevents interaction with the background when a modal is open.

It is useful because it handles more than just accessibility-tree visibility—it also prevents user interaction and focus.

Remember:

`aria-hidden` hides from assistive technologies; `inert` makes content non-interactive as well.

7. aria-live

An ARIA live region tells assistive technology that dynamically changing content should be announced.

```
<div aria-live="polite"></div>
```

polite

Waits until the screen reader has an appropriate opportunity.

Good for:

- "Saved successfully"
- Search result count
- Background updates

assertive

Interrupts the current announcement.

Use sparingly for urgent information.

8. Accessible Custom Dropdown

A custom dropdown is much harder to make accessible than a native `<select>`.

If possible:

Use native `<select>` first.

If custom behavior is required, you need:

- Correct ARIA roles
- Keyboard navigation
- Focus management
- Selected state
- Expanded/collapsed state
- Screen-reader announcements

Typical structure:

```
<div role="combobox"  
  aria-expanded="false">
```

Options may use:

```
<div role="option">
```

Keyboard support should include:

- Arrow Up/Down

- Enter
 - Escape
 - Home/End where appropriate
 - Typeahead where appropriate
-

9. Accessible Form Validation

A good accessible validation strategy includes:

1. Associate every input with a label.
2. Identify invalid fields.
3. Provide text errors.
4. Associate errors using aria-describedby.
5. Use aria-invalid="true".
6. Move focus appropriately after submission.
7. Don't rely only on color.

Example:

```
<label for="email">Email</label>
```

```
<input  
  id="email"  
  aria-invalid="true"  
  aria-describedby="email-error"  
>
```

```
<p id="email-error">  
  Enter a valid email address.  
</p>
```

10. async vs defer vs type="module"

Normal script

```
<script src="app.js"></script>
```

HTML parsing pauses while the script is fetched and executed.

defer

`<script src="app.js" defer></script>`

- Downloads while HTML parses
- Executes after HTML parsing
- Maintains execution order
- Good default for traditional scripts

async

`<script src="analytics.js" async></script>`

- Downloads in parallel
- Executes as soon as downloaded
- Execution order isn't guaranteed
- Good for independent scripts

Module

`<script type="module" src="app.js"></script>`

Modules are deferred by default and support ES modules.

Interview summary:

defer preserves order and runs after parsing, async runs as soon as the script is ready, and modules are deferred by default while providing native module semantics.

11. preload vs prefetch vs preconnect

preload

"Download this resource because the current page needs it soon."

`<link rel="preload" href="/font.woff2" as="font">`

prefetch

"Download this resource because it may be needed later."

`<link rel="prefetch" href="/next-page.js">`

preconnect

"Establish the connection early."

`<link rel="preconnect" href="https://api.example.com">`

dns-prefetch

Only performs DNS resolution early.

`<link rel="dns-prefetch" href="//api.example.com">`

Memory trick:

preload → resource now
prefetch → resource later
preconnect → connection now
dns-prefetch → DNS now

12. <dialog> and showModal()

<dialog> is a native HTML element for dialogs.

```
<dialog id="dialog">  
  <h2>Delete account?</h2>  
</dialog>
```

JavaScript:

```
dialog.showModal();
```

showModal() creates a modal dialog behavior and places the dialog into the browser's top-layer model.

It provides important built-in behavior such as modal interaction semantics and backdrop support.

You still need to provide:

- Meaningful title
 - Appropriate controls
 - Good focus behavior
 - Correct content
-

CSS

13. Flexbox Main Axis / Cross Axis

Flexbox has two axes.

```
.container {  
  display: flex;  
}
```

By default:

main axis → horizontal

cross axis → vertical

flex-direction determines the main axis.

```
flex-direction: row;
```

means:

main → horizontal

cross → vertical

With:

flex-direction: column;

the axes switch.

14. Flexbox vs Grid

Flexbox

One-dimensional layout.

Use it for:

- Navigation bars
- Button groups
- Aligning items in a row
- Component-level layouts

Grid

Two-dimensional layout.

Use it for:

- Page layouts
- Card grids
- Dashboard layouts
- Rows + columns

Interview answer:

Flexbox is primarily one-dimensional, while Grid is two-dimensional. I use Flexbox for component-level alignment and Grid when I need control over both rows and columns.

15. CSS Positioning

static

Normal document flow.

relative

Remains in normal flow but can be visually offset.

Also establishes the containing block commonly used by absolutely positioned descendants.

absolute

Removed from normal flow and positioned relative to an appropriate containing block.

fixed

Positioned relative to the viewport in the common case and stays fixed during scrolling.

sticky

Behaves relatively until a threshold is reached, then behaves like a sticky positioned element.

```
header {  
  position: sticky;  
  top: 0;  
}
```

16. Stacking Context and z-index

z-index controls stacking order **within stacking contexts**.

A large value doesn't necessarily beat another element because they may belong to different stacking contexts.

Stacking contexts can be created by things such as:

- Positioned elements with z-index
- opacity < 1
- transform
- Certain contain values
- isolation
- Other CSS properties

So:

```
z-index: 999999;
```

doesn't mean "always on top."

17. CSS Specificity

Generally, specificity is compared roughly as:

!important

inline styles

ID

class / attribute / pseudo-class

element / pseudo-element

Example:

```
#app .button {}
```

is more specific than:

```
.button {}
```

If specificity is equal, the later rule generally wins.

18. Responsive Design

I usually approach responsive design using:

- Mobile-first CSS
- Flexible layouts
- Flexbox/Grid
- Relative units
- Media queries
- Container queries where appropriate
- Responsive typography
- Responsive images
- Touch-friendly controls

Example:

```
.card {  
  width: 100%;  
}
```

```
@media (min-width: 768px) {  
  .card {  
    width: 50%;  
  }  
}
```

19. Reflow, Repaint and Compositing

Reflow/Layout

Browser recalculates geometry.

Examples:

- Changing width
- Changing height
- Changing position affecting layout

Repaint

Pixels are redrawn.

Example:

color: red;

Composite

Browser combines already-rendered layers.

Properties such as transform and opacity can often be handled efficiently at the compositing stage.

Performance goal:

Minimize unnecessary layout and paint work, especially during animations.

20. TypeScript Fundamentals

TypeScript is a statically typed superset of JavaScript.

It helps catch errors during development:

```
function add(a: number, b: number) {  
    return a + b;  
}
```

Calling:

```
add("10", 20);
```

produces a compile-time error.

Benefits:

- Safer refactoring
- Better autocomplete
- Better documentation
- Earlier error detection

- Better large-codebase maintainability

Important:

TypeScript types are erased at runtime.

21. Union vs Intersection

Union

```
type ID = string | number;
```

Means the value can be **one of these types**.

Intersection

```
type Employee = Person & Developer;
```

Means the value must satisfy **both types**.

Memory:

Union → OR

Intersection → AND

22. Interface vs Type

Both can describe object shapes.

```
interface User {  
  name: string;  
}
```

```
type User = {  
  name: string;  
};
```

Interfaces support declaration merging and are commonly useful for extensible object contracts.

Types are more flexible for:

- Unions
- Intersections
- Tuples
- Mapped/conditional compositions

Interview answer:

For object-oriented/extensible object contracts I often use interfaces; for unions, intersections and type-level composition I use type aliases. Both are valid for many object-shape cases.

23. Type Narrowing

Type narrowing means reducing a broader type to a more specific type.

```
function print(value: string | number) {  
  if (typeof value === "string") {  
    console.log(value.toUpperCase());  
  }  
}
```

Inside the if, TypeScript knows value is a string.

Common techniques:

typeof

instanceof

in

=== checks

truthiness checks

custom type guards

discriminated unions

24. Discriminated Unions

A discriminated union uses a common property to determine which variant you have.

```
type Result =  
  | { status: "success"; data: string }  
  | { status: "error"; message: string };
```

Then:

```
if (result.status === "success") {  
  console.log(result.data);  
}
```

TypeScript automatically narrows the type.

Very useful for:

- API states
 - Redux state
 - UI states
 - Async operations
-

25. Generics

Generics allow reusable code while preserving type information.

```
function identity<T>(value: T): T {  
  return value;  
}
```

```
const result = identity("hello");
```

TypeScript knows:

```
result: string
```

Instead of using any, generics preserve relationships between input and output types.

26. keyof

keyof produces a union of property names.

```
interface User {  
  id: number;  
  name: string;  
}
```

```
type UserKey = keyof User;
```

Equivalent to:

```
"id" | "name"
```

Very useful with generics:

```
function getValue<T, K extends keyof T>(  
  obj: T,  
  key: K  
) {  
  return obj[key];
```

```
}
```

This ensures the key actually exists.

27. Utility Types

Common utility types:

`Partial<T>`

`Pick<T, K>`

`Omit<T, K>`

`Required<T>`

`Readonly<T>`

`Record<K, T>`

Example:

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}
```

```
type UpdateUser = Partial<User>;
```

Now all properties are optional.

28. React Props / Events / Refs

Props

```
interface ButtonProps {  
  label: string;  
  disabled?: boolean;  
}
```

```
function Button({ label, disabled }: ButtonProps) {  
  return <button disabled={disabled}>{label}</button>;  
}
```

Events

```
const handleChange = (
```

```
    event: React.ChangeEvent<HTMLInputElement>
  ) => {};
```

Refs

```
const inputRef = useRef<HTMLInputElement>(null);
```

This gives:

```
inputRef.current?.focus();
```

29. API Response Validation

A TypeScript interface only provides compile-time information.

It does **not** validate actual runtime JSON.

The server could return:

```
{
  "id": "wrong"
}
```

even if you wrote:

```
interface User {
  id: number;
}
```

For runtime validation, use tools such as Zod.

```
const UserSchema = z.object({
  id: z.number(),
  name: z.string()
});
```

Senior answer:

TypeScript protects my code at compile time, but API data is external runtime data, so I validate untrusted responses at runtime using a schema library such as Zod.

30. unknown vs any

any disables type checking.

let value: any;

```
value.foo.bar();
```


TypeScript allows it.

unknown is safer:

```
let value: unknown;
```

You must narrow it before using it.

```
if (typeof value === "string") {  
  value.toUpperCase();  
}
```

Rule:

Prefer unknown for data whose type you don't know.

Networking

31. URL → DNS → TCP → TLS → HTTP → Rendering

A simplified flow:

URL

↓

DNS lookup

↓

TCP connection

↓

TLS handshake

↓

HTTP request

↓

Server response

↓

HTML parsing

↓

DOM + CSSOM

↓

Render tree

↓

Layout

↓

Paint

↓

Composite

Modern HTTP/3 uses QUIC over UDP rather than TCP, so the exact transport steps depend on the protocol.

32. TCP vs UDP

TCP

- Connection-oriented
- Reliable
- Ordered
- Retransmission
- Flow/congestion control

UDP

- Connectionless
- No built-in delivery guarantee
- Lower protocol overhead
- Useful where speed/latency matters

Examples:

TCP:

HTTP/1.1

HTTP/2

UDP-based:

HTTP/3 / QUIC

33. HTTP/1.1 vs HTTP/2 vs HTTP/3

HTTP/1.1

- Text-based protocol
- Usually one request/response per connection at a time
- Multiple connections often used

HTTP/2

- Binary framing
- Multiplexing
- Header compression
- Multiple streams over one TCP connection

HTTP/3

- Uses QUIC
 - Runs over UDP
 - Multiplexed streams
 - Avoids TCP-level head-of-line blocking between streams
-

34. REST Principles

Important REST ideas include:

- Client-server separation
- Stateless requests
- Cacheability
- Uniform interface
- Resource-oriented design
- Layered architecture

Example:

GET /users/123

represents a resource.

35. HTTP Methods and Idempotency

GET

Retrieve data.

POST

Create/process a resource; generally not idempotent.

PUT

Replace a resource.

PATCH

Partially update a resource.

DELETE

Delete a resource.

Idempotent means making the same operation multiple times has the same intended final effect as making it once.

Typically:

GET → idempotent

PUT → idempotent

DELETE → idempotent

POST → not generally idempotent

PATCH → depends on operation

36. HTTP Status Codes

2xx — Success

200 OK

201 Created

204 No Content

3xx — Redirection

301

302

304

4xx — Client-side request problem

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

429 Too Many Requests

5xx — Server-side failure

500 Internal Server Error

502 Bad Gateway

503 Service Unavailable

504 Gateway Timeout

37. 401 vs 403

401

Authentication is missing or invalid.

Example:

"You aren't authenticated."

403

The server understands who you are but you aren't allowed to perform the operation.

Example:

"You are authenticated, but don't have permission."

Memory:

401 → Who are you?

403 → You're known, but not allowed.

38. DOM / CSSOM / Render Tree

DOM

Tree representing HTML.

CSSOM

Tree representing CSS rules/styles.

Browser combines information from both to construct the render tree.

Simplified:

HTML

↓

DOM

CSS

↓

CSSOM

DOM + CSSOM

↓

Render Tree

↓

Layout

↓

Paint

39. Event Bubbling / Capturing / Delegation

Event propagation has phases:

Capturing

↓

Target

↓

Bubbling

Example:

```
parent.addEventListener("click", handler);
```

A click on a child can bubble to the parent.

Event delegation takes advantage of this:

```
list.addEventListener("click", event => {  
  const item = event.target.closest("li");  
});
```

Instead of attaching listeners to every child.

Benefits:

- Fewer listeners
 - Works well with dynamic children
 - Centralized event handling
-

40. Same-Origin Policy and CORS

An origin is:

scheme + host + port

Example:

<https://example.com:443>

Same-Origin Policy restricts how scripts interact with resources from different origins.

CORS allows a server to explicitly permit cross-origin browser requests.

Example:

Access-Control-Allow-Origin: <https://app.example.com>

Important:

CORS is primarily a browser security mechanism. It isn't the API server "blocking Postman."

41. WebSocket

WebSocket provides persistent, bidirectional communication between browser and server.

Unlike normal HTTP request/response communication:

Client ↔ Server

Both sides can send messages independently.

Useful for:

- Chat
 - Live notifications
 - Multiplayer games
 - Real-time dashboards
 - Collaboration
-

42. SSE vs WebSocket

SSE

Server → Client.

Good for:

- Notifications
- Live feeds
- Streaming server responses
- Progress updates

WebSocket

Client ↔ Server.

Good for:

- Chat
- Games

- Collaborative applications
- Bidirectional real-time communication

Memory:

SSE → server → browser

WebSocket → both directions

43. Browser Storage

Storage	Persistence	Sent automatically with HTTP?	Typical use
localStorage	Persistent	No	Preferences
sessionStorage	Tab/session	No	Temporary state
Cookies	Configurable	Yes, when applicable	Sessions
IndexedDB	Persistent	No	Large structured client data

Cookies can have security attributes such as:

HttpOnly

Secure

SameSite

44. JWT Storage

Avoid automatically saying:

"JWT should always be stored in localStorage."

localStorage is accessible to JavaScript, so an XSS vulnerability can expose a token stored there.

A common safer architecture is:

HttpOnly + Secure + SameSite cookie

for session/refresh credentials, combined with appropriate CSRF protection depending on the architecture.

Senior answer:

I don't treat JWT storage as a one-size-fits-all decision. I generally avoid storing sensitive long-lived tokens in localStorage when an HttpOnly secure cookie architecture is appropriate because JavaScript cannot directly read HttpOnly cookies.

45. HTTPS / Web Security

HTTPS is HTTP over a secure cryptographic connection.

It provides:

Confidentiality

Attackers shouldn't be able to read traffic in transit.

Integrity

Attackers shouldn't be able to silently modify traffic.

Authentication

TLS certificates help the browser authenticate the server's identity.

HTTPS does **not** automatically protect against:

- XSS
 - CSRF
 - SQL/NoSQL injection
 - Broken authorization
 - Vulnerable application logic
-

Tier 2 — Strongly Recommended

46. ARIA Roles and States

ARIA provides accessibility semantics when native HTML isn't sufficient.

Three useful categories:

Roles

What something is.

`role="dialog"`

States

Current state.

`aria-expanded="true"`

`aria-checked="true"`

Properties

Additional information.

`aria-label="Close"`

`aria-describedby="help"`

Best practice:

Prefer native HTML semantics before adding ARIA.

47. aria-activedescendant

Allows a composite widget to keep DOM focus on one container while indicating which descendant is logically active.

Example:

```
<div
  role="combobox"
  tabindex="0"
  aria-activedescendant="option-2">
</div>
```

Useful for:

- Comboboxes
 - Listboxes
 - Grids
 - Tree views
-

48. aria-selected

Indicates that an option/item is currently selected.

```
<div role="option" aria-selected="true">
```

React

```
</div>
```

Important distinction:

aria-selected

→ selection state

aria-activedescendant

→ which item currently has logical focus

49. Accessible Error Summary

For forms with many errors:

1. Display a summary near the top.
2. Explain the number/type of errors.
3. Link each error to the corresponding field.
4. Move focus to the summary after failed submission when appropriate.

Example:

```
<a href="#email">  
  Email is invalid  
</a>
```

This helps keyboard and screen-reader users quickly understand what needs fixing.

50. role="status" vs role="alert"

role="status"

Used for non-urgent updates.

Equivalent conceptually to a polite live region.

Example:

"Profile saved"

role="alert"

Used for urgent information that should generally be announced promptly.

Example:

"Payment failed"

Use alerts sparingly.

51. Container Queries

Media queries respond to the viewport.

```
@media (min-width: 768px) {}
```

Container queries respond to the size of a containing element.

```
@container (min-width: 500px) {}
```

This is useful for reusable components because the component can adapt to its available space rather than the entire viewport.

52. CSS Custom Properties / Design Tokens

Custom properties:

```
:root {  
  --color-primary: #2563eb;  
  --spacing-md: 16px;  
}
```

Use:

```
button {  
  background: var(--color-primary);  
}
```

They are useful for:

- Theming
- Dark mode
- Design systems
- Consistent spacing
- Runtime customization

Design tokens are standardized values representing things such as:

colors

spacing

typography

border radius

shadows

53. **clamp()**

clamp() allows a value to have a minimum, preferred, and maximum.

```
font-size: clamp(1rem, 2vw, 2rem);
```

Meaning:

minimum → 1rem

preferred → 2vw

maximum → 2rem

Useful for responsive typography and spacing.

54. **:has()**

:has() is a relational CSS pseudo-class.

```
form:has(input:invalid) {  
  border-color: red;  
}
```

It allows selecting an element based on what it contains or is related to.

It is sometimes described as a CSS "parent selector", although its capabilities are broader.

55. CSS @layer

@layer lets you explicitly control cascade precedence between groups of CSS rules.

@layer reset, base, components, utilities;

This reduces specificity battles and makes large CSS systems easier to reason about.

56. FLIP

FLIP stands for:

First

Last

Invert

Play

Basic idea:

1. Measure initial position.
2. Apply final state.
3. Measure final position.
4. Calculate the difference.
5. Use transforms to visually animate from old to new position.

It helps create performant layout animations.

57. View Transitions API

The View Transitions API provides browser support for animated transitions between visual states.

It can be used for:

- SPA route transitions
- Page transitions

- Element continuity

Instead of manually coordinating every animation, the browser can capture visual states and animate between them.

58. prefers-reduced-motion

Users may configure their OS to reduce motion.

CSS:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation-duration: 0.01ms;  
    transition-duration: 0.01ms;  
  }  
}
```

This is important for users who experience discomfort or other issues from animations.

59. prefers-color-scheme

Detects the user's system preference.

```
@media (prefers-color-scheme: dark) {  
  body {  
    background: black;  
  }  
}
```

It can be combined with CSS custom properties for theming.

60. satisfies

satisfies checks that a value conforms to a type **without changing the inferred type of the expression to that type**.

```
const config = {  
  retries: 3,  
  mode: "production"  
} satisfies Config;
```

This gives you validation while preserving useful literal/inferred information.

61. Conditional Types

Conditional types behave like type-level if.

```
type IsString<T> =
```

```
  T extends string ? true : false;
```

Example:

```
type A = IsString<string>; // true
```

```
type B = IsString<number>; // false
```

Useful for advanced reusable type utilities.

62. infer

infer allows TypeScript to infer part of a type inside a conditional type.

Example:

```
type ElementType<T> =
```

```
  T extends Array<infer U> ? U : T;
```

```
type A = ElementType<string[]>;
```

```
// string
```

63. Template Literal Types

Template literal types construct string types.

```
type EventName =
```

```
  `on${"Click" | "Change"}`;
```

Result:

```
"onClick" | "onChange"
```

Useful for strongly typed event names, routes, CSS-like tokens, etc.

64. Generic React Components

Example:

```
interface ListProps<T> {
```

```
  items: T[];
```

```
  renderItem: (item: T) => React.ReactNode;
```

```
}
```

```
function List<T>({
  items,
  renderItem
}: ListProps<T>) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>
          {renderItem(item)}
        </li>
      ))}
    </ul>
  );
}
```

Now the component works with different data types while preserving type safety.

65. Zustand Typing

A Zustand store can be typed with an interface.

```
interface CounterStore {
  count: number;
  increment: () => void;
}
```

Then:

```
const useCounter = create<CounterStore>((set) => ({
  count: 0,
  increment: () =>
    set(state => ({
      count: state.count + 1
    }))
}));
```

66. Redux Toolkit Typing

Typical pattern:

```
type RootState = ReturnType<typeof store.getState>;
```

```
type AppDispatch = typeof store.dispatch;
```

Then create typed hooks:

```
useSelector((state: RootState) => state.user);
```

and typed dispatch.

Redux Toolkit's createSlice also infers many types automatically.

67. HOCs and Render Props

HOC

A Higher-Order Component is a function that takes a component and returns an enhanced component.

```
const withAuth = (Component) => {  
  // ...  
};
```

Render prop

A component receives a function that determines what to render.

```
<DataProvider  
  render={({data}) => <List data={data} />}  
>
```

Modern React often prefers hooks and composition, but understanding these patterns is still useful in interviews.

68. Service Workers

A Service Worker is a background browser script that can intercept network requests.

Common uses:

- Offline support
- Caching
- PWA functionality
- Background processing

- Push notifications

It runs separately from the page's main JavaScript execution context.

69. Web Workers

Web Workers execute JavaScript in a background thread.

Useful for CPU-intensive tasks:

Large calculations

Parsing

Data processing

Image processing

They prevent heavy JavaScript from blocking the main UI thread.

70. Scaling WebSockets

Challenges include:

- Multiple server instances
- Connection state
- Load balancing
- Reconnection
- Authentication
- Message delivery
- Presence
- Horizontal scaling

A common architecture uses:

Client

↓

Load Balancer

↓

WebSocket servers

↕

Redis Pub/Sub

Redis can allow messages to propagate between different WebSocket server instances.

71. Intersection Observer

Intersection Observer detects when an element enters or leaves a viewport/root.

Useful for:

- Lazy loading
- Infinite scrolling
- Analytics
- Scroll-based UI
- Visibility detection

Example:

```
const observer = new IntersectionObserver(entries => {  
  // ...  
});
```

It avoids manually calculating positions on every scroll event.

72. ResizeObserver

ResizeObserver detects changes to an element's size.

```
const observer = new ResizeObserver(entries => {  
  // element size changed  
});
```

Useful for components that need to respond to their container size.

73. AbortController

Allows cancellation of abortable asynchronous operations.

Example:

```
const controller = new AbortController();
```

```
fetch("/api/users", {  
  signal: controller.signal  
});
```

```
controller.abort();
```

Useful for:

- Canceling fetches
 - Search autocomplete
 - Component cleanup
 - Avoiding unnecessary work
-

74. HSTS

HTTP Strict Transport Security tells browsers to use HTTPS for a domain.

Strict-Transport-Security:

max-age=31536000;

includeSubDomains

It helps prevent downgrade attacks and accidental HTTP access.

75. Mixed Content

Mixed content occurs when an HTTPS page loads insecure HTTP resources.

Example:

https://example.com

↓

http://example.com/image.jpg

Browsers block or restrict certain types of mixed content.

Best practice:

Serve all application resources over HTTPS.

🟡 Tier 3 — Final Revision

76. aria-atomic

Controls whether a live region announcement includes the entire region or only the changed portion.

```
<div aria-live="polite" aria-atomic="true">
```

Cart: 3 items

```
</div>
```

With true, the updated announcement can include the complete relevant region.

77. Progressive Enhancement

Build the basic functionality using standard web technologies first, then enhance it for capable browsers.

Example:

```
<form action="/checkout" method="POST">
```

The form works without JavaScript.

JavaScript can then add:

- Client-side validation
 - Loading states
 - Optimistic UI
 - Enhanced interactions
-

78. Decorative vs Informative Images

Informative

Needs meaningful alt.

```

```

Decorative

Doesn't provide meaningful information.

```

```

An empty alt tells screen readers to ignore the image.

79. Print CSS

Used to control how a page prints.

```
@media print {  
  nav,  
  .advertisement {  
    display: none;  
  }  
}
```

Useful for:

- Invoices
- Reports

- Receipts
 - Documentation
-

80. auto-fit vs auto-fill

Often used with CSS Grid:

grid-template-columns:

```
repeat(auto-fit, minmax(200px, 1fr));
```

auto-fit

Collapses empty tracks and allows existing items to expand.

auto-fill

Keeps available tracks even if they are empty.

Simple memory:

auto-fit → fit existing items

auto-fill → fill available tracks

81. align-items vs align-content

align-items

Aligns individual flex/grid items along the cross axis.

align-content

Controls the distribution of multiple lines/tracks within the container.

Important:

align-content generally matters when there is extra space and multiple flex lines or grid tracks.

82. will-change

Hints to the browser that an element is likely to change.

```
.element {  
  will-change: transform;  
}
```

It can help certain animations, but excessive use consumes resources.

Don't use it everywhere.

83. .d.ts

A .d.ts file contains TypeScript declarations without implementation.

Example:

```
declare module "some-library" {  
  export function hello(name: string): void;  
}
```

Common uses:

- Describing JavaScript libraries
 - Global declarations
 - Module declarations
 - Type definitions
-

84. Declaration Merging

TypeScript can combine multiple declarations with the same name.

```
interface User {  
  name: string;  
}
```

```
interface User {  
  age: number;  
}
```

The resulting interface is effectively:

```
interface User {  
  name: string;  
  age: number;  
}
```

85. Module Augmentation

Module augmentation allows you to add declarations to an existing module.

Common use:

```
declare module "some-library" {  
  interface Config {
```

```
    customOption: boolean;  
  }  
}
```

Useful when extending third-party library types.

86. moduleResolution: "bundler"

This tells TypeScript to resolve modules in a way aligned with modern bundlers.

It is useful with tools such as:

Vite

Webpack

esbuild

Rollup

It supports modern package resolution behavior and is commonly paired with modern module configurations.

87. isolatedModules

Ensures each file can be safely transformed independently.

This is important for transpilers that process files individually rather than performing the entire TypeScript type-checking process.

It can reveal patterns that aren't safe for isolated transpilation.

88. GraphQL vs REST

REST

GET /users

GET /users/123

GET /users/123/orders

Multiple endpoints.

GraphQL

Usually one endpoint:

POST /graphql

Client specifies the data it needs.

GraphQL advantages:

- Flexible queries

- Avoids some over-fetching
- Strong schema
- Useful for complex client data requirements

REST advantages:

- Simple
 - HTTP-native
 - Easy caching
 - Easy monitoring/debugging
 - Excellent for resource-oriented APIs
-

89. GraphQL N+1 Problem

Suppose:

```
users {  
  posts  
}
```

You fetch 100 users.

Naively:

1 query → users

100 queries → posts

Total:

101 queries

That's the N+1 problem.

Common solution:

DataLoader batches and caches related requests.

Instead of:

100 individual queries

you can perform:

1 batched query

90. Shared Workers

A Shared Worker can be accessed by multiple browser contexts, such as tabs/windows from the same origin.

Useful for sharing:

- State
- Connections
- Coordination
- Background work

Unlike a normal Dedicated Worker, multiple browsing contexts can connect to the same Shared Worker.

91. Polling vs Long Polling

Polling

Client repeatedly asks:

GET /updates

wait

GET /updates

wait

GET /updates

Long polling

Client sends a request and server holds it until new data is available or a timeout occurs.

Then the client immediately opens another request.

Long polling reduces unnecessary empty responses compared with frequent polling, but WebSockets/SSE are often better for true real-time use cases.

92. IndexedDB

IndexedDB is a browser database for storing larger amounts of structured data.

Supports:

- Objects
- Indexes
- Transactions
- Asynchronous operations

Useful for:

- Offline applications
- Caching large datasets

- PWA data
 - Client-side databases
-

93. structuredClone()

structuredClone() creates a deep clone using the structured clone algorithm.

```
const copy = structuredClone(original);
```

Unlike:

```
JSON.parse(JSON.stringify(obj))
```

it supports more data types and avoids many JSON serialization limitations.

It still cannot clone every JavaScript value, such as functions.

94. BroadcastChannel

Allows communication between browsing contexts of the same origin.

```
const channel = new BroadcastChannel("app");
```

```
channel.postMessage({  
  type: "LOGOUT"  
});
```

Another tab can receive:

```
channel.onmessage = event => {  
  console.log(event.data);  
};
```

Useful for:

- Cross-tab logout
 - Synchronizing state
 - Notifications
 - Coordinating tabs
-

95. HTTPS Certificate Validation

When connecting through HTTPS, the browser validates the server's certificate chain.

Simplified:

Server certificate



Trusted CA chain



Certificate validity



Hostname matches



Certificate not expired/revoked according to applicable checks



Secure TLS connection

The certificate helps establish that the browser is communicating with the intended domain and enables the cryptographic handshake.

Q96. What is XSS?

Answer:

XSS (Cross-Site Scripting) is an attack where an attacker injects malicious content that executes in another user's browser.

Types:

- **Stored XSS** → Stored in database/server.
- **Reflected XSS** → Reflected immediately in the response.
- **DOM XSS** → Caused by unsafe client-side DOM manipulation.

Remember: Stored → Reflected → DOM

Q97. How do you prevent XSS?

Answer:

- Use safe output/encoding.
- Prefer textContent over innerHTML.
- React JSX escapes values by default.
- Sanitize HTML when required.
- Use CSP.
- Use secure cookies.
- Validate input on the server.

Interview line:

"I prevent XSS using safe output handling, sanitization when necessary, CSP, secure cookies, and server-side validation."

Q98. Is React completely safe from XSS?

Answer:

No. React provides strong default XSS protection, but developers can bypass it.

Common risks:

- dangerouslySetInnerHTML
- Unsafe URLs
- javascript: URLs
- eval()
- Unsanitized HTML

Interview line:

"React provides strong default XSS protection, but unsafe APIs and URL handling can still introduce XSS."

Q99. What is CSRF?

Answer:

CSRF (Cross-Site Request Forgery) tricks an authenticated user's browser into performing an unwanted action.

Example:

User logged into bank

↓

Visits malicious website

↓

Malicious site sends request

↓

Bank cookie is automatically sent

↓

Unwanted action occurs

Q100. How do you prevent CSRF?

Answer:

- Use SameSite cookies.

- Use CSRF tokens.
- Validate Origin/Referer.
- Use Authorization headers where appropriate.

Remember:

Cookie authentication → CSRF protection is important.

Q101. Does CSRF affect Bearer-token APIs?

Answer:

Usually, classic CSRF does not apply when authentication uses:

Authorization: Bearer <token>

because browsers don't automatically attach this header to cross-site requests.

However, **XSS can still steal tokens accessible to JavaScript.**

Q102. What is clickjacking?

Answer:

Clickjacking tricks users into clicking something different from what they see.

Usually, the attacker embeds the victim website inside an iframe.

Attacker page

↓

Invisible iframe

↓

Victim website

↓

User clicks

↓

Victim action occurs

Q103. How do you prevent clickjacking?

Answer:

Use:

Content-Security-Policy: frame-ancestors 'self'

For legacy support:

X-Frame-Options: DENY

Remember:

Modern → frame-ancestors

Legacy → X-Frame-Options

Q104. Can SQL injection happen from the frontend?**Answer:**

The actual SQL injection vulnerability exists in the **backend/database layer**.

Frontend validation can be bypassed.

The backend should use:

- Parameterized queries
- Prepared statements
- Safe ORM APIs
- Server-side validation

Interview line:

"Frontend validation is not a security boundary. SQL injection must be prevented on the backend."

Q105. What is NoSQL injection?**Answer:**

NoSQL injection occurs when attacker-controlled input modifies a NoSQL query.

Prevention:

- Validate input.
 - Use schemas.
 - Allow only expected fields.
 - Don't accept arbitrary query objects.
 - Restrict allowed operators.
-

Authentication & Authorization**Q106. What is the difference between authentication and authorization?****Answer:**

Authentication = Who are you?

Authorization = What can you do?

Example:

Login → Authentication

↓

User identified

↓

/admin access → Authorization

Easy memory trick:

Authentication → Identity

Authorization → Permissions

Q107. JWT vs Session authentication?

Answer:

Session	JWT
Server stores session	Token contains claims
Easier to revoke	Revocation is harder
Stateful	Usually stateless
Often uses cookies	Can use headers/cookies

Interview line:

"Sessions are easier to revoke, while JWTs can be verified without a server-side session lookup."

Q108. Where should JWTs be stored in a browser?

Answer:

Avoid storing long-lived authentication tokens in localStorage.

A common architecture is:

Access token

↓

Memory

Refresh token

↓

HttpOnly + Secure cookie

Q109. Why is localStorage risky for authentication tokens?

Answer:

JavaScript can read localStorage.

```
localStorage.getItem("token");
```

If XSS occurs, an attacker may steal the token.

An HttpOnly cookie cannot be directly read by JavaScript.

Q110. What is refresh token rotation?

Answer:

Every time a refresh token is used, the old token is invalidated and a new one is issued.

Refresh Token A

↓

Refresh

↓

Refresh Token B

↓

Token A invalid

It helps limit token replay and theft.

Q111. What happens with refresh token rotation across multiple tabs?

Answer:

Multiple tabs may try to refresh simultaneously, causing a race condition.

Possible solutions:

- BroadcastChannel
- Request locking
- Shared refresh state
- Single-tab refresh coordination

Interview line:

"I would coordinate refresh requests so only one tab refreshes the token and shares the result with other tabs."

Q112. What is OAuth 2.0?

Answer:

OAuth 2.0 is an **authorization framework** that allows an application to access resources on behalf of a user without knowing their password.

Remember:

OAuth → Authorization

OIDC → Authentication / Identity

Q113. What is PKCE?**Answer:**

PKCE stands for **Proof Key for Code Exchange**.

It protects the OAuth Authorization Code flow.

code_verifier

↓

code_challenge

↓

Authorization request

Later:

code_verifier

↓

Token request

The authorization server verifies the verifier.

Q114. Why is PKCE important for SPAs?**Answer:**

SPAs are public clients and cannot safely hide a client secret.

PKCE protects the authorization code without requiring a client secret.

Remember:

SPA → Authorization Code + PKCE

Q115. Why shouldn't SPAs use the OAuth implicit flow?**Answer:**

The implicit flow exposes access tokens directly through the browser redirect.

Modern applications prefer:

Authorization Code + PKCE

because it provides a safer token exchange.

Input Validation

Q116. What is the difference between validation and sanitization?

Answer:

Validation: Is the input allowed?

Sanitization: Can the input be made safe?

Example:

Validation → Age must be 18+

Sanitization → Remove dangerous HTML

Q117. Client-side vs server-side validation?

Answer:

Client-side validation = UX

Server-side validation = Security

Frontend validation can be bypassed.

Therefore, **always validate on the backend.**

Q118. How would you validate data in React?

Answer:

I can use a schema library such as **Zod**.

```
const schema = z.object({  
  email: z.string().email(),  
  password: z.string().min(8)  
});
```

Use validation on both sides:

Frontend → UX

Backend → Security

Q119. What is contextual output encoding?

Answer:

Encoding must match where the data is used.

HTML → HTML encoding

URL → URL encoding

JavaScript → JS-safe handling

CSS → CSS-safe handling

There is **no single encoding method for every context**.

Security Headers & Infrastructure

Q120. What is CSP?

Answer:

CSP stands for **Content Security Policy**.

It controls which resources and scripts can execute in a webpage.

Example:

Content-Security-Policy:

default-src 'self';

script-src 'self' 'nonce-xyz';

It provides an additional layer of XSS protection.

Q121. What is a CSP nonce?

Answer:

A nonce is a random value generated for a response.

script-src 'self' 'nonce-abc123'

<script nonce="abc123">

Only scripts with the correct nonce are allowed.

Q122. Why is unsafe-inline dangerous?

Answer:

script-src 'unsafe-inline'

allows inline scripts.

This weakens CSP because injected inline scripts may also execute.

Prefer:

- Nonces
 - Hashes
-

Q123. What is CORS?

Answer:

CORS stands for **Cross-Origin Resource Sharing**.

It controls which browser origins can access resources from another origin.

Example:

Access-Control-Allow-Origin: https://app.example.com

Important:

CORS is a browser security mechanism, not authentication.

Q124. What is a CORS preflight request?

Answer:

For certain cross-origin requests, the browser first sends an OPTIONS request.

OPTIONS /api/users

The server tells the browser which:

- Origins
- Methods
- Headers

are allowed.

Q125. Why can't * be used with credentials in CORS?

Answer:

Credentialed requests require a specific origin.

 Invalid:

Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true

 Correct:

Access-Control-Allow-Origin: https://app.example.com

Access-Control-Allow-Credentials: true

Q126. Why does CORS affect browsers but not Postman?

Answer:

CORS is enforced by browsers.

Browser → Enforces CORS

Postman → Doesn't enforce browser CORS

curl → Doesn't enforce browser CORS

The backend still needs authentication and authorization.

HSTS

Q127. What is HSTS?

Answer:

HSTS stands for **HTTP Strict Transport Security**.

It tells browsers to use HTTPS only.

Strict-Transport-Security: max-age=31536000; includeSubDomains

Q128. Why isn't an HTTP → HTTPS redirect enough?

Answer:

The first HTTP request can still happen over an unencrypted connection.

HSTS allows the browser to upgrade to HTTPS **before sending the HTTP request**.

Q129. What is HSTS preload?

Answer:

HSTS preload allows supported browsers to know that a domain should use HTTPS even on the **first visit**.

SRI

Q130. What is SRI?

Answer:

SRI stands for **Subresource Integrity**.

It verifies that a third-party CDN resource hasn't been modified.

```
<script  
  src="https://cdn.example.com/app.js"  
  integrity="sha384-..."
```

```
>
```

```
</script>
```

If the hash doesn't match, the browser blocks the resource.

Q131. When should you use SRI?

Answer:

Use SRI mainly for static third-party resources such as:

- CDN JavaScript
- CDN CSS

It helps detect unexpected modifications.

Security Headers**Q132. What security headers should a frontend application use?****Answer:**

Important headers include:

Content-Security-Policy

Strict-Transport-Security

X-Content-Type-Options

X-Frame-Options

Referrer-Policy

Permissions-Policy

Cross-Origin-Opener-Policy

The exact configuration depends on the application.

Q133. What does X-Content-Type-Options: nosniff do?**Answer:**

It tells the browser not to guess the MIME type.

X-Content-Type-Options: nosniff

This helps prevent incorrect interpretation of content as executable code.

Q134. What is Referrer-Policy?**Answer:**

It controls how much referrer information the browser sends to other sites.

A common value is:

Referrer-Policy: strict-origin-when-cross-origin

It helps reduce URL information leakage.

Q135. What is Permissions-Policy?

Answer:

Permissions-Policy controls access to browser features such as:

- Camera
- Microphone
- Geolocation

Example:

Permissions-Policy: camera=(), microphone=(), geolocation=()

Supply Chain Security**Q136. What is a supply-chain attack?****Answer:**

A supply-chain attack occurs when an attacker compromises a dependency used by your application.

Application

↓

npm package

↓

Compromised dependency

↓

Malicious code

Q137. How do you secure npm dependencies?**Answer:**

- Commit lockfiles.
 - Run npm audit.
 - Keep dependencies updated.
 - Review dependency changes.
 - Minimize dependencies.
 - Use Dependabot/Renovate.
 - Use overrides when necessary.
-

Q138. Why should lockfiles be committed?

Answer:

Lockfiles record the resolved dependency versions.

They provide:

- Reproducible builds
 - Consistent installations
 - Better dependency control
-

Q139. What is npm audit?**Answer:**

npm audit checks dependencies for known security vulnerabilities.

npm audit

It identifies vulnerable packages and can suggest remediation.

Security Audit**Q140. What is a frontend security audit?****Answer:**

It is a systematic review of an application for security issues.

I would check:

Authentication

Authorization

XSS

CSRF

CORS

Security headers

Input validation

Dependencies

Secrets

Third-party scripts

Q141. What security checks would you perform before releasing a React/Next.js application?**Answer:**

My checklist:

HTTPS

↓

HSTS

↓

Security headers

↓

Authentication

↓

Secure cookies

↓

XSS protection

↓

CSRF protection

↓

Input validation

↓

CORS

↓

Dependency audit

↓

Secret scanning

↓

Security testing

↓

Monitoring

Interview line:

"Before release, I would verify authentication, cookies, XSS, CSRF, CORS, security headers, input validation, dependencies, secrets, and automated security tests."

Production Incident

Q142. How would you handle an XSS vulnerability in production?

Answer:

I would:

1. **Assess** the impact.
2. **Patch** the vulnerability.
3. **Rotate/revoke** compromised tokens or secrets.
4. **Check logs** for exploitation.
5. **Notify users** if required.
6. Add **tests and security controls** to prevent recurrence.

Q143. What is Client-Side Rendering (CSR)?

Answer:

Client-Side Rendering means the server initially sends a minimal HTML shell, and JavaScript running in the browser builds the UI and fetches the required data.

Typical flow:

Browser



HTML shell



Download JavaScript



Execute React



Fetch API data



Render UI



Interactive page

For example:

```
<div id="root"></div>
```

```
<script src="/bundle.js"></script>
```

React eventually renders the application inside `#root`.

Advantages:

- Excellent for highly interactive applications
- Good fit for SPAs

- Fast navigation after the initial load
- Less server-side rendering work

Disadvantages:

- Initial rendering can be slow
- Large JavaScript bundles can hurt performance
- SEO can be more difficult
- Users may initially see a loading state

Interview line:

CSR means the browser is primarily responsible for rendering the application's UI after the JavaScript loads.

Q144. What are the trade-offs of CSR?

Answer:

The biggest trade-off is between **client-side interactivity and initial-load performance**.

Advantage	Disadvantage
Highly interactive	Large JS bundles
Great for SPAs	Slower initial rendering
Less server rendering	SEO can be harder
Excellent for dashboards	More client CPU usage
Simple navigation model	Potential layout shifts

Good use cases:

- Admin dashboards
- Internal applications
- Authenticated applications
- Highly interactive tools

Poor use cases:

- SEO-heavy product pages
- Blogs
- Marketing pages

- Content-heavy public websites
-

Q145. What happens when a user opens a CSR React application?

Answer:

A simplified flow is:

1. Browser requests HTML
↓
2. Server returns HTML shell
↓
3. Browser downloads JavaScript
↓
4. JavaScript is parsed and executed
↓
5. React mounts
↓
6. API requests are made
↓
7. Data is received
↓
8. UI is rendered
↓
9. Application becomes interactive

The initial HTML may contain very little actual application content.

Q146. Why can CSR have poor FCP and LCP?

Answer:

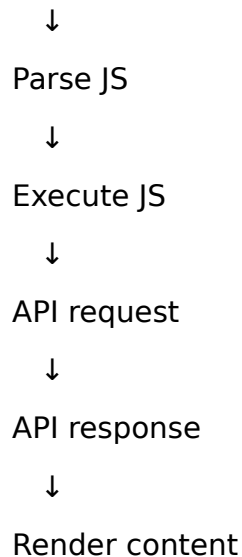
Because meaningful content may depend on JavaScript execution and API requests.

For example:

HTML arrives

↓

Download JS



The user has to wait through several stages before seeing the main content.

Therefore, CSR can negatively affect **FCP** and **LCP**, especially on slow networks or devices.

Q147. How would you improve CSR performance without moving to SSR?

Answer:

I would optimize the client-side application using:

1. Code splitting
2. Lazy loading
3. Route-level chunking
4. Tree shaking
5. Removing unnecessary dependencies
6. Preloading critical resources
7. Caching API responses
8. Starting data fetching early
9. Skeleton screens
10. Service worker caching where appropriate

For example:

```
const Reports = lazy(() => import('./Reports'));
```

This prevents the Reports component from being included in the initial JavaScript chunk.

● Server-Side Rendering

Q148. What is Server-Side Rendering (SSR)?

Answer:

SSR means the server generates HTML for a request and sends that HTML to the browser.

Flow:

Browser



GET /products



Server



Fetch data



Render HTML



Browser displays HTML



JavaScript loads



Hydration



Interactive page

The important benefit is that the browser can display meaningful content before the client-side JavaScript finishes executing.

Q149. What are the advantages of SSR?

Answer:

The major advantages are:

- Better initial content visibility
- Better SEO potential
- Better FCP/LCP in many cases
- Useful for dynamic content

- Good for personalized pages
- HTML can be rendered before hydration

Typical use cases include:

- E-commerce
 - News websites
 - Search results
 - Product pages
 - Personalized pages
-

Q150. What are the disadvantages of SSR?

Answer:

SSR adds server-side work for requests.

Potential disadvantages include:

- Higher server compute requirements
- Increased server response complexity
- TTFB can become high if data fetching is slow
- Requires caching strategies at scale
- Hydration still needs to happen for interactive components

A slow SSR request can look like:

Request

↓

Database query

↓

API call

↓

React rendering

↓

HTML response

If the database or API is slow, the response can also become slow.

Q151. What is hydration?

Answer:

Hydration is the process where React takes server-rendered HTML and makes it interactive on the client.

Suppose the server sends:

```
<button>Buy Now</button>
```

The browser can display it immediately.

After JavaScript loads, React hydrates the existing DOM and connects the component's behavior:

```
<button onClick={handleBuy}>
```

```
  Buy Now
```

```
</button>
```

So:

SSR provides the initial HTML, while hydration makes that HTML interactive.

Q152. Does hydration re-render the entire page?

Answer:

Not in the simple sense of throwing away the server HTML and starting from an empty DOM.

React uses the existing server-rendered markup and performs hydration so that the client-side React tree corresponds to the existing DOM and interactive behavior can work.

The important interview point is:

Hydration connects React's client-side behavior to server-rendered HTML.

Q153. What is a hydration mismatch?

Answer:

A hydration mismatch occurs when the server-generated HTML doesn't match the HTML React expects during client hydration.

Example:

```
function Greeting() {  
  return (  
    <p>  
      Current time: {new Date().toLocaleString()}  
    </p>  
  );  
}
```

```
}
```

The server could render:

10:00:01

while the client renders:

10:00:03

React detects that the output differs.

Q154. What commonly causes hydration mismatches?

Answer:

Common causes include:

- `new Date()`
- `Math.random()`
- Browser-only APIs
- `window`
- `localStorage`
- Different locale formatting
- Server/client conditional rendering
- Browser extensions modifying the DOM

For example:

```
if (typeof window !== 'undefined') {  
  // client-specific logic  
}
```

If this changes the rendered markup between server and client, it can cause a mismatch.

Q155. How can you avoid hydration mismatches?

Answer:

Ensure the initial server and client output is deterministic.

For client-only values, you can update the UI after hydration:

```
function Greeting() {  
  const [time, setTime] = useState(null);
```

```
useEffect(() => {
  setTime(new Date().toLocaleString());
}, []);

return <p>{time ?? 'Loading...'}</p>;
}
```

The initial markup is consistent, and the browser-specific value is added after hydration.

● Static Site Generation

Q156. What is Static Site Generation (SSG)?

Answer:

SSG means pages are generated into HTML at **build time** rather than for every request.

Build time

API / Database



Generate HTML



Static files



CDN

Then:

User



CDN



Pre-generated HTML

No React page rendering is required for every request.

Q157. When would you use SSG?

Answer:

SSG is ideal when content:

- Changes infrequently
- Is the same for many users
- Needs excellent performance
- Benefits from CDN caching

Examples:

- Documentation
 - Blogs
 - Marketing pages
 - About pages
 - Static landing pages
-

Q158. What is the biggest limitation of SSG?

Answer:

The main limitation is **content freshness**.

If content changes after deployment, the generated HTML may remain outdated until the site is rebuilt or another regeneration mechanism is used.

For example:

Build:

Price = ₹999

Database:

Price = ₹799

The statically generated page could still display ₹999 until regeneration.

This is one reason ISR exists.

Comparing Rendering Strategies

Q159. What is the difference between CSR, SSR and SSG?

Answer:

Strategy	Rendering time	Typical use
CSR	Browser	Interactive

Strategy	Rendering time	Typical use applications
SSR	Request time	Dynamic pages
SSG	Build time	Static content

Easy memory trick:

CSR → Browser

SSR → Server per request

SSG → Build time

Q160. Which is faster: SSR or SSG?

Answer:

Generally, SSG can be faster because the HTML is already generated and can be served directly from a CDN.

SSG:

User

↓

CDN

↓

HTML

SSR:

User

↓

Server

↓

Fetch data

↓

Render

↓

HTML

However, SSG sacrifices some freshness and personalization.

● Incremental Static Regeneration

Q161. What is ISR?

Answer:

ISR stands for **Incremental Static Regeneration**.

It allows statically generated pages to be updated without rebuilding the entire application.

It combines:

SSG

+

Regeneration

For example:

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
});
```

The cached result can be revalidated according to the framework's caching semantics.

Q162. How does ISR work?

Answer:

Simplified:

Page generated

↓

Cached

↓

Users receive cached page

↓

Revalidation becomes due

↓

Cached/stale page may be served

↓

Fresh version is generated

↓

Cache is updated

The major benefit is that users don't necessarily have to wait for regeneration before receiving a response.

Q163. What is stale-while-revalidate?

Answer:

Stale-while-revalidate means:

Serve the existing cached response immediately while a fresh version is generated in the background.

For example:

Cached page = ₹999

New price = ₹799

Request arrives

↓

Serve ₹999

↓

Regenerate page

↓

Cache ₹799

↓

Next request gets ₹799

This prioritizes fast responses while allowing the cache to become fresh.

Q164. What is on-demand ISR?

Answer:

On-demand revalidation means regenerating content when the underlying data actually changes rather than waiting for a timer.

For example, a CMS can trigger a webhook when a product changes.

CMS

↓

Webhook

↓

Revalidate product page

↓

Fresh page generated

This is useful when content changes are event-driven.

Q165. What is the difference between SSG and ISR?

Answer:

SSG:

Build

↓

Generate page

↓

Serve page

The page generally remains unchanged until another build/regeneration occurs.

ISR:

Generate page

↓

Serve cached page

↓

Revalidate

↓

Generate updated page

Therefore:

SSG is ideal for stable content, while ISR is useful when static content needs periodic or event-driven updates.

Q166. What is the difference between ISR and SSR?

Answer:

ISR

Uses cached/static output

SSR

Generates response per request

ISR

Can serve stale content

Excellent for repeated content

Lower server workload

Great for product/catalog pages

Use ISR when a small amount of staleness is acceptable.

Use SSR when freshness or personalization is required for each request.

SSR

Usually generates current response

Good for request-specific content

Higher server workload

Great for personalized pages

● Streaming SSR

Q167. What is Streaming SSR?

Answer:

Streaming SSR allows the server to send HTML progressively instead of waiting for the entire page to finish rendering.

Traditional SSR:

Fetch everything

↓

Render everything

↓

Send complete HTML

Streaming SSR:

Render shell

↓

Send shell

↓

Slow data resolves

↓

Send section

↓

Another section resolves

↓

Send section

This allows users to see useful content sooner.

Q168. How does Suspense help Streaming SSR?

Answer:

Suspense creates boundaries around parts of the UI that may take longer to render.

```
<Suspense fallback={<ProductSkeleton />}>  
  <ProductDetails />  
</Suspense>
```

```
<Suspense fallback={<ReviewsSkeleton />}>  
  <Reviews />  
</Suspense>
```

The server can send the shell and fallback UI first.

When each component becomes ready, its content can be streamed.

Q169. Why should you use multiple Suspense boundaries?

Answer:

Multiple boundaries allow independent sections to resolve independently.

Suppose:

Product API = 300ms

Reviews API = 800ms

With separate boundaries:

0ms

↓

Shell + skeletons

300ms

↓

Product details

800ms



Reviews

The slow reviews API doesn't have to block the product details.

Q170. What happens if ProductDetails takes 300ms and Reviews takes 800ms?

Answer:

The product details can become visible around 300ms while the reviews skeleton remains visible.

Then, when the reviews data is ready around 800ms, the reviews content can appear.

This is the primary UX advantage of streaming:

Slow parts don't necessarily block fast parts.

React Server Components

Q171. What are React Server Components?

Answer:

React Server Components are components that execute on the server and whose component implementation doesn't need to be shipped to the browser as client-side JavaScript.

In Next.js App Router, components are Server Components by default unless marked with:

`'use client';`

Server Components are useful for:

- Data fetching
 - Database access
 - Server-side processing
 - Rendering static content
 - Reducing client-side JavaScript
-

Q172. What are the benefits of React Server Components?

Answer:

They can:

- Fetch data directly on the server
- Access databases
- Access internal services
- Keep secrets on the server
- Reduce client JavaScript
- Keep heavy server-only dependencies out of the browser bundle

For example:

Server Component



Database



Render result



Browser

The database credentials remain server-side.

Q173. Can a Server Component use useState?

Answer:

No.

Server Components cannot use client-side state such as:

useState()

They also cannot directly use:

useEffect()

onClick

window

localStorage

navigator

If interaction is required, move that part into a Client Component.

Q174. What is a Client Component?

Answer:

A Client Component is a component that can use client-side React features such as:

- `useState`
- `useEffect`
- Event handlers
- Browser APIs
- Client-side subscriptions

In Next.js App Router, it is marked using:

`'use client';`

Example:

`'use client';`

```
import { useState } from 'react';
```

```
export default function Counter() {
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```
    <button onClick={() => setCount(count + 1)}>
```

```
      {count}
```

```
    </button>
```

```
  );
```

```
}
```

Q175. What is the difference between Server Components and Client Components?

Answer:

Server Component	Client Component
Server-side execution	Client-side execution
Default in App Router	Requires "use client"
Can access server resources	Can access browser APIs
No <code>useState</code>	Supports <code>useState</code>

Server Component

No useEffect

No event handlers

Doesn't require its implementation to ship as client JS

Client Component

Supports useEffect

Supports events

Client JavaScript is required

Interview line:

I start with Server Components by default and introduce Client Components only when the UI needs browser-side interactivity or state.

Q176. Why should "use client" be placed as deep as possible?

Answer:

Because "use client" creates a client-side boundary.

Suppose:

ProductPage

└─ ProductInfo

└─ ProductImages

└─ AddToCart

Only AddToCart needs interactivity.

It is better to have:

ProductPage → Server

ProductInfo → Server

ProductImages → Server

AddToCart → Client

rather than making the entire page a Client Component.

This can reduce the amount of JavaScript sent to the browser.

● Server/Client Data Boundary

Q177. Can Server Components pass data to Client Components?

Answer:

Yes.

Example:

```

async function ProductPage() {
  const product = await getProduct();

  return (
    <AddToCart
      productId={product.id}
      price={product.price}
    />
  );
}

```

The data crossing the boundary needs to be supported by the framework's serialization rules.

Typical safe values include:

string

number

boolean

arrays

plain objects

You should not treat arbitrary functions or server-only objects as ordinary serializable props.

Q178. Can a Server Component pass a function to a Client Component?

Answer:

Not as an ordinary serializable prop.

For example:

```
<AddToCart onAdd={someFunction} />
```

is not the normal way to pass a server-side function across the Server/Client Component boundary.

If client-side behavior is required, define the function inside the Client Component.

If you need server-side mutations, use the appropriate framework mechanism such as Server Actions where supported.

Progressive Hydration

Q179. What is Progressive Hydration?

Answer:

Progressive Hydration means making different parts of a server-rendered page interactive at different times or according to priority.

Instead of:

Hydrate everything

↓

Everything becomes interactive

you prioritize:

Critical components

↓

Hydrate first

Less important components

↓

Hydrate later

For example:

Add to Cart → Immediate

Reviews → When visible

Footer → When idle

FAQ → When interacted with

Q180. Why is Progressive Hydration useful?

Answer:

A large page can contain hundreds of components.

Hydrating everything immediately consumes:

- CPU
- Memory
- JavaScript execution time
- Main-thread time

This can delay important interactions.

Progressive approaches prioritize important UI, allowing critical interactions to become available sooner.

Q181. What is the hydration problem in traditional SSR?

Answer:

SSR can create a situation where:

HTML is visible

↓

But JavaScript hasn't hydrated yet

↓

User clicks button

↓

Interaction may not work yet

So the page **looks ready before it is fully interactive.**

This is one reason hydration performance matters.

🟡 Islands Architecture

Q182. What is Islands Architecture?

Answer:

Islands Architecture treats the page as mostly static HTML with small independent interactive components called **islands**.

For example:

Static HTML

└─ Article

└─ Images

└─ Author

|

└─ Interactive Island

| └─ Comments

|

└─ Interactive Island

| └─ Share buttons

|

└─ Footer

Only the interactive islands require client-side JavaScript.

Q183. Why does Islands Architecture improve performance?

Answer:

Because the page doesn't need to hydrate everything.

Traditional approach:

Entire page

↓

JavaScript

↓

Hydrate everything

Islands:

Static HTML

+

Small interactive components

This can reduce:

- JavaScript download
- JavaScript parsing
- JavaScript execution
- Hydration work

Q184. Which framework is strongly associated with Islands Architecture?

Answer:

Astro is one of the best-known examples.

For example:

```
<TableOfContents client:visible />
```

```
<Comments client:visible />
```

```
<ShareButtons client:idle />
```

Common Astro hydration directives include:

client:load

client:idle

client:visible

client:media

They control when the interactive component should load/hydrate.

🔥 Architecture & Scenario Questions

Q185. CSR vs SSR — which would you choose?

Answer:

I would choose based on:

- SEO requirements
- Initial-load performance
- Personalization
- Interactivity
- Data freshness
- Infrastructure requirements

For example:

Admin dashboard

→ CSR / Client Components

Product page

→ SSR / ISR

Marketing page

→ SSG

I wouldn't automatically choose SSR for every page.

Q186. SSR vs SSG — which would you choose?

Answer:

I would use **SSG** when content is mostly static and can be generated ahead of time.

I would use **SSR** when content needs to be generated based on the current request.

For example:

Documentation

→ SSG

Personalized dashboard

→ SSR

Q187. ISR vs SSR — when would you choose each?

Answer:

Use ISR when slightly stale content is acceptable:

Blog

Product catalog

Documentation

Use SSR when the response must be generated based on the current request:

Personalized dashboard

User account

Request-specific search

Q188. How would you choose a rendering strategy for an e-commerce website?

Answer:

I would use a hybrid approach.

Homepage

→ SSG / ISR

Product pages

→ ISR / SSR

Checkout

→ SSR + Client Components

Admin dashboard

→ Client Components

Reviews

→ Streaming SSR

Add to Cart

→ Client Component

The important point is:

Rendering strategy should be selected per route or component based on its requirements.

Q189. Can an application use CSR, SSR, SSG and ISR together?

Answer:

Yes.

A modern application can use multiple rendering strategies.

For example:

/about

→ SSG

/products/[slug]

→ ISR

/search

→ SSR

/admin

→ Client-side rendering

/cart-button

→ Client Component

This is commonly referred to as a **hybrid rendering architecture**.

Q190. What is the difference between RSC and Islands Architecture?

Answer:

Both reduce unnecessary client-side JavaScript, but they solve the problem differently.

RSC

- React-specific
- Server/Client boundary exists within the React component tree
- Server Components don't require their implementation to ship as client JS
- Client Components handle interactivity

Islands

- Architectural pattern
- Page is primarily static HTML
- Interactive components are isolated
- Islands can potentially use different UI technologies

Simple mental model:

RSC:

React application

└─ Server Components
└─ Client Components

Islands:

HTML page

└─ Static HTML
└─ React island
└─ Svelte island
└─ JS island

Scenario-Based Questions

Q191. A React application has poor FCP because of a large JavaScript bundle. What would you do?

Answer:

I would first optimize the existing CSR architecture.

I would consider:

1. Code splitting

2. Lazy loading
3. Route-based chunks
4. Tree shaking
5. Removing unnecessary dependencies
6. Optimizing images
7. Preloading critical resources
8. Starting API requests earlier
9. Caching
10. Skeleton screens

If the page is public and SEO-sensitive, I would also evaluate SSR, SSG, or ISR.

Q192. A product page needs SEO and its content changes every few hours. Which rendering strategy would you choose?

Answer:

I would strongly consider **ISR**.

The page can be statically served for fast delivery while being regenerated periodically or when the underlying product changes.

If the price must always be absolutely current for every request, I would consider SSR or separately fetching the critical dynamic value.

Q193. Would you use SSG for a personalized dashboard?

Answer:

Usually no.

SSG produces pre-generated content and isn't naturally suited for highly user-specific pages.

For a personalized dashboard, I would consider:

SSR

+

Client Components

or client-side data fetching depending on the requirements.

Q194. A blog has 5,000 articles, but only 10 articles change each day. What would you choose?

Answer:

I would consider **SSG combined with ISR** or another incremental/static caching strategy.

The goal is to avoid rebuilding all 5,000 pages every time a few articles change.

ISR allows individual pages to be regenerated independently.

Q195. A product API takes 300ms and the reviews API takes 800ms. How would Streaming SSR improve the page?

Answer:

I would use separate Suspense boundaries:

```
<Suspense fallback={<ProductSkeleton />}>
```

```
  <ProductDetails />
```

```
</Suspense>
```

```
<Suspense fallback={<ReviewsSkeleton />}>
```

```
  <Reviews />
```

```
</Suspense>
```

The result could be:

0ms

→ Page shell + skeletons

300ms

→ Product details

800ms

→ Reviews

The slow reviews request doesn't need to prevent the faster product information from becoming visible.

★ Rapid-Fire Questions

Q196. What is the easiest way to remember CSR, SSR and SSG?

Answer:

CSR → Browser renders

SSR → Server renders per request

SSG → Build process renders

ISR → Static page + regeneration

Q197. Which rendering strategy is best for SEO?

Answer:

There isn't one universal answer, but **SSR, SSG, and ISR** are generally better fits for SEO-sensitive pages because they can provide meaningful HTML without requiring the crawler to execute the entire client application first.

Q198. Which strategy is best for a highly interactive dashboard?

Answer:

Usually CSR or Client Components, because dashboards often require:

- Local state
- Event handlers
- Filters
- Charts
- Real-time updates
- Client-side interactions

SEO generally isn't a primary requirement for authenticated dashboards.

Q199. Which strategy gives the fastest static delivery?

Answer:

Generally **SSG**, because the HTML is already generated and can be served from a CDN.

ISR can provide similar delivery characteristics for cached pages.

Q200. What is hybrid rendering?

Answer:

Hybrid rendering means using different rendering strategies within the same application.

For example:

Marketing page → SSG

Product page → ISR

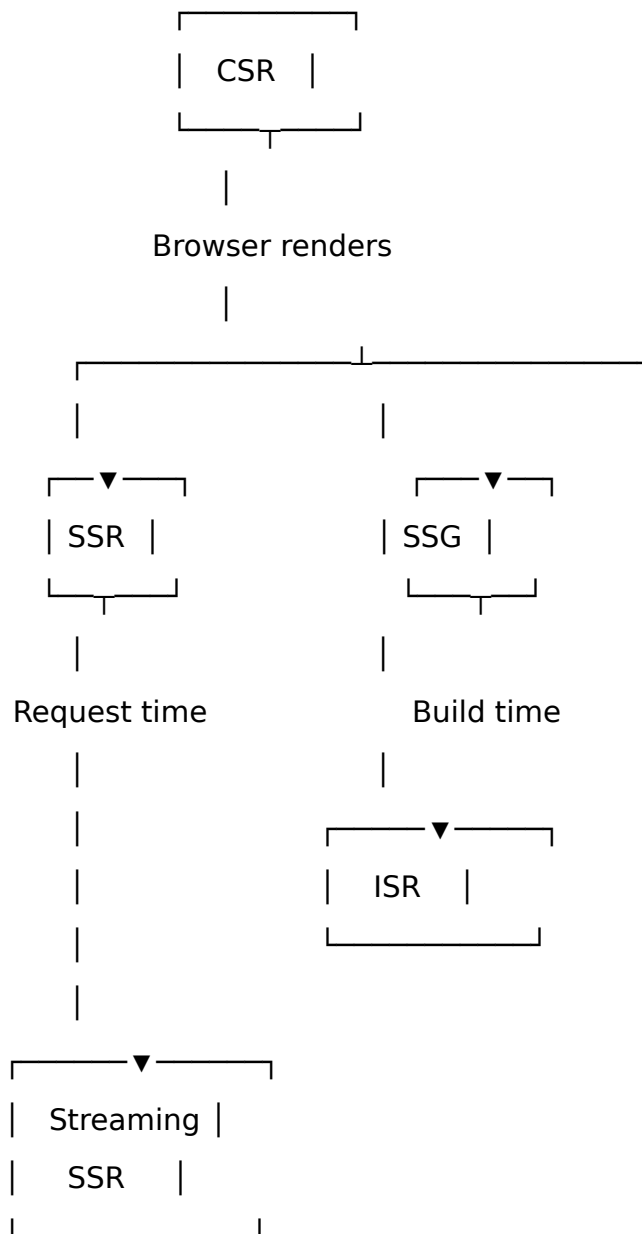
Search page → SSR

Dashboard → CSR

Interactive UI → Client Component

This is often the most practical approach for large applications.

RENDERING STRATEGIES



Q201. What happens from the moment a browser receives HTML until the page is displayed?

Answer:

The browser roughly follows this process:

HTML received



Parse HTML



Build DOM



Parse CSS



Build CSSOM



DOM + CSSOM



Render Tree



Layout



Paint



Composite



Pixels on screen

JavaScript can interrupt parsing and modify the DOM/CSSOM, which can affect rendering.

Interview line:

"The browser parses HTML into the DOM, CSS into the CSSOM, combines them into the render tree, calculates layout, paints pixels, and composites layers."

Q202. What is the difference between layout, paint, and composite?

Answer:

These are different stages of rendering.

Layout

Determines **where and how large** elements are.

width

height

position

margin

Paint

Draws visual properties.

color

background

border

shadow

text

Composite

Combines already-painted layers into the final image.

Properties such as:

transform

opacity

can often be handled efficiently during compositing.

Easy memory trick:

Layout → Where?

Paint → What does it look like?

Composite → How are layers combined?

Q203. What is a reflow?

Answer:

Reflow, commonly called **layout**, happens when the browser needs to recalculate the geometry of elements.

For example:

```
element.style.width = "500px";
```

Changing width can affect:

- Element size
- Child positions
- Sibling positions
- Parent layout

Therefore the browser may need to recalculate layout.

Interview line:

"Reflow is the recalculation of layout after changes that affect element geometry."

Q204. What is repaint?

Answer:

A repaint occurs when the browser needs to redraw visual properties without necessarily recalculating layout.

For example:

```
element.style.backgroundColor = "red";
```

The element's geometry doesn't change, but its pixels need to be redrawn.

Layout change

→ Layout + Paint

Visual-only change

→ Paint

Q205. What is layout thrashing?

Answer:

Layout thrashing happens when JavaScript repeatedly **writes to the DOM and then immediately reads layout information**, forcing the browser to perform synchronous layout calculations repeatedly.

Bad pattern:

```
for (const item of items) {  
  item.style.width = "100px";  
  
  console.log(item.offsetWidth);  
}
```

The browser may repeatedly perform layout calculations.

Better:

```
const widths = items.map(item => item.offsetWidth);
```

```
items.forEach(item => {  
  item.style.width = "100px";  
});
```

Interview line:

"I avoid layout thrashing by batching DOM reads separately from DOM writes."

Q206. Which JavaScript operations can force synchronous layout?

Answer:

Examples include reading properties such as:

offsetWidth

offsetHeight

clientWidth

clientHeight

scrollWidth

scrollHeight

getBoundingClientRect()

after DOM/style changes.

For example:

```
element.style.width = "500px";
```

```
const width = element.offsetWidth;
```

The browser may need to calculate layout before returning the value.

Q207. How would you reduce browser rendering work?

Answer:

I would consider:

- Reduce unnecessary DOM nodes
- Avoid layout thrashing
- Use CSS containment
- Use content-visibility
- Virtualize large lists
- Avoid expensive selectors

- Prefer transform/opacity for animations
- Reduce unnecessary React renders
- Split long JavaScript tasks
- Remove unnecessary CSS

The goal is to reduce:

DOM work

+

Style calculation

+

Layout

+

Paint

+

JavaScript

● Loading Performance

Q208. What is TTFB?

Answer:

TTFB means **Time to First Byte**.

It measures how long it takes from the request until the browser receives the first byte of the response.

Conceptually:

Request

↓

Server processing

↓

Network

↓

First byte received

High TTFB can be caused by:

- Slow server processing
- Database queries

- Cold starts
 - Network latency
 - Missing caching
 - Poor backend architecture
-

Q209. How would you improve TTFB?

Answer:

I would investigate the server and network path first.

Possible improvements:

- CDN
- HTTP caching
- Server-side caching
- Database optimization
- Reduce backend processing
- Optimize API calls
- Avoid unnecessary redirects
- Use geographically closer servers
- Improve server infrastructure
- Streaming where appropriate

Interview line:

"I would first determine whether TTFB is dominated by network latency, server processing, or database work before optimizing."

Q210. What is the difference between TTFB and LCP?

Answer:

They measure different things.

TTFB

→ How quickly does the server start responding?

LCP

→ How quickly does the main visible content appear?

For example:

TTFB = 300ms

LCP = 3.5s

The server responds quickly, but rendering or loading the LCP resource may still be slow.

Q211. What is First Contentful Paint (FCP)?

Answer:

FCP measures how long it takes for the browser to render the **first piece of DOM content**.

It could be:

- Text
- Image
- SVG
- Canvas

Example:

Navigation starts



HTML/CSS processed



First visible content



FCP

FCP tells us when users first see meaningful visual output, although it does not necessarily represent the main content.

Q212. FCP vs LCP?

Answer:

FCP

→ First visible content

LCP

→ Largest important visible content

Example:

Page starts



Small logo appears



FCP



Large hero image appears



LCP

Q213. What is the difference between preload and normal resource discovery?

Answer:

Normally, the browser discovers resources while parsing HTML and CSS.

For a critical resource, preload tells the browser:

"This resource will be needed soon. Start fetching it early."

Example:

```
<link
  rel="preload"
  href="/hero.webp"
  as="image"
/>
```

It can be useful when resource discovery would otherwise happen too late.

Q214. Can excessive preload hurt performance?

Answer:

Yes.

Preloading too many resources can compete for bandwidth and delay genuinely important resources.

For example, preloading:

10 images

5 fonts

3 JavaScript files

can create unnecessary competition.

Interview line:

"Preload should be reserved for genuinely critical resources; overusing it can make performance worse."

Q215. What is resource prioritization?

Answer:

Resource prioritization determines which resources the browser should fetch first.

For example:

Critical HTML

↓

Critical CSS

↓

LCP image

↓

Important JavaScript

↓

Below-the-fold images

↓

Non-critical resources

Hints such as:

`fetchpriority="high"`

`fetchpriority="low"`

can influence priority when appropriate.

● **JavaScript Performance**

Q216. Why does JavaScript have such a large impact on frontend performance?

Answer:

JavaScript consumes CPU and runs primarily on the main thread for UI work.

Heavy JavaScript can block:

User input

Rendering

Layout

Painting

Event handling

For example:

300ms JavaScript task



Browser cannot respond immediately



Interaction feels slow

Therefore reducing JavaScript is often more important than simply reducing file size.

Q217. Is a smaller JavaScript bundle always faster?

Answer:

No.

Bundle size is important, but execution cost also matters.

For example:

100 KB JS

+

300ms execution

can be worse than:

150 KB JS

+

50ms execution

We should consider:

- Transfer size
- Parse time
- Compilation
- Execution
- Number of modules
- Main-thread blocking
- Runtime behavior

Interview line:

"I optimize both JavaScript transfer size and JavaScript execution cost."

Q218. What is JavaScript parse and execution cost?**Answer:**

After JavaScript is downloaded, the browser still needs to:

Download

↓

Parse

↓

Compile

↓

Execute

Large JavaScript bundles can therefore hurt low-end devices even when network speed is good.

This is why code splitting and removing unnecessary JavaScript are important.

Q219. What is dynamic import?**Answer:**

Dynamic import loads a module asynchronously when it is needed.

```
const module = await import("./analytics");
```

Unlike a normal static import:

```
import analytics from "./analytics";
```

dynamic import can enable code splitting.

Example:

```
async function openEditor() {  
  const { Editor } = await import("./Editor");  
  // use Editor  
}
```

The editor code doesn't have to be included in the initial bundle.

Q220. When should you use dynamic imports?

Answer:

Use them when functionality is:

- Rarely used
- Expensive
- Route-specific
- User-triggered
- Below the fold
- Not required for initial rendering

Examples:

PDF viewer

Chart library

Rich text editor

Excel export

Admin dashboard

Image editor

React Performance

Q221. Why can unnecessary React re-renders hurt performance?

Answer:

A re-render causes React to evaluate the component tree and determine what needs to change.

With a large or expensive tree, unnecessary renders can consume CPU.

For example:

State update

↓

Parent renders

↓

Many children render

↓

Expensive calculations

↓

Slow UI

The solution isn't always React.memo; first identify **why the components are rendering**.

Q222. How do you identify unnecessary React re-renders?

Answer:

I would use:

- React DevTools Profiler
- React DevTools component inspection
- Browser Performance panel
- Component render tracking

I would investigate:

Which component rendered?

Why did it render?

How long did it take?

Did its props actually change?

Then optimize the actual bottleneck.

Q223. Why can excessive use of useMemo hurt performance?

Answer:

useMemo isn't free.

React must:

- Store the memoized value
- Track dependencies
- Compare dependencies
- Maintain additional complexity

For a cheap calculation:

const value = a + b;

adding useMemo may provide no benefit.

Interview line:

"I use useMemo when the computation or referential-stability benefit justifies the memoization cost."

Q224. Why can excessive use of useCallback hurt performance?

Answer:

useCallback also has overhead and adds dependency-management complexity.

This:

```
const handleClick = useCallback(() => {  
  console.log("clicked");  
}, []);
```

isn't automatically better than:

```
const handleClick = () => {  
  console.log("clicked");  
};
```

It becomes useful when function identity matters, such as passing a callback to a memoized child.

Q225. What is the difference between debouncing and throttling?**Answer:****Debouncing**

Wait until events stop.

Typing:

a → ab → abc → abcd

↓ wait 300ms

API call

Useful for:

- Search
- Autocomplete
- Validation

Throttling

Allow execution at most once within a specific interval.

scroll events

0ms → execute

50ms → ignore

100ms → execute

150ms → ignore

200ms → execute

Useful for:

- Scroll
- Mouse movement
- Resize
- Continuous events

Easy memory trick:

Debounce → after activity stops

Throttle → at a controlled frequency

Advanced Rendering

Q226. What is hydration?

Answer:

Hydration is the process where client-side JavaScript attaches behavior and event handlers to HTML that was already rendered on the server.

Conceptually:

Server

↓

HTML

↓

Browser displays HTML

↓

JavaScript loads

↓

React hydrates

↓

Interactive UI

This is common with frameworks such as Next.js.

Q227. What is hydration mismatch?

Answer:

A hydration mismatch occurs when the HTML generated on the server differs from what React expects on the client.

Example:

```
<div>
  {new Date().toLocaleTimeString()}
</div>
```

The server and browser can generate different values.

This can cause hydration warnings or incorrect UI.

Common causes:

- Random values
 - Current timestamps
 - Browser-only APIs
 - Different server/client data
 - Conditional rendering based on browser state
-

Q228. How do you prevent hydration mismatches?

Answer:

I would make server and client rendering deterministic.

Avoid directly using browser-specific values during initial render:

window

localStorage

document

when rendering server content.

For client-only behavior, use appropriate client-side effects or client-only boundaries.

For example:

```
useEffect(() => {
  setWidth(window.innerWidth);
}, []);
```

The key is:

Server-rendered output and initial client-rendered output should match.

Q229. What is streaming rendering?

Answer:

Streaming allows the server to send HTML progressively instead of waiting for the entire page to be generated.

Instead of:

Wait for everything



Send complete HTML

the server can send:

Shell



Header



Main content



Additional content

This can improve perceived performance and allow users to see useful content sooner.

Q230. What is progressive rendering?

Answer:

Progressive rendering means showing useful parts of the UI as they become available instead of waiting for everything.

Example:

Page shell



Header



Product information



Hero image



Reviews



Recommendations

This improves perceived performance.

● Senior-Level Scenario Questions

Q231. Your page has a 2 MB JavaScript bundle. How would you investigate it?

Answer:

I'd start with bundle analysis.

Build



Bundle analyzer



Largest dependencies



Duplicate modules



Unused code



Optimization

I would check:

- Large dependencies
- Duplicate packages
- Moment-like heavy libraries
- Unused exports
- Missing tree shaking
- Route-level splitting
- Lazy-loaded components
- Dynamically imported features

Then I'd measure again.

Q232. A developer says, "Let's add useMemo everywhere." How would you respond?

Answer:

I would avoid blanket memoization.

I'd first profile the application:

Identify bottleneck

↓

Measure render cost

↓

Determine why it re-renders

↓

Apply targeted optimization

useMemo is valuable when there is an expensive computation or referential equality requirement.

Senior-level answer:

"Performance optimization should be measurement-driven, not based on applying memoization everywhere."

Q233. Your API response takes 2 seconds, but the React component renders in 20ms. Where is the bottleneck?

Answer:

The bottleneck is likely **network/backend latency**, not React rendering.

I'd investigate:

Browser

↓

Request timing

↓

DNS

↓

Connection

↓

TTFB

↓

Server processing

↓

Database

I'd use the Network panel and backend observability to determine where the 2 seconds are spent.

Q234. Your API is fast, but the page is still slow. What could be wrong?

Answer:

The bottleneck may be client-side.

I'd investigate:

- JavaScript execution
- React rendering
- Large DOM
- Expensive calculations
- Image loading
- Font loading
- Layout/reflow
- Long tasks
- Third-party scripts
- Hydration
- Main-thread blocking

For example:

API = 100ms

JS execution = 800ms

React rendering = 500ms

The API isn't the problem.

Q235. A search page makes 20 API requests while the user types. How would you fix it?

Answer:

I'd combine:

Debouncing

+

AbortController

+

Caching/deduplication

For example:

User types

↓

Wait 300ms

↓

Cancel previous request

↓

Send latest request

This prevents unnecessary requests and avoids stale results.

Q236. A user clicks a button and the UI freezes for 800ms. How would you debug it?

Answer:

I'd inspect the interaction in Chrome DevTools Performance.

Click

↓

Event handler

↓

Long task?

↓

Expensive computation?

↓

React render?

↓

Layout?

↓

Paint?

Then I would:

- Break long tasks

- Move CPU-intensive work to a Web Worker
 - Optimize React rendering
 - Use useTransition where appropriate
 - Virtualize large lists
 - Avoid synchronous expensive work
-

Q237. How would you optimize a dashboard with many charts?

Answer:

I'd consider:

1. Lazy-load charts below the fold
2. Dynamically import the charting library
3. Avoid rendering invisible charts
4. Virtualize long lists
5. Reduce dataset size
6. Aggregate data server-side
7. Cache API responses
8. Memoize expensive calculations where justified
9. Use Web Workers for CPU-heavy processing
10. Render only the required resolution/data

The biggest goal is to avoid loading and rendering everything immediately.

Q238. How would you optimize a large table containing 100,000 records?

Answer:

I would not render all 100,000 rows into the DOM.

I'd use:

Server-side pagination/filtering

+

Virtualization

+

Debounced search

+

Efficient data fetching

For example:

100,000 records

↓

Server filtering

↓

Relevant records

↓

Virtualized visible rows

↓

~20-100 DOM rows

Q239. A page has good LCP but terrible INP. What does that tell you?

Answer:

It suggests the page **loads its important content quickly**, but interactions are slow.

I'd focus on:

- Long JavaScript tasks
- Expensive event handlers
- React rendering
- Large component trees
- Synchronous calculations
- Third-party scripts
- Main-thread blocking

In other words:

Loading performance → Good

Interaction performance → Bad

Q240. A page has good LCP and INP but bad CLS. What would you investigate?

Answer:

I'd focus specifically on unexpected layout shifts.

I'd investigate:

Images



Missing dimensions

Fonts



Layout changes

Ads



Unreserved space

Dynamic content



Inserted above existing content

Lazy components



Missing placeholders

The solution is usually to reserve the correct space before content arrives.

Final Senior Interview Questions

Q241. What is your overall frontend performance optimization strategy?

Answer:

My approach is:

Measure



Identify bottleneck



Prioritize by user impact



Optimize



Measure again



Monitor production

I divide performance into:

Loading

→ LCP, FCP, TTFB

Interaction

→ INP, long tasks

Visual stability

→ CLS

Network

→ API latency, caching

JavaScript

→ Bundle size, execution

Rendering

→ React renders, layout, paint

Production

→ RUM

Strong senior-level interview line:

"I don't optimize based on assumptions. I measure the bottleneck, make the smallest high-impact change, verify the improvement, and then monitor the result in production."

Q242. Which performance optimizations would you prioritize first?

Answer:

I'd prioritize based on the actual bottleneck, but generally:

1. Fix slow server/API response
2. Improve LCP
3. Remove unnecessary JavaScript
4. Reduce long main-thread tasks
5. Optimize critical images/fonts
6. Reduce unnecessary React rendering
7. Virtualize large lists
8. Optimize caching
9. Defer non-critical work
10. Optimize third-party scripts

I would **not blindly apply all optimizations**.

Q243. What is the most important principle of frontend performance optimization?

Answer:

The most important principle is:

Measure before optimizing.

A good process is:

Measure

↓

Find bottleneck

↓

Optimize bottleneck

↓

Measure again

For example, if a page is slow because of a 2-second API call, adding React.memo won't solve the real problem.

Q244. How would you explain frontend performance to a non-technical stakeholder?

Answer:

I'd explain it in terms of user experience:

"Frontend performance means how quickly users can see useful content, interact with the application, and complete their tasks without the interface feeling slow or unstable."

I'd track:

How quickly content appears

→ LCP

How quickly interactions respond

→ INP

Whether the page moves unexpectedly

→ CLS

This connects technical performance directly to user experience.

Q245. What performance metrics would you monitor for a production React application?

Answer:

I'd monitor:

Core Web Vitals

LCP

INP

CLS

Additional metrics

FCP

TTFB

Long Tasks

JavaScript errors

API latency

Resource timing

Memory usage

Bundle size

I'd segment the data by:

Device

Browser
Network
Geography
Route

Application version

And I'd pay particular attention to **p75**, rather than relying only on averages.

Q246. How do you prevent performance regressions in a team?

Answer:

I'd combine development-time and production controls:

Performance budgets

↓

CI checks

↓

Bundle analysis

↓

Lighthouse CI

↓

RUM

↓

Alerts

For example:

PR

↓

Bundle grows by 500 KB

↓

CI detects budget violation

↓

Developer investigates

This makes performance a continuous engineering concern rather than a one-time optimization task.

Q247. Give a real-world example of frontend performance optimization.

Answer:

A strong senior-level example would be:

"In a React application, I first used Chrome DevTools and React Profiler to identify unnecessary rendering and large JavaScript bundles. I then introduced route-level code splitting, lazy-loaded heavy components, optimized images, reduced unnecessary renders, and used caching for API responses. I measured the application again using Lighthouse and production metrics to verify the improvement."

If you have a quantified result, make it stronger:

"These changes reduced initial JavaScript and improved the application's loading and interaction performance by approximately 25%."

Q248. What is the difference between optimization and premature optimization?

Answer:

Optimization addresses a measured performance problem.

Premature optimization adds complexity before proving that a problem exists.

Example:

`useMemo(...)`

`useCallback(...)`

`React.memo(...)`

everywhere without profiling is potentially premature optimization.

Better:

Profile

↓

Find expensive component

↓

Optimize

↓

Measure

Interview line:

"I optimize based on evidence and user impact rather than optimizing code simply because a technique is considered a best practice."

Q249. If you had only 30 minutes to improve a slow production page, what would you do?

Answer:

I'd quickly identify the highest-impact bottleneck.

1. Check RUM/Core Web Vitals
2. Check Network panel
3. Identify LCP element
4. Check TTFB
5. Check JavaScript long tasks
6. Check large resources
7. Check third-party scripts
8. Apply the highest-impact fix
9. Measure again

For example:

LCP = 5s

↓

Hero image = 1.5 MB

↓

Optimize image + correct priority

↓

Measure LCP again

I would avoid spending the 30 minutes on micro-optimizations such as memoizing trivial calculations.

Q250. Explain your complete frontend performance mindset in one answer.

Answer:

"I treat performance as a measurement-driven engineering problem. I first measure real user experience using Core Web Vitals and RUM, then use browser and React profiling tools to identify the actual bottleneck. For loading performance I focus on TTFB, LCP, critical resources, images, fonts, caching, and JavaScript delivery. For runtime performance I focus on long tasks, React rendering, large DOM trees, virtualization, and moving CPU-heavy work off the main thread when necessary. Finally, I validate the optimization with measurements and prevent regressions through performance budgets and production monitoring."

🔥 Best one-line answer to memorize

"Measure → identify the bottleneck → optimize the highest-impact issue → measure again → monitor in production."

React-specific:

Server Components → server

Client Components → browser

Hydration → make SSR HTML interactive

Suspense → control async UI boundaries

Architecture:

Progressive Hydration → hydrate by priority

Islands Architecture → static HTML + small interactive islands

Hybrid Rendering → combine strategies per route/component

Q251. What is a design pattern?

Answer:

A design pattern is a **reusable solution to a commonly occurring software design problem**.

It is not a ready-made piece of code. Instead, it provides a proven way to structure code and responsibilities.

For example:

- Factory → object creation
- Strategy → interchangeable algorithms
- Observer → notification of changes
- Adapter → incompatible interfaces
- Facade → simplify complex systems

Interview answer:

A design pattern is a general, reusable approach for solving a recurring software design problem while improving maintainability, flexibility, and communication between developers.

Q252. Why are design patterns important in frontend development?

Answer:

Design patterns help frontend applications become:

- Easier to maintain
- Easier to test
- More reusable
- Less tightly coupled
- Easier to extend
- Easier for teams to understand

For example, instead of putting every payment implementation inside a large React component:

```
if (method === "card") {  
  // ...  
} else if (method === "upi") {  
  // ...  
} else if (method === "paypal") {  
  // ...  
}
```

we can use the **Strategy pattern**.

```
const paymentStrategies = {  
  card: cardPayment,  
  upi: upiPayment,  
  paypal: paypalPayment  
};
```

The component becomes simpler and adding new payment methods becomes easier.

Q253. Are design patterns specific to object-oriented programming?**Answer:**

No.

Traditional design patterns were often described using object-oriented programming, but JavaScript allows patterns to be implemented using:

- Functions
- Closures

- Objects
- Classes
- Modules
- Higher-order functions
- Composition
- Hooks

For example, the Strategy pattern can simply use functions:

```
const strategies = {  
  tax: calculateTax,  
  discount: calculateDiscount  
};
```

Interview point:

Design patterns describe design ideas, not necessarily a specific programming language or syntax.

Q254. What is the difference between a design pattern and an architecture?

Answer:

A **design pattern** solves a relatively focused design problem.

An **architecture** defines the overall structure of an application.

For example:

Architecture

↓

Clean Architecture

↓

Design patterns

↓

Factory / Strategy / Observer / Adapter

Examples of architecture:

- MVC
- Clean Architecture
- Hexagonal Architecture
- Microservices

- Layered Architecture

Examples of design patterns:

- Factory
- Observer
- Strategy
- Adapter
- Decorator

Easy distinction:

Architecture defines the overall structure; design patterns solve recurring design problems within that structure.

Q255. What is the difference between a pattern and an anti-pattern?

Answer:

A **design pattern** is a commonly useful solution to a recurring problem.

An **anti-pattern** is a commonly seen approach that creates more problems than it solves.

Example:

```
const globalState = {};
```

If everything in the application directly modifies this global object, it can create:

- Hidden dependencies
- Difficult debugging
- Difficult testing
- Tight coupling

This can become an anti-pattern.

Interview answer:

A pattern provides a proven design approach, while an anti-pattern is a recurring approach that tends to produce poor maintainability or other design problems.

Creational Patterns

Q256. What are creational design patterns?

Answer:

Creational patterns deal with **object creation**.

Common examples:

Factory

Abstract Factory

Builder

Prototype

Singleton

They help control how objects are created and reduce unnecessary coupling between code that uses an object and code that creates it.

Interview shortcut:

Creational patterns answer: "How should objects be created?"

Q257. When would you use a Factory instead of new?

Answer:

Use a Factory when the caller shouldn't need to know the exact class or construction logic.

Without Factory:

```
const payment = new UpiPayment();
```

With Factory:

```
const payment = createPayment("upi");
```

The Factory can decide:

```
function createPayment(type) {  
  switch (type) {  
    case "upi":  
      return new UpiPayment();  
  
    case "card":  
      return new CardPayment();  
  
    default:  
      throw new Error("Unsupported payment");  
  }  
}
```

Now object creation is centralized.

Q258. What is the benefit of a Factory in a large application?

Answer:

The main benefit is **decoupling**.

Instead of:

Component

↓

Specific class

we have:

Component

↓

Factory

↓

Specific implementation

This makes it easier to:

- Replace implementations
 - Add new implementations
 - Test the caller
 - Centralize creation logic
-

Q259. What is a simple factory?

Answer:

A Simple Factory is a function or object responsible for creating different types of objects.

Example:

```
function createLogger(type) {  
  if (type === "console") {  
    return new ConsoleLogger();  
  }  
  
  if (type === "file") {  
    return new FileLogger();  
  }  
}
```

```
    throw new Error("Unknown logger");  
}
```

Usage:

```
const logger = createLogger("console");
```

The caller doesn't directly construct the implementation.

Q260. What is the difference between Factory Method and Simple Factory?

Answer:

A **Simple Factory** usually uses a centralized function to decide which object to create.

Factory Method allows subclasses or implementations to determine which product should be created.

Simple Factory:

```
createPayment("upi");
```

Factory Method conceptually:

```
class PaymentProcessor {  
    createPayment() {}  
}
```

Different subclasses can override:

```
class UpiProcessor extends PaymentProcessor {  
    createPayment() {  
        return new UpiPayment();  
    }  
}
```

Interview shortcut:

Simple Factory centralizes creation; Factory Method delegates creation to subclasses or specialized implementations.

Q261. What problem does Builder solve?

Answer:

Builder solves the problem of constructing objects that have:

- Many optional properties
- Multiple configuration steps
- Complex construction logic

Instead of:

```
createReport(  
  title,  
  author,  
  sections,  
  charts,  
  filters,  
  format,  
  pagination  
);
```

we can use:

```
const report = new ReportBuilder()  
  .title("Sales Report")  
  .author("Priyanka")  
  .addChart(chart)  
  .addSection(section)  
  .format("pdf")  
  .build();
```

This improves readability.

Q262. Why does Builder commonly return this?

Answer:

Returning this enables **method chaining**.

```
class Builder {  
  setName(name) {  
    this.name = name;  
    return this;  
  }  
}
```



```
setAge(age) {  
  this.age = age;  
  return this;  
}  
}
```

Now:

```
new Builder()  
  .setName("Priyanka")  
  .setAge(25);
```

Each method returns the same builder instance.

Q263. When would Prototype be useful in JavaScript?

Answer:

Prototype can be useful when many objects should share behavior through a common prototype.

```
const userMethods = {  
  login() {  
    console.log("Login");  
  }  
};
```

```
const user = Object.create(userMethods);
```

The login function isn't copied directly onto every object.

It is looked up through the prototype chain.

This can reduce duplication and is fundamental to JavaScript's object model.

Q264. Is JavaScript class inheritance the same as Prototype inheritance?

Answer:

JavaScript classes are primarily **syntactic sugar over the prototype system**.

For example:

```
class User {  
  login() {}  
}
```

```
}
```

Methods are placed on:

User.prototype

An instance:

```
const user = new User();
```

can access login() through the prototype chain.

Interview answer:

JavaScript's class syntax provides an easier syntax for working with prototype-based inheritance; underneath, JavaScript still uses prototypes.

Q265. Why can Singleton be dangerous?

Answer:

Singleton introduces globally shared state.

Example:

```
Logger.getInstance().log("Hello");
```

Many parts of the application can now depend on the same mutable object.

Problems:

- Hidden dependencies
- Difficult unit testing
- Difficult mocking
- Global state
- Tight coupling

A better approach in many applications is **dependency injection**.

Structural Patterns

Q266. What are structural design patterns?

Answer:

Structural patterns deal with **how objects and components are composed**.

Common examples:

Adapter

Decorator

Facade

Proxy

Composite

Flyweight

They help build larger systems from smaller components.

Interview shortcut:

Structural patterns answer: "How should objects/components be connected?"

Q267. Give a real-world frontend example of Adapter.

Answer:

Suppose our application expects:

```
{
  id: "123",
  name: "Priyanka"
}
```

but an API returns:

```
{
  user_id: "123",
  full_name: "Priyanka"
}
```

We can adapt the response:

```
function adaptUser(apiUser) {
  return {
    id: apiUser.user_id,
    name: apiUser.full_name
  };
}
```

Now the rest of the application doesn't care about the external API format.

Q268. Why is Adapter useful when integrating third-party APIs?

Answer:

It prevents external API details from leaking throughout the application.

Without Adapter:

React Component

↓

Knows Google API format

↓

Knows GitHub API format

↓

Knows Internal API format

With Adapter:

Google API —┐

GitHub API —┴→ Adapter → AppUser

Internal API ┘

The application works with one consistent interface.

Q269. What is a Facade in a frontend application?

Answer:

A Facade provides a simple API over multiple complex operations.

For example:

```
checkoutService.checkout(cart);
```

Internally:

```
await inventory.reserve(cart);
```

```
await payment.charge(cart);
```

```
await order.create(cart);
```

```
await notification.send(cart);
```

The UI only interacts with:

```
checkoutService.checkout(cart);
```

This reduces complexity at the caller.

Q270. What is the difference between Facade and Adapter?

Answer:

This is very important.

Adapter:

Converts one interface into another.

Interface A



Adapter



Interface B

Facade:

Simplifies access to a complex subsystem.

Complex subsystem



Facade



Simple API

Memory trick:

Adapter = compatibility

Facade = simplicity

Q271. What is a Decorator in JavaScript?

Answer:

Decorator adds behavior around an existing function or object without modifying its original implementation.

Example:

```
function withLogging(fn) {  
  return (...args) => {  
    console.log("Calling function");  
  
    const result = fn(...args);  
  
    console.log("Completed");  
  
    return result;  
  };  
}
```

Usage:

```
const enhancedFetch = withLogging(fetchData);
```

The original fetchData remains unchanged.

Q272. What is the difference between Decorator and inheritance?

Answer:

Inheritance creates a relationship between classes.

Parent

↓

Child

Decorator adds behavior dynamically:

Original

↓

Decorator

↓

Enhanced object

Decorator generally favors **composition over inheritance**.

This can be more flexible because multiple decorators can be combined.

Q273. What is a Proxy in JavaScript?

Answer:

Proxy allows us to intercept operations performed on an object.

Example:

```
const user = {  
  name: "Priyanka"  
};
```

```
const proxy = new Proxy(user, {  
  get(target, property) {  
    console.log(`Reading ${property}`);  
    return target[property];  
  }  
});
```

Now:

proxy.name;

can be intercepted.

Common traps include:

get

set

has

deleteProperty

apply

construct

Q274. What is the difference between Proxy and Decorator?

Answer:

Decorator:

Focuses on **adding behavior**.

Function

↓

Logging decorator

↓

Enhanced function

Proxy:

Focuses on **intercepting access or operations**.

Caller

↓

Proxy

↓

Object

Interview shortcut:

Decorator enhances behavior; Proxy controls or intercepts access.

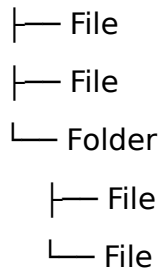
Q275. What is Composite pattern?

Answer:

Composite represents objects as a **tree structure** and allows individual objects and groups to be treated uniformly.

Example:

Folder



Both File and Folder can expose:

getSize();

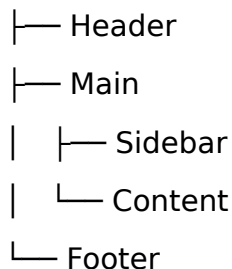
The client doesn't need to know whether it is dealing with a leaf or container.

Q276. How is Composite related to React?

Answer:

React applications naturally form a component tree.

App



Some components are leaf nodes.

Others contain children.

This is conceptually similar to Composite because the UI is represented as a hierarchical structure.

Important:

React isn't formally "the Composite pattern", but its component tree has strong conceptual similarities to Composite.

Q277. What problem does Flyweight solve?

Answer:

Flyweight reduces memory usage by **sharing common immutable or reusable data** between many objects.

For example:

10000 markers

Instead of each marker storing its own:

icon

color

font

style

we can share those resources and keep only unique information such as:

latitude

longitude

Q278. What is the difference between intrinsic and extrinsic state in Flyweight?

Answer:

Intrinsic state is shared and doesn't change between instances.

Example:

icon

color

font

Extrinsic state is unique to each object.

Example:

x

y

latitude

longitude

Flyweight stores intrinsic state centrally and supplies extrinsic state when needed.

● Behavioral Patterns

Q279. What are behavioral design patterns?

Answer:

Behavioral patterns focus on **communication and responsibility between objects**.

Examples:

Observer

Strategy

Command

State

Iterator

Mediator

Chain of Responsibility

Template Method

Interview shortcut:

Behavioral patterns answer: "How should objects communicate and behave?"

Q280. What is Observer pattern?

Answer:

Observer establishes a one-to-many relationship.

When the subject changes, registered observers are notified.

```
const store = {
```

```
  listeners: new Set(),
```

```
  subscribe(listener) {
```

```
    this.listeners.add(listener);
```

```
    return () => this.listeners.delete(listener);
```

```
  },
```

```
  notify(data) {
```

```
    this.listeners.forEach(listener => listener(data));
```

```
  }
```

```
};
```

Usage:

```
store.subscribe(data => {  
  console.log(data);  
});
```

Q281. Why should an Observer subscription return an unsubscribe function?

Answer:

To prevent:

- Memory leaks
- Duplicate subscriptions
- Stale listeners
- Unnecessary updates

Example:

```
const unsubscribe = store.subscribe(handleUpdate);
```

Later:

```
unsubscribe();
```

This removes the listener.

This idea is particularly important in frontend applications where components mount and unmount frequently.

Q282. How can the Observer pattern cause memory leaks?

Answer:

If an object subscribes to an event source but never unsubscribes:

```
store.subscribe(handleUpdate);
```

the store may continue holding a reference to handleUpdate.

Even after the UI component is removed, the listener can remain registered.

The solution is cleanup:

```
useEffect(() => {  
  const unsubscribe = store.subscribe(handleUpdate);  
  
  return unsubscribe;  
}, []);
```

Q283. What is Pub/Sub?

Answer:

Pub/Sub means **Publisher/Subscriber**.

The publisher sends messages to an intermediary such as an event bus.

Publisher



Event Bus



Subscribers

Example:

```
eventBus.publish("orderCreated", order);
```

Subscribers:

```
eventBus.subscribe(  
    "orderCreated",  
    handleOrderCreated  
);
```

The publisher doesn't directly know who receives the message.

Q284. What is the Mediator pattern?

Answer:

Mediator centralizes communication between multiple objects.

Instead of:

A ↔ B

A ↔ C

B ↔ C

we use:

A ⊣

B ⊣ → Mediator

C ⊣

This reduces direct coupling.

Example:

```
chatRoom.send(user, message);
```

The chat room acts as a mediator between users.

Q285. Mediator vs Pub/Sub — what is the difference?

Answer:

Both reduce direct communication, but their intent differs.

Mediator:

A central object coordinates interactions between participants.

Pub/Sub:

An event/message system distributes events between publishers and subscribers.

Mediator:

A → Mediator → B

Pub/Sub:

Publisher → Event Bus → Subscribers

Shortcut:

Mediator coordinates interactions; Pub/Sub distributes events.

Q286. What is Strategy pattern?

Answer:

Strategy allows different algorithms or behaviors to be selected interchangeably.

Example:

```
const strategies = {  
  price: products =>  
    [...products].sort((a, b) => a.price - b.price),  
  
  name: products =>  
    [...products].sort((a, b) =>  
      a.name.localeCompare(b.name)  
    )  
};
```

Then:

```
strategies.price(products);
```

or:

```
strategies.name(products);
```

The calling code doesn't need separate if/else branches.

Q287. What is the main advantage of Strategy over a large switch?

Answer:

Strategy separates each algorithm into its own implementation.

Instead of:

```
switch (type) {  
  case "card":  
    // lots of code  
    break;
```

```
  case "upi":  
    // lots of code  
    break;
```

```
}
```

we have:

```
const strategies = {  
  card: cardStrategy,  
  upi: upiStrategy  
};
```

Benefits:

- Easier testing
 - Easier extension
 - Smaller functions
 - Reduced conditional complexity
 - Better separation of concerns
-

Q288. What is Command pattern?

Answer:

Command encapsulates an operation as an object.

```
class DeleteCommand {  
    constructor(item) {  
        this.item = item;  
    }  
  
    execute() {  
        // delete item  
    }  
  
    undo() {  
        // restore item  
    }  
}
```

This allows commands to be:

- Stored
 - Queued
 - Logged
 - Replayed
 - Undone
 - Redone
-

Q289. How would you implement undo/redo using Command?

Answer:

Maintain two stacks:

History Stack

Redo Stack

When executing:

execute command

↓

history.push(command)

↓

clear redo stack

Undo:

history.pop()



command.undo()



redo.push(command)

Redo:

redo.pop()



command.execute()



history.push(command)

This is useful for editors, drawing applications, and complex form interactions.

Q290. What is State pattern?

Answer:

State pattern changes an object's behavior according to its current state.

For example:

Order



Pending



Approved



Shipped



Delivered

Each state defines valid operations.

This is useful when a large conditional structure becomes difficult to maintain.

Q291. Strategy vs State — explain with an example.

Answer:

Strategy is about choosing an algorithm.

Payment

└─ Card strategy

└─ UPI strategy

└─ PayPal strategy

The caller chooses the strategy.

State is about behavior changing based on current state.

Order

↓

Pending

↓ approve

Approved

↓ ship

Shipped

The current state determines valid behavior.

Memory trick:

Strategy = "Which algorithm?"

State = "What behavior is valid now?"

Q292. What is Chain of Responsibility?

Answer:

A request passes through a sequence of handlers.

Request

↓

Logger

↓

Authentication

↓

Validation

↓

Controller

Each handler can:

- Process the request
 - Modify it
 - Pass it forward
 - Stop processing
-

Q293. Why is Express middleware considered Chain of Responsibility?

Answer:

Express middleware forms a processing chain:

```
app.use(logger);
```

```
app.use(auth);
```

```
app.use(validate);
```

```
app.use(controller);
```

Each middleware can call:

```
next();
```

to continue.

Or stop the chain:

```
return res.status(401).json({  
  message: "Unauthorized"  
});
```

Therefore, middleware is a classic practical example of Chain of Responsibility.

Q294. What is Template Method pattern?

Answer:

Template Method defines the **overall algorithm structure** while allowing individual steps to vary.

Example:

```
fetchData()
```

↓

```
parseData()
```

↓

```
validateData()
```

↓

transformData()

The overall workflow stays fixed, while subclasses or implementations customize individual steps.

Traditionally, Template Method uses inheritance.

In modern JavaScript, composition is often preferred.

Q295. What is Iterator pattern?

Answer:

Iterator provides a standard way to traverse a collection without exposing its internal implementation.

JavaScript supports this through:

Symbol.iterator

and:

for...of

Example:

```
const numbers = [10, 20, 30];
```

```
for (const number of numbers) {  
  console.log(number);  
}
```

Async iteration is supported through:

Symbol.asyncIterator

and:

```
for await (const item of items) {  
}
```

React & Frontend Design Pattern Questions

Q296. Which design patterns are commonly used in React applications?

Answer:

Common patterns include:

Pattern React/Frontend usage

Observer State subscriptions

Pattern React/Frontend usage

Strategy Different behaviors/algorithms

Factory Dynamic component/object creation

Adapter API normalization

Facade Service APIs

Decorator HOCs / function wrappers

Composite Component tree

State Complex UI workflows

Command Undo/redo

Pub/Sub Event-driven systems

However, React applications often favor **composition and hooks** rather than forcing classical GoF patterns everywhere.

Q297. Is a Custom Hook a design pattern?

Answer:

A Custom Hook is not one of the classical GoF design patterns.

It is a **React-specific composition technique** for reusing stateful logic.

Example:

```
function useOnlineStatus() {  
  const [online, setOnline] = useState(true);
```

```
  // subscription logic...
```

```
  return online;
```

```
}
```

Multiple components can reuse the behavior:

```
const online = useOnlineStatus();
```

Interview point:

Custom Hooks are an example of composition and logic reuse rather than a classical GoF pattern.

Q298. Are Higher-Order Components still commonly preferred in modern React?

Answer:

They are still valid, but for many use cases, **Custom Hooks and composition are preferred.**

HOC:

```
const ProtectedPage = withAuth(Page);
```

Custom Hook:

```
function Page() {  
  const user = useAuth();  
}
```

Hooks often provide:

- Easier composition
- Less wrapper nesting
- Better readability
- Easier logic reuse

HOCs are still useful in some libraries and cross-cutting concerns.

Q299. What is the most important principle when applying design patterns in React?

Answer:

Don't use a pattern just because you know it.

Start with the problem.

For example:

If you have:

3 simple conditional branches

don't immediately create:

Factory

Strategy

AbstractFactory

Instead, ask:

Does introducing the pattern reduce complexity?

A good React design often prefers:

Simple component



Composition



Custom Hook



Service / Utility



Pattern when complexity requires it

Senior-level answer:

Patterns should reduce complexity, not add abstraction for its own sake.

Q300. How would you explain your approach to design patterns in a senior frontend interview?

Answer:

A strong answer would be:

"I don't start by looking for a design pattern. I first identify the design problem, such as excessive conditional logic, tight coupling, complex object creation, duplicated behavior, or difficult state transitions. Then I choose the simplest pattern that addresses that problem. For example, I use Strategy for interchangeable algorithms, Factory for object creation, Adapter for normalizing external APIs, Facade for hiding complex services, Observer for subscriptions, and State or state machines for complex workflows. In React specifically, I prefer composition, hooks, and simple functions before introducing classical patterns."

★ **Senior-level takeaway**

Remember this sequence:

Identify the problem



Understand responsibilities



Reduce coupling



Prefer simple composition



Choose a pattern if it adds value



Keep the abstraction justified

Q301. How do you decide between local state, lifted state, and global state?

Answer:

I decide the state scope based on **who needs the state and how frequently it changes**.

1. Local State — useState

Use local state when only one component needs the data.

Examples:

- Modal open/close
- Input value
- Accordion expanded state
- Tooltip visibility
- Local UI state

```
function SearchBar() {  
  const [query, setQuery] = useState("");  
  
  return (  
    <input  
      value={query}  
      onChange={(e) => setQuery(e.target.value)}  
    />  
  );  
}
```

2. Lifted State

If multiple nearby components need the same state, move it to their **closest common parent**.

```
function ProductPage() {  
  const [selectedItems, setSelectedItems] = useState([]);
```

```

return (
  <>
    <ProductList onSelect={setSelectedItems} />
    <CartSummary items={selectedItems} />
  </>
);
}

```

This avoids duplicating the same state in multiple components.

3. Context

If many components need the same data and it changes relatively infrequently, Context can be appropriate.

Examples:

- Theme
- Locale
- Authentication information
- Feature configuration

4. Global State

If unrelated parts of the application need frequently changing **client-side state**, I would consider Zustand or Redux Toolkit.

Examples:

- Shopping cart
- Complex client-side workflows
- Global filters
- Client-side preferences

5. Server State

If the data comes from an API, I generally use a server-state solution such as **TanStack Query or RTK Query** rather than putting every API response into Redux or Zustand.

Interview Summary

"I start with the smallest possible state scope. I use local state by default, lift state when nearby components need to share it, use Context for broadly consumed and relatively stable values, and use a global state library for frequently changing client state shared across unrelated components. For API data, I prefer a server-state solution such as TanStack Query."

Common mistake: Putting everything into a global store.

● **Q302. What are the performance traps of React Context? When should you not use it?**

Answer:

The main performance issue with Context is that **components consuming a context re-render when the context value changes**.

For example:

```
const AppContext = createContext();
```

```
function AppProvider({ children }) {  
  const [theme, setTheme] = useState('light');  
  const [notifications, setNotifications] = useState([]);
```

```
  return (  
    <AppContext.Provider  
      value={{  
        theme,  
        setTheme,  
        notifications,  
        setNotifications,  
      }}  
    >  
      {children}  
    </AppContext.Provider>  
  );  
}
```

Suppose notifications changes.

The context value is a new object, so consumers of AppContext can re-render even if they only use theme.

```
function ThemeToggle() {  
  const { theme } = useContext(AppContext);
```

```
    return <button>{theme}</button>;  
  }
```

How can I improve this?

1. Split Contexts

Instead of:

AppContext

└─ user

└─ theme

└─ notifications

use:

AuthContext

ThemeContext

NotificationContext

Now components subscribe only to the context they actually need.

2. Memoize the Provider Value

```
const value = useMemo(  
  () => ({ theme, setTheme }),  
  [theme]  
);
```

This keeps the context value reference stable when unrelated state changes.

3. Keep Context Focused

Avoid creating one huge Context containing the entire application's state.

When should I avoid Context?

I would avoid using Context for:

- Frequently changing state
- High-frequency updates such as typing or animation
- Very large state objects
- State requiring fine-grained subscriptions
- Complex state requiring selectors or advanced middleware

For those situations, Zustand or Redux Toolkit may be more appropriate.

Interview Summary

"Context is great for broadly consumed and relatively stable values such as theme, locale, and authentication information. Its main limitation is that consumers respond to context value changes, and Context doesn't provide built-in selector-based subscriptions. So I avoid using one large Context for frequently changing application state."

● Q303. How does Redux Toolkit work? What is RTK Query?

Answer:

Redux Toolkit (RTK) is the recommended way to write Redux applications. It reduces the boilerplate associated with traditional Redux.

The main APIs include:

- `configureStore`
- `createSlice`
- `createAsyncThunk`
- `createSelector`
- `createApi` for RTK Query

createSlice

A slice typically contains:

- Initial state
- Reducers
- Generated action creators

```
const counterSlice = createSlice({  
  name: 'counter',
```

```
  initialState: {  
    value: 0,  
  },
```

```
  reducers: {  
    increment: (state) => {  
      state.value += 1;  
    },
```

```
    decrement: (state) => {
```

```
    state.value -= 1;
  },
},
});
```

Although this looks like mutation:

```
state.value += 1;
```

Redux Toolkit uses **Immer** internally to produce immutable state updates.

Store

```
const store = configureStore({
  reducer: {
    counter: counterSlice.reducer,
  },
});
```

Async Operations

`createAsyncThunk` can be used to manage asynchronous operations:

```
const fetchProducts = createAsyncThunk(
  'products/fetch',
  async () => {
    const response = await fetch('/api/products');
    return response.json();
  }
);
```

The reducer can respond to:

pending

fulfilled

rejected

What is RTK Query?

RTK Query is Redux Toolkit's solution for **server-state management**.

It provides:

- Data fetching
- Caching

- Request deduplication
- Loading/error states
- Cache invalidation
- Refetching
- Generated hooks

Example:

```
const api = createApi({
  baseQuery: fetchBaseQuery({
    baseUrl: '/api',
  }),
```

```
  tagTypes: ['Product'],
```

```
  endpoints: (builder) => ({
    getProducts: builder.query({
      query: () => '/products',
      providesTags: ['Product'],
    }),
```

```
    updateProduct: builder.mutation({
      query: ({ id, ...data }) => ({
        url: `/products/${id}`,
        method: 'PATCH',
        body: data,
      }),
```

```
      invalidatesTags: ['Product'],
    }),
  });
```

When the mutation invalidates the Product tag, RTK Query can refetch affected queries.

Simple Mental Model

Redux Toolkit



Client/application state

RTK Query



Server/API state

Interview Summary

"Redux Toolkit simplifies Redux through APIs such as `createSlice` and `configureStore`, while Immer handles immutable updates behind the scenes. RTK Query extends Redux Toolkit with server-state capabilities such as fetching, caching, invalidation, and refetching."

● Q304. What is Zustand? When does it beat Redux?

Answer:

Zustand is a lightweight state-management library that provides global client state with minimal boilerplate.

Example:

```
import { create } from 'zustand';
```

```
const useCartStore = create((set) => ({  
  items: [],
```

```
  addItem: (item) =>  
    set((state) => ({  
      items: [...state.items, item],  
    })),
```

```
  removeItem: (id) =>  
    set((state) => ({  
      items: state.items.filter((item) => item.id !== id),  
    })),
```

```
}});
```

A component can subscribe directly to a selected piece of state:

```
function CartBadge() {  
  const count = useCartStore(  
    (state) => state.items.length  
  );  
  
  return <span>{count}</span>;  
}
```

The selector:

```
(state) => state.items.length
```

allows the component to subscribe to the specific value it needs.

Why is Zustand simpler than Redux?

With Zustand, you generally don't need:

Action types

Action creators

Reducers

Dispatch

Provider

Instead:

Store

↓

State + Actions

↓

Selector

↓

Component

When would I choose Zustand?

I would consider Zustand when:

- The application has moderate client-side state
- I want minimal boilerplate
- I need global state

- I need selector-based subscriptions
- I need to access state outside React
- Redux's additional structure isn't necessary

When would Redux Toolkit be better?

Redux Toolkit may be preferable when:

- The application is large
- Many developers work on the state layer
- Strong conventions are important
- Complex middleware is required
- Redux DevTools are valuable
- The organization already has Redux expertise

Interview Summary

"I prefer Zustand when I need simple global client state with minimal boilerplate and selector-based subscriptions. I prefer Redux Toolkit when the application or team benefits from stronger conventions, middleware, debugging tools, and a more structured architecture."

● Q305. What is the difference between server state and client state? How does TanStack Query change the picture?

Answer:

This is one of the most important modern state-management concepts.

Client State

Client state is **created and owned by the frontend**.

Examples:

- Modal open/closed
- Selected tab
- Theme
- Sidebar state
- Form input
- Local preferences

Server State

Server state originates from the backend and the frontend maintains a **cached copy**.

Examples:

- Users
- Products
- Orders
- Comments
- Notifications
- Dashboard data

Server state is different because it can:

- Become stale
 - Change outside the current browser
 - Require refetching
 - Require caching
 - Require synchronization
 - Have loading and error states
-

How TanStack Query Helps

Without a server-state library, we may have to manually manage:

data

loading

error

cache

refetch

retry

invalidation

With TanStack Query:

```
const { data, isLoading, error } = useQuery({
  queryKey: ['products', category],

  queryFn: () =>
    fetch(`/api/products?category=${category}`)
      .then((res) => res.json()),
});
```

TanStack Query handles much of the server-state lifecycle.

Important Features

- Caching
- Request deduplication
- Background refetching
- Retry
- Cache garbage collection
- Query invalidation
- Optimistic updates
- Pagination
- Infinite queries

For example:

```
queryClient.invalidateQueries({  
  queryKey: ['products'],  
});
```

This marks matching cached data as invalid so it can be refreshed according to the query's configuration.

Modern Architecture

UI State

↓

useState / useReducer

Global Client State

↓

Zustand / Redux Toolkit

Server State

↓

TanStack Query / RTK Query / SWR

URL State

↓

URL Search Parameters

Complex Form State



React Hook Form

Interview Summary

"I separate client state from server state. Client state is owned by the browser, while server state is a cached representation of backend data and therefore needs synchronization, caching, refetching, and invalidation. TanStack Query handles these server-state concerns so I don't need to put every API response into Redux or Zustand."

● Q306. What are state machines? When would you use XState over useState?

Answer:

A **state machine** models an application as a set of explicit states and the events that can transition between those states.

It is particularly useful for complex workflows.

Problem with Multiple Boolean Flags

Consider:

```
const [isLoading, setIsLoading] = useState(false);  
const [isSuccess, setIsSuccess] = useState(false);  
const [isError, setIsError] = useState(false);  
const [isRetrying, setIsRetrying] = useState(false);
```

This can create invalid combinations such as:

isLoading = true

isSuccess = true

isError = true

A state machine avoids many of these impossible combinations by explicitly defining valid states.

idle



processing



success

processing

↓

failed

↓

retry

↓

processing

Example

```
const paymentMachine = createMachine({
```

```
  id: 'payment',
```

```
  initial: 'idle',
```

```
  states: {
```

```
    idle: {
```

```
      on: {
```

```
        SUBMIT: 'processing',
```

```
      },
```

```
    },
```

```
    processing: {
```

```
      on: {
```

```
        SUCCESS: 'success',
```

```
        FAILURE: 'failed',
```

```
      },
```

```
    },
```

```
    failed: {
```

```
      on: {
```

```
        RETRY: 'processing',
```

```
        CANCEL: 'idle',
```

```
    },  
  },  
  
  success: {  
    type: 'final',  
  },  
},  
});
```

When would I use XState?

I would consider XState for:

- Payment workflows
- Checkout
- Multi-step onboarding
- Authentication flows
- Complex wizards
- Media players
- Complex asynchronous workflows

Especially when the number of states and transitions becomes difficult to reason about with ordinary React state.

When is useState better?

For simple state:

```
const [isOpen, setIsOpen] = useState(false);
```

A state machine would be unnecessary complexity.

Interview Summary

"I use useState for simple independent state. I consider XState when the UI represents a complex workflow with many states and transitions, especially when invalid state combinations could cause business or user-facing problems."

● Q307. How do you handle state synchronization, optimistic updates, conflict resolution, and offline support?

Answer:

I think about state synchronization in three main areas:

1. **Optimistic updates**

2. Conflict resolution

3. Offline synchronization

1. Optimistic Updates

An optimistic update changes the UI **before the server confirms the operation**.

For example, when a user likes a post:

User clicks Like

↓

Update UI immediately

↓

Send API request

↓

Success → Keep update

Failure → Rollback

With TanStack Query, I can save the previous state and roll back if the request fails:

```
onMutate: async () => {  
  await queryClient.cancelQueries({  
    queryKey: ['post', postId],  
  });  
  
  const previous =  
    queryClient.getQueryData(['post', postId]);  
  
  queryClient.setQueryData(  
    ['post', postId],  
    (old) => ({  
      ...old,  
      liked: true,  
      likeCount: old.likeCount + 1,  
    })  
  );
```

```
    return { previous };  
  },  
  
  onError: (error, variables, context) => {  
    queryClient.setQueryData(  
      ['post', postId],  
      context.previous  
    );  
  },  
}
```

This provides a fast user experience while still allowing rollback.

2. Conflict Resolution

Conflicts happen when multiple clients modify the same data.

Last-Write-Wins

The latest update overwrites the previous update.

It's simple but can cause data loss.

Optimistic Locking

The server maintains a version number:

Document version = 5

The client sends:

expectedVersion = 5

If another client already modified the document:

Server version = 6

the server can return:

409 Conflict

The application can then:

- Reload the latest version
- Show a diff
- Merge changes
- Ask the user to resolve the conflict

CRDTs

CRDTs (Conflict-free Replicated Data Types) are useful for collaborative applications where concurrent changes need to be merged automatically.

They are relevant to:

- Collaborative editors
 - Real-time documents
 - Multi-user applications
-

3. Offline Synchronization

For applications that need offline support, I can use a pattern such as:

User action

↓

Persist locally

↓

User continues working offline

↓

Connection restored

↓

Replay queued operations

↓

Resolve conflicts

For example:

```
window.addEventListener('online', () => {  
  offlineQueue.replay();  
});
```

The important requirement is that offline operations should be **durable, replayable, and safe to retry**.

For production applications, I would use a robust persistence/synchronization strategy rather than relying only on a simple localStorage array.

Cross-Tab Synchronization

For multiple browser tabs, BroadcastChannel can be used to communicate state changes:

```
const channel = new BroadcastChannel('app-state');
```



```
channel.postMessage({  
  type: 'STATE_UPDATED',  
});
```

Another tab can listen:

```
channel.onmessage = (event) => {  
  // Synchronize relevant state  
};
```

Interview Summary

"For synchronization, I use optimistic updates when the operation is safe to predict and can be rolled back. For concurrent modifications, I consider optimistic locking or other conflict-resolution strategies. For offline applications, I persist actions or data locally and replay them when connectivity returns. The appropriate strategy depends on the business requirements and how much stale or conflicting data the application can tolerate."

Frontend System Design — Interview Questions & Answers

308. How do frontend system design interviews work?

Answer:

Frontend system design interviews evaluate how I approach a large UI problem from **requirements → architecture → data → performance → trade-offs**.

A typical flow is:

1. Clarify requirements
↓
2. Define architecture
↓
3. Design components
↓
4. Define state and data flow
↓
5. Design API/communication
↓
6. Discuss performance
↓
7. Handle accessibility and edge cases



8. Explain trade-offs

I would not immediately start coding.

First, I clarify:

- Who are the users?
- What are the core features?
- How much data is expected?
- Is SEO important?
- Is real-time functionality required?
- Is offline support required?
- What are the performance expectations?
- What platforms need to be supported?

Then I design the component architecture, rendering strategy, state management, API communication, and optimization strategy.

Key point:

Frontend system design interviews evaluate engineering decisions and trade-offs more than React syntax.

309. What are interviewers evaluating in a frontend system design interview?

Answer:

I would focus on six major areas:

1. Requirement gathering

Can I clarify functional and non-functional requirements before designing?

2. Component architecture

Can I break a complex application into reusable and maintainable components?

3. State management

Can I correctly separate:

- Local state
- Global client state
- Server state
- URL state

4. API and communication design

Can I choose appropriately between:

- REST
- GraphQL
- WebSocket
- SSE
- Polling

5. Performance

Do I consider:

- Code splitting
- Lazy loading
- Caching
- Virtualization
- Image optimization
- Rendering performance

6. Trade-offs

Can I explain **why** I selected one approach over another?

For a senior candidate, saying:

"I'll use Redux."

is not enough.

I should explain why Redux is needed, what alternatives exist, and what trade-offs I am accepting.

310. What is the RADIO framework?

Answer:

RADIO is a useful framework for structuring a frontend system design interview.

R → Requirements

A → Architecture

D → Data Model

I → Interface / API

O → Optimization

R — Requirements

Identify functional and non-functional requirements.

Functional:

- Create post
- Like post
- Comment
- Search

Non-functional:

- Performance
- Accessibility
- Scalability
- SEO
- Offline support
- Real-time updates

A — Architecture

Define:

- Component tree
- Routing
- Rendering strategy
- Client/server boundaries

D — Data Model

Define:

- Server state
- Client state
- URL state
- Data structures

I — Interface/API

Define:

- REST/GraphQL APIs
- WebSocket/SSE events
- Component interfaces

O — Optimization

Discuss:

- Caching
- Virtualization
- Lazy loading
- Code splitting
- Image optimization
- Error handling
- Accessibility

Interview tip:

"I'll use the RADIO framework to systematically cover requirements, architecture, data, APIs, and optimization."

311. How do you decide between REST, GraphQL, WebSocket, and SSE?

Answer:

I choose based on the communication pattern.

Technology	Best suited for
REST	CRUD and conventional APIs
GraphQL	Complex/nested data requirements
WebSocket	Bidirectional real-time communication
SSE	Server-to-client real-time updates
Polling	Simple periodic updates

Examples:

Chat

→ WebSocket + REST

Notifications

→ SSE + REST

Collaborative editor

→ WebSocket

E-commerce

→ REST

Search

→ REST

My decision is based on the **actual communication requirements**, rather than choosing the most advanced technology.

312. How do you decide where state should live?

Answer:

I classify state into four categories:

Local UI State

↓

Global Client State

↓

Server State

↓

URL State

Local state

Use useState or useReducer.

Examples:

- Modal open/close
- Form values
- Accordion state

Global client state

Use Redux or Zustand when multiple unrelated components need shared client state.

Examples:

- Theme
- Shopping cart UI
- Active chat
- Global UI preferences

Server state

Use a server-state library such as TanStack Query.

Examples:

- Products
- Posts
- Comments
- Notifications

URL state

Use query parameters when state should be:

- Shareable
- Bookmarkable
- Navigable with browser history

Example:

/search?q=phone&brand=Samsung&sort=price

Principle:

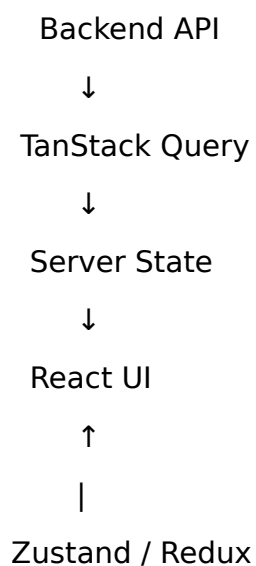
Keep state as close as possible to where it is used. Don't put everything into global state.

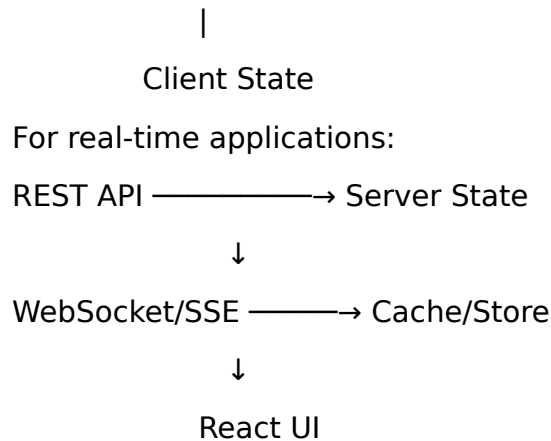
313. How would you design data flow in a frontend application?

Answer:

I separate **server state** from **client state**.

A typical architecture is:





This keeps responsibilities clear and prevents components from directly managing every type of state.

314. When would you use cursor pagination instead of offset pagination?

Answer:

I prefer **cursor pagination for frequently changing datasets**, especially feeds.

Offset pagination:

GET /posts?page=2&limit=20

Cursor pagination:

GET /posts?cursor=abc123&limit=20

Suppose a user loads page 1 and new posts are inserted.

With offset pagination, existing records can shift positions, potentially causing:

- Duplicate posts
- Missing posts
- Inconsistent pages

Cursor pagination continues from a stable position.

Therefore, for a real-time social feed, I would generally choose cursor pagination.

Trade-off:

Cursor pagination makes arbitrary page jumping more difficult.

315. How do you approach frontend performance in system design?

Answer:

I consider performance at multiple levels:

Network



Loading



JavaScript



Rendering



DOM



Images



Interactions

I would consider:

- Code splitting
- Lazy loading
- Image optimization
- CDN
- Browser caching
- API caching
- Virtualization
- Reducing unnecessary re-renders
- Preloading critical resources
- Web Workers for CPU-heavy work

For example, if I have:

10,000 items

I would avoid creating 10,000 DOM nodes.

Instead:

10,000 records



Virtualization



~30-100 visible DOM items

316. Why is virtualization important?

Answer:

Virtualization renders only the items currently visible in the viewport instead of rendering the entire dataset.

Suppose a chat contains:

50,000 messages

Rendering all of them can create excessive:

- DOM nodes
- Memory usage
- Layout work
- Paint work

Instead:

50,000 messages

↓

Virtualized list

↓

Visible messages + buffer

Virtualization is useful for:

- Chat
- Feeds
- Tables
- Spreadsheets
- Large search results
- Live dashboards

Libraries such as [@tanstack/react-virtual](#) can help implement this.

317. How do you estimate frontend payload size?

Answer:

I break the initial page into:

HTML

- + JavaScript
- + CSS
- + Fonts
- + API JSON
- + Images/video

For example:

JavaScript: 200 KB

CSS: 40 KB

Fonts: 70 KB

API JSON: 20 KB

Images: 500 KB

≈ 830 KB

Then I identify the largest contributors.

Common optimizations include:

- WebP/AVIF images
- Responsive images
- Lazy loading
- Code splitting
- Compression
- CDN
- Preloading critical resources

318. What is the 60 FPS frame budget?

Answer:

At 60 FPS:

$1000 / 60 \approx 16.6 \text{ ms}$

The browser has approximately **16.6 milliseconds per frame**.

That time includes:

JavaScript

+

Style calculation

+
Layout
+
Paint
+
Composite

If JavaScript blocks the main thread for too long, the UI becomes janky.

To improve responsiveness, I can use:

- Virtualization
 - Batching
 - Debouncing
 - Web Workers
 - Efficient rendering
 - Avoiding unnecessary updates
-

319. How would you design a chat application like WhatsApp Web?

Answer:

I would structure it as:

ChatApplication

```
├── Sidebar
|   ├── Search
|   ├── ChatList
|   └── NewChat
|
├── ChatWindow
|   ├── ChatHeader
|   ├── MessageList
|   ├── TypingIndicator
|   └── MessageInput
|
└── WebSocketManager
```

Communication

Use REST for:

- Fetching chat history
- Loading conversations
- Uploading media

Use WebSocket for:

- Sending/receiving messages
- Typing indicators
- Read receipts
- Online status

Message flow:

User types



Optimistic message



WebSocket



Server



Recipient



ACK



sent → delivered → read

For large conversations, I would use a virtualized message list.

For offline support:

Offline



IndexedDB queue



Connection restored



Send queued messages

320. Why would you choose WebSocket for chat instead of SSE?

Answer:

Chat requires **bidirectional communication**.

Both client and server need to send data:

Client → Server

Server → Client

WebSocket supports both directions over a persistent connection.

SSE is primarily:

Server → Client

Therefore, WebSocket is a better fit for chat.

Trade-off:

WebSocket infrastructure is generally more complex and stateful than simple HTTP-based communication.

321. When would you use SSE instead of WebSocket?

Answer:

I use SSE when communication is primarily:

Server → Client

Examples:

- Notifications
- Live scores
- Stock updates
- Dashboard metrics
- Feed updates
- AI response streaming

Architecture:

Browser

↑

| SSE

|

Server

If the client doesn't need to continuously send real-time messages back, SSE can be simpler than WebSocket.

322. How would you design a notification system?

Answer:

I would use:

NotificationBell

└─ Badge

└─ NotificationDropdown

└─ NotificationList

Communication:

REST

→ Fetch notification history

→ Mark notifications as read

SSE

→ Receive new notifications

When an SSE event arrives:

SSE event

↓

Update query cache

↓

Increment unread count

↓

Update notification UI

I would also consider grouping notifications.

Instead of:

A liked your post

B liked your post

C liked your post

D liked your post

I could show:

A and 3 others liked your post

323. How would you design a collaborative editor like Google Docs?

Answer:

The biggest challenge is **concurrent editing and conflict resolution**.

Suppose two users modify the same document simultaneously.

I would consider:

Operational Transform — OT

Operations are transformed against concurrent operations.

CRDT

Data structures are designed so replicas can merge changes consistently.

Examples include:

- Yjs
- Automerge

For a collaborative editor, I would generally consider OT or CRDT rather than simple last-write-wins.

Architecture:

Editor

↓

WebSocket

↓

Collaboration Server

↓

Other Clients

Cursor positions and selections can be synchronized separately from document content.

324. What is the difference between OT and CRDT?

Answer:

Operational Transform

Client A → Operation

Client B → Operation

↓

Server transforms operations

↓

Consistent document

OT focuses on transforming concurrent operations so that all clients converge.

CRDT

CRDTs use data structures and operations designed to merge distributed changes consistently.

They are particularly useful for:

- Offline editing
- Distributed collaboration
- Collaborative applications

Trade-off:

CRDT implementations can introduce additional memory/storage overhead depending on the data structure.

325. How would you design a video conferencing application?

Answer:

I would structure the frontend as:

MeetingRoom

├─ VideoGrid

| └─ VideoTile × N

├─ ControlBar

├─ ChatSidebar

└─ MediaManager

For media communication, I would use **WebRTC**.

A signaling server can use WebSocket to exchange:

SDP offers/answers

ICE candidates

For a small meeting, peer-to-peer communication can be reasonable.

For larger meetings, I would use an **SFU — Selective Forwarding Unit**.

User A └─

User B └─┬→ SFU → Participants

User C └─┘

The SFU receives streams and forwards appropriate streams to participants.

326. Why use an SFU for large video meetings?

Answer:

Without an SFU, participants may need direct connections with many other participants.

With N participants, the number of peer relationships can grow approximately as:

$$N \times (N - 1)$$

With an SFU:

Participant

↓

SFU

↙ ↓ ↘

Users

Each participant can primarily upload their stream to the SFU, while the SFU forwards streams to the other participants.

This significantly reduces client-side connection and upload complexity.

327. How would you design a YouTube-like video platform?

Answer:

Frontend architecture:

VideoPage

├─ VideoPlayer

├─ VideoInfo

├─ Recommendations

├─ Comments

└─ Description

For video delivery, I would use adaptive streaming such as:

HLS / DASH

The video can be encoded into multiple qualities:

1080p

720p

480p

360p

The player dynamically chooses an appropriate quality based on:

- Network conditions
- Buffer health
- Device capabilities

This helps reduce buffering and improves playback experience.

328. What is adaptive bitrate streaming?

Answer:

Adaptive Bitrate Streaming, or ABR, means the video is available in multiple encoded qualities and the player dynamically switches between them.

For example:

Good network

↓

1080p

Poor network

↓

480p

The player can switch:

1080p → 720p → 480p

without requiring the user to manually reload the video.

329. How would you design a social media news feed?

Answer:

I would use:

FeedPage

└─ CreatePost

└─ FeedList

└─ PostCard × N

└─ NewPostBanner

For the initial page:

SSR

↓

Initial feed



Hydration



API fetching

For subsequent updates:

REST → Feed data

SSE/WebSocket → Real-time updates

I would use:

- Cursor pagination
- Virtualization for large feeds
- Optimistic likes
- Cached server state

If new posts arrive while the user is reading, I would display:

"3 new posts"

instead of immediately inserting them.

330. Why shouldn't new posts automatically appear at the top while the user is scrolling?

Answer:

Automatically inserting content above the current viewport can cause:

- Layout shifts
- Scroll position changes
- User losing their reading position
- Poor user experience

Instead:

User reading feed



New posts arrive



"3 new posts"



User clicks



Insert new posts

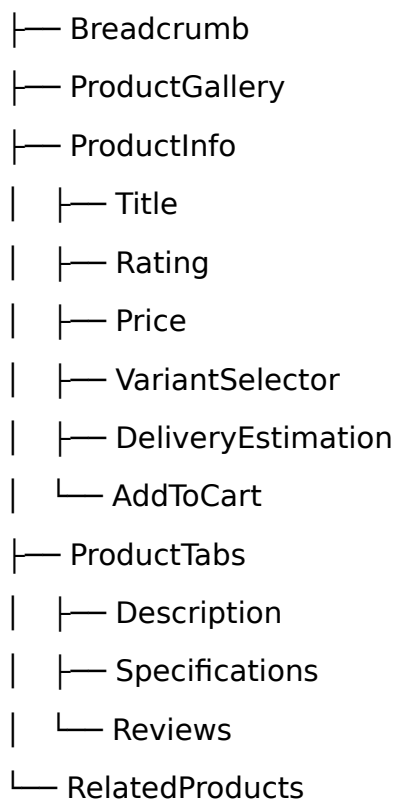
This preserves the user's current reading position.

331. How would you design an e-commerce product page?

Answer:

I would structure it as:

ProductPage



I would consider SSR for the initial product information because product pages are:

- SEO-sensitive
- Public
- Important for initial rendering

Below-the-fold content such as reviews and recommendations can be lazy-loaded.

332. How would you handle Add to Cart optimistically?

Answer:

I would update the UI immediately and then synchronize with the backend.

Click Add to Cart



Update UI immediately



API request



Success → Keep update

Failure → Rollback

Example:

Cart count = 2



User adds product



Cart count = 3 immediately



API request



Success → 3

Failure → 2 + error

Optimistic updates are suitable when the operation is reversible and the user experience benefits from immediate feedback.

333. Why shouldn't payment confirmation use optimistic UI?

Answer:

Payment is different from actions such as liking a post.

A failed payment must not be displayed as successful.

The flow should be:

Click Pay



Loading



Payment gateway



Server confirmation



Order ID received



Show "Order placed"

I would never display successful payment confirmation before server-side confirmation.

I would also use **idempotency keys** to prevent duplicate charges during retries.

334. Where should search filter state live?

Answer:

Important search and filter state should generally live in the **URL query parameters**.

Example:

/search?q=headphones

&brand=Sony,JBL

&price=1000-5000

&sort=price_asc

&page=2

Benefits:

- Shareable URLs
- Bookmarkable searches
- Browser back/forward support
- SSR compatibility
- Better navigation
- Potential SEO benefits where applicable

I wouldn't keep important search filters only in React state.

335. How would you design a spreadsheet like Google Sheets?

Answer:

The major challenge is handling a huge number of cells efficiently.

Architecture:

Spreadsheet

|— Toolbar

|— FormulaBar

|— ColumnHeaders

|— RowNumbers

└ VirtualizedCellGrid

I would render only:

Visible viewport

+

Small buffer

For cell storage, a sparse structure such as:

Map<string, CellData>

can avoid storing unnecessary empty cells.

For formulas, I would maintain a dependency graph.

Example:

A1

↓

B1

↓

C1

If A1 changes, only affected cells need recalculation.

336. How does a spreadsheet formula engine handle dependencies?

Answer:

Suppose:

$B1 = A1 + 10$

$C1 = B1 * 2$

$D1 = C1 + 5$

The dependency graph is:

A1

↓

B1



C1



D1

If A1 changes, I would:

1. Find cells depending on A1.
2. Traverse their dependents.
3. Identify affected cells.
4. Recalculate them in dependency order.

This can be done using a graph traversal and topological ordering.

The advantage is that unrelated spreadsheet cells don't need to be recalculated.

337. How would you design a Kanban board like Jira?

Answer:

Architecture:

KanbanBoard

├─ BoardHeader

└─ ColumnList

 └─ Column

 ├─ ColumnHeader

 └─ CardList

 └─ Card

For drag-and-drop, I could use a library such as dnd-kit.

When a card is moved:

Drag



Optimistic local update



API request



Success → Keep

Failure → Rollback

This provides immediate feedback while still maintaining server consistency.

338. How would you design a calendar application?

Answer:

I would support:

Day

Week

Month

Architecture:

Calendar

├─ Header

├─ Sidebar

└─ CalendarView

 ├─ MonthView

 ├─ WeekView

 └─ DayView

For week/day views, event positioning can be calculated using:

top → start time

height → duration

left → overlap column

width → available column width

For overlapping events, I would use a column-packing strategy so events remain visible side by side.

339. How would you design a frontend design system?

Answer:

I would create reusable components such as:

Button

Input

Modal

Select

Table

Tooltip

Dropdown

The most important part is the **component API**.

Example:

```
<Button  
  variant="primary"  
  size="md"  
  loading={isLoading}  
>  
  Save  
</Button>
```

I would avoid components with dozens of unrelated props.

The architecture could be:

Design Tokens

↓

Components

↓

Storybook

↓

Applications

The system should also provide accessibility behavior centrally so every application does not have to reimplement it.

340. What are design tokens?

Answer:

Design tokens are centralized values representing design decisions.

Examples:

colors.primary

spacing.md

radius.sm

font.heading

They can be exposed as CSS variables:

--ds-color-primary

--ds-spacing-md

--ds-radius-sm

The main benefit is consistency.

If the primary brand color changes, I can update the token rather than modifying dozens of individual components.

341. What is a micro-frontend architecture?

Answer:

Micro-frontends apply the concept of independently owned services to frontend applications.

For example:

Shell

|— Products → Team A

|— Checkout → Team B

|— Payments → Team C

|— Account → Team D

Teams can potentially develop and deploy their applications independently.

Common approaches include:

1. Build-time composition
 2. Runtime composition
 3. Module Federation
 4. Route-based composition
-

342. What are the trade-offs of micro-frontends?

Answer:

Benefits

- Independent team ownership
- Independent deployment
- Clear team boundaries
- Technology isolation when necessary

Costs

- More infrastructure
- Shared dependency management
- Cross-application communication
- Authentication coordination
- Performance overhead
- Version compatibility issues

For example, Module Federation can support independently deployed modules but introduces dependency and runtime compatibility considerations.

I would not choose micro-frontends simply because they are popular.

343. When would you avoid micro-frontends?

Answer:

I would avoid micro-frontends when:

- The team is small.
- Features are tightly coupled.
- Applications share significant state.
- Independent deployment isn't actually needed.
- A modular monolith provides sufficient boundaries.

The key question is:

Do we have an organizational or deployment problem that micro-frontends actually solve?

If not, micro-frontends may introduce unnecessary complexity.

344. How would you design an analytics dashboard?

Answer:

I would create configurable widgets:

Dashboard

└─ MetricWidget

└─ ChartWidget

└─ TableWidget

└─ FunnelWidget

Each widget could contain:

Query configuration

Chart type

Filters

Cached data

Layout

For large datasets, the frontend should not download millions of raw events.

Instead:

10 million raw events



Backend aggregation



365 daily data points



Frontend chart

The frontend should primarily visualize already-aggregated data.

345. How would you design frontend authentication?

Answer:

I would consider:

Login

Signup

Forgot Password

OAuth

Protected Routes

Role-based UI

Session Management

The application needs to distinguish between:

Loading

Authenticated

Unauthenticated

Unauthorized

Example:

Loading

↓

Check session

↓

Authenticated → Application

↓

Unauthenticated → Login

↓

Unauthorized → Access Denied

For multi-tab logout synchronization, I could use BroadcastChannel.

346. How would you design an A/B testing system?

Answer:

The key requirement is **consistent user bucketing**.

For example:

hash(userId + experimentId)

↓

bucket

↓

Control / A / B

The same user should consistently receive the same variant.

For example:

User 123

↓

checkout-v2

↓

hash

↓

Variant B

Ideally, the variant is determined during SSR or middleware when possible.

Otherwise, client-side evaluation after rendering can cause:

Control

↓

Variant B

which creates visual flicker and can affect experiment accuracy.

347. How would you design a CMS-driven website?

Answer:

The CMS manages content while the frontend provides reusable rendering components.

Architecture:

CMS

↓

Page JSON

↓

Frontend

↓

Block mapping

↓

React components

Example:

hero → HeroBlock

feature_grid → FeatureGrid

testimonials → Testimonials

cta → CTA

This allows content editors to rearrange sections without requiring a frontend deployment.

For performance and SEO, I would consider:

SSG + ISR

and trigger revalidation when CMS content changes.

348. How would you design frontend error monitoring?

Answer:

I would capture errors from multiple sources:

window.error

+

unhandledrejection

+

React Error Boundaries

↓

Error Monitoring Service

Each error should include useful context:

- Error message
- Stack trace
- URL
- Browser
- User/session context where appropriate
- Breadcrumbs
- Component stack

For example:

Clicked checkout

↓

Opened payment modal

↓

Selected UPI

↓

API failed

↓

Unhandled exception

Breadcrumbs make debugging much easier.

349. How do you prevent thousands of duplicate errors in monitoring?

Answer:

I would normalize and fingerprint errors.

Dynamic information such as:

user IDs

query parameters

dynamic values

should not cause every occurrence to become a separate issue.

Instead:

Raw errors



Normalize



Generate fingerprint



Group similar errors



Single issue

For example:

Payment API TypeError

Occurrences: 18,423

Affected users: 4,201

This is much more useful than thousands of separate issues.

350. How would you design a Progressive Web App?

Answer:

A PWA typically relies on:

Manifest

Service Worker

HTTPS

Manifest

Defines:

- App name
- Icons
- Theme
- Display mode

Service Worker

Can handle:

- Caching
- Offline fallback
- Background synchronization
- Push notifications

Offline architecture:

Online

↓

API

↓

Render

↓

Cache

Offline

↓

Service Worker

↓

Cached response

↓

Render

Mutations can be queued in IndexedDB and synchronized when connectivity returns.

351. A page has 10,000 items. What would you do?

Answer:

First, I would determine whether the user actually needs all 10,000 items at once.

If not, I would use:

Pagination / Infinite Scroll

If the user needs to navigate through a large list:

Virtualization

+

Pagination

+

Memoized row components

+

Efficient data fetching

For example:

10,000 records

↓

Virtualized list

↓

~50 visible records

This significantly reduces DOM and rendering overhead.

352. A React application is becoming slow. How would you debug it?

Answer:

I would **measure before optimizing**.

I would investigate:

1. React Profiler
2. Browser Performance panel
3. Network waterfall
4. Bundle size
5. Long tasks
6. Excessive re-renders
7. Large DOM
8. Expensive calculations
9. Image sizes
10. API latency

Then I would optimize the actual bottleneck.

Examples:

Large list

→ Virtualization

Large bundle

→ Code splitting

Expensive computation

→ Memoization / Web Worker

Repeated API calls

→ Caching / deduplication

Large images

→ Responsive images / CDN

I wouldn't automatically add useMemo or useCallback everywhere.

353. When would you use a Web Worker?

Answer:

I use Web Workers for CPU-intensive operations that could block the browser's main thread.

Examples:

- Large data processing
- Spreadsheet calculations
- Image processing
- Parsing large files
- Complex transformations

Architecture:

Main Thread



Web Worker



Heavy computation



Result



Main Thread

I wouldn't use Workers for every small calculation because communication and serialization also have overhead.

354. How do you handle offline functionality in a frontend system?

Answer:

I separate read and write operations.

Reads

API

↓

Cache

↓

IndexedDB / Cache Storage

When offline:

Request

↓

Service Worker

↓

Cached data

Writes

User mutation

↓

IndexedDB queue

↓

Connection restored

↓

Background sync

↓

Server

I also need a conflict-resolution strategy when offline changes conflict with newer server data.

355. How do you approach accessibility in frontend system design?

Answer:

Accessibility should be part of the architecture rather than a final checklist.

I consider:

- Semantic HTML
- Keyboard navigation
- Focus management
- Screen-reader support
- ARIA when necessary
- Color contrast
- Reduced motion
- Accessible error messages
- Accessible loading states

Examples:

`<article>`

for an independent feed item.

`<button aria-pressed="true">`

for a toggle button.

For dialogs, focus should move into the dialog and return to the triggering element when the dialog closes.

356. Why did you choose a particular technology in your system design?

Answer:

I wouldn't say:

"Because it is commonly used."

Instead, I would explain:

"I chose X because the communication pattern requires Y. The main benefit is Z, while the trade-off is A. If requirement B changes, I would consider another solution."

This demonstrates that the decision is based on requirements rather than personal preference.

357. What happens if the API fails?

Answer:

I always design for more than the happy path.

I consider:

Loading



Success



Empty



Error



Retry

For example:

API failure



Show user-friendly error



Retry action



Recover when API becomes available

For complex dashboards, I would also consider **partial failures**, where one widget fails while the remaining widgets continue working.

358. What happens if the user goes offline?

Answer:

I would handle:

Online



Normal API communication

Offline



Cached data



Offline indicator

↓

Queue supported mutations

↓

Connection restored

↓

Retry/synchronize

I would also consider conflict resolution when queued changes are synchronized.

359. How would you scale a frontend application?

Answer:

I would consider several dimensions:

Network

→ CDN + caching

JavaScript

→ Code splitting

Rendering

→ Virtualization

API

→ Pagination + caching

Images

→ CDN + responsive formats

Real-time

→ WebSocket/SSE infrastructure

Client memory

→ Bounded caches

Teams

→ Design system + modular boundaries

Frontend scalability isn't only about backend throughput.

It also means keeping the **browser responsive as users, data, and features grow**.

360. What trade-offs would you mention at the end of a system design?

Answer:

I usually summarize two or three major architectural decisions.

For example:

"I chose cursor pagination because the feed changes frequently. The benefit is avoiding duplicate or missing posts, but the trade-off is that users can't directly jump to page 50."

Another example:

"I chose SSE for notifications because communication is server-to-client only. The benefit is lower complexity, while the trade-off is that if we later require bidirectional communication, WebSocket may be more appropriate."

This shows that I am making **intentional architectural decisions** rather than simply selecting popular

★ Formula to Memorize

For **any frontend system design problem**, follow this sequence:

1. Requirements

↓

2. Scale / Constraints

↓

3. Component Architecture

↓

4. Rendering Strategy

↓

5. State Management

↓

6. API / Communication

↓

7. Data Flow

↓

8. Performance

↓

9. Accessibility

↓

10. Loading / Error / Empty / Offline States

↓

11. Trade-offs

12A. Modules & Bundlers — Interview Questions & Answers

● Must Know

Q361. What's the difference between CommonJS and ES Modules? What is `import.meta`?

Answer:

JavaScript has two major module systems:

- **CommonJS (CJS)** — traditionally used in Node.js
- **ES Modules (ESM)** — the JavaScript standard used by modern browsers and bundlers

CommonJS

```
// math.js
```

```
module.exports = {  
  add,  
  subtract,  
};
```

```
const { add } = require("./math");
```

CommonJS uses `require()` and `module.exports`.

A key characteristic is that `require()` can be dynamic:

```
const name = condition ? "./a" : "./b";
```

```
const module = require(name);
```

Because the module path can be determined at runtime, static analysis is harder.

ES Modules

```
// math.js
```

```
export function add(a, b) {
```

```
    return a + b;
}
```

```
export function subtract(a, b) {
    return a - b;
}
```

```
import { add } from "./math.js";
```

ESM imports and exports are statically analyzable:

```
import { add } from "./math.js";
```

This allows bundlers to understand the module dependency graph at build time and perform **tree shaking**.

Dynamic import()

ESM also supports dynamic imports:

```
const module = await import("./heavyModule.js");
```

```
module.doSomething();
```

Unlike `require()`, `dynamic import()` is asynchronous and returns a Promise.

It is commonly used for code splitting:

```
const Dashboard = lazy(() => import("./Dashboard"));
```

What is import.meta?

`import.meta` provides metadata about the current ES module.

For example:

```
console.log(import.meta.url);
```

In a browser, this can be something like:

```
https://example.com/src/utils.js
```

In Node.js:

```
file:///home/user/project/src/utils.js
```

Vite also provides:

```
import.meta.env.VITE_API_URL;
```

```
import.meta.env.MODE;
```

```
import.meta.env.DEV;
```

```
import.meta.env.PROD;
```

Interview summary

CommonJS

require()

module.exports

Traditionally Node.js

Can be dynamically resolved

Harder to tree-shake reliably

Mostly synchronous loading model

ES Modules

import

export

JavaScript standard

Statically analyzable

Excellent for tree shaking

Supports static and dynamic imports

Best answer: For modern frontend applications, I prefer **ES Modules** because they are standardized, statically analyzable, and enable tree shaking and better bundler optimization.

Q362. What is a bundler? Why do frontend applications need bundlers?

Answer:

A **bundler** takes multiple source files and their dependencies and produces optimized files that can be efficiently delivered to the browser.

For example:

src/

├─ main.tsx

├─ App.tsx

├─ components/

│ └─ Header.tsx

│ └─ Dashboard.tsx

├─ styles.css

└─ utils.ts

The application has many dependencies:

main.tsx

↓

App.tsx

↓

Header.tsx

Dashboard.tsx

↓

utils.ts

The bundler analyzes this dependency graph and generates optimized output:

dist/

```
|— index.html
|— assets/
|   |— main.abc123.js
|   |— dashboard.def456.js
|   └— styles.xyz789.css
```

A modern bundler can perform:

- Module resolution
- TypeScript/JSX transformation
- Tree shaking
- Code splitting
- Minification
- Asset processing
- CSS processing
- Dependency optimization
- Cache-busting through hashed filenames

Why not simply send all source files?

Because production applications can contain thousands of modules.

Sending everything individually can cause:

- More network requests
- Larger payloads
- Poor loading performance
- More parsing overhead

Bundlers optimize the final application for production delivery.

Interview answer

A bundler analyzes the application's module graph, transforms source code, removes unused code, splits code into optimized chunks, and generates browser-ready assets.

Q363. Compare Webpack, Vite, Rspack, and Turbopack.

Answer:

All four are tools involved in modern frontend build workflows, but they have different design goals.

Webpack

Webpack is the mature and highly configurable option.

It has a huge ecosystem of:

- Loaders
- Plugins
- Configuration options
- Existing integrations

The downside is that large projects can have slower development startup and rebuild times.

Use it when:

- An existing application already uses Webpack
 - You need a specialized Webpack plugin
 - The team has strong Webpack expertise
-

Vite

Vite focuses heavily on fast development.

During development, it serves source modules using native ESM and transforms files on demand rather than creating one large bundle upfront.

For production, Vite performs an optimized production build.

Use it when:

- Starting a new standalone React/Vue/Svelte application
 - Fast development startup is important
 - You want minimal configuration
-

Rspack

Rspack is a Rust-based bundler designed to be highly compatible with Webpack concepts and configurations.

Its goal is essentially:

Webpack-like ecosystem and compatibility with significantly faster build performance.

Use it when:

- You have a large Webpack application
 - You want faster builds
 - Migrating completely to Vite would be expensive
-

Turbopack

Turbopack is a Rust-based bundler deeply integrated with Next.js.

It uses incremental computation so that unchanged parts of the application don't need to be recomputed.

Use it when:

- Building a Next.js application
- You want the Next.js-native build/dev experience

Quick comparison

Tool	Main strength	Typical choice
Webpack Ecosystem & flexibility		Existing complex apps
Vite	Fast development	New standalone frontend apps
Rspack	Webpack compatibility + speed	Large Webpack migrations
Turbopack	Next.js integration + incremental builds	Next.js

Interview answer

For a new standalone React application, I'd generally choose Vite. For Next.js, I'd use the Next.js-supported Turbopack workflow. For an existing large Webpack application where migration cost is high, Rspack can be attractive.

Q364. Why is Vite fast? Explain development vs production.

Answer:

Vite is fast primarily because its **development workflow avoids bundling the entire application upfront**.

Traditional bundler development

A traditional workflow might do:

Start server

↓

Build dependency graph

↓

Bundle thousands of modules



Serve application

As the project grows, startup can become slower.

Vite development

Vite uses native ES Modules:

Start server



Serve index.html



Browser requests main.tsx



Vite transforms it



Browser requests imported modules



Vite transforms them on demand

Only the modules needed by the browser are processed.

Dependency pre-bundling

Vite also pre-bundles dependencies such as:

react

react-dom

lodash

date-fns

This avoids the browser having to request hundreds of internal dependency modules individually.

HMR

Vite also provides fast Hot Module Replacement.

If you modify:

Button.tsx

Vite can update that module instead of rebuilding the entire application.

Production

Development and production are different.

For production, Vite creates optimized output involving:

- Bundling
- Tree shaking
- Code splitting
- Minification
- Optimized assets

Interview answer

Vite is fast in development because it serves source modules as native ESM and transforms them on demand instead of bundling the entire application upfront. It also pre-bundles dependencies and provides efficient HMR. Production builds are optimized into deployable assets.

Q365. What is tree shaking? Why does ESM enable better tree shaking than CommonJS?

Answer:

Tree shaking is the process of removing code that is imported but never actually used.

Suppose we have:

```
// utils.js
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

And:

```
import { add } from "./utils.js";
```

```
console.log(add(2, 3));
```

The application doesn't use subtract().

A bundler can remove it:

Before:

```
add()
subtract()
```

After tree shaking:

```
add()
```

Why ESM works well

ESM has static imports and exports:

```
import { add } from "./utils.js";
```

The bundler can determine the module graph at build time.

CommonJS

CommonJS can do:

```
const moduleName = condition
  ? "./moduleA"
  : "./moduleB";
```

```
const module = require(moduleName);
```

The exact dependency may not be known until runtime.

Therefore, reliable static analysis is more difficult.

Interview answer

Tree shaking removes unused code from the final bundle. ESM enables effective tree shaking because imports and exports are statically analyzable, while CommonJS allows dynamic `require()` calls that make the dependency graph harder to determine at build time.

12B. Tooling & Environment — Interview Questions & Answers

Q366. What is the difference between Babel, SWC, and esbuild?

Answer:

All three can transform JavaScript or TypeScript, but they differ in architecture, capabilities, and performance.

Babel

Babel is a highly configurable JavaScript transformation tool.

It can transform:

Modern JavaScript

JSX

TypeScript syntax

Custom syntax

Its biggest advantage is its huge plugin ecosystem.

The trade-off is performance compared with newer Rust/Go-based tools.

SWC

SWC is written in Rust and focuses on fast compilation.

It supports:

- JavaScript
- TypeScript
- JSX
- Minification
- Modern transformations

It is heavily used in modern frameworks such as Next.js.

esbuild

esbuild is written in Go and is designed primarily for extremely fast builds and transformations.

It is commonly used in tooling such as Vite.

Comparison

	Babel	SWC	esbuild
Written in	JavaScript	Rust	Go
Speed	Slower	Very fast	Extremely fast
Plugin ecosystem	Very large	Smaller	More limited
JSX	Yes	Yes	Yes
TypeScript syntax	Yes	Yes	Yes
Main strength	Flexibility	Speed + features	Speed

When would I choose each?

Babel:

When I need a specialized transformation or an existing plugin ecosystem.

SWC:

When working with Next.js or when I need fast transformations with strong JavaScript/TypeScript support.

esbuild:

When transformation speed is the priority and I don't need Babel's extensive plugin ecosystem.

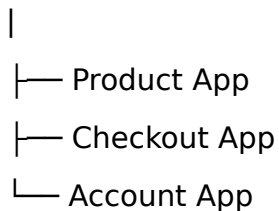
Q367. What is Module Federation? How does it enable micro-frontends?

Answer:

Module Federation allows independently built applications to **share modules at runtime**.

Imagine an organization with:

Shell Application



Different teams can build and deploy these applications independently.

The shell can consume a remote component:

```
const ProductList = lazy(
  () => import("productApp/ProductList")
);
```

The Product App can be deployed separately.

Traditional approach

Without federation:

Build Product App



Build Shell



Product App becomes part of Shell build



Deploy Shell

This creates stronger coupling.

Module Federation

With federation:

Shell deployed independently

+

Product App deployed independently

↓

Shell downloads ProductList at runtime

Shared dependencies

Common dependencies can be shared:

shared: ["react", "react-dom"]

This can avoid unnecessarily loading multiple copies.

Benefits

- Independent deployment
- Team ownership
- Runtime module sharing
- Gradual migration
- Micro-frontend architecture

Challenges

- Version compatibility
- Shared dependency management
- CSS isolation
- Runtime failures
- Debugging complexity
- Cross-application state management

When would I use it?

I would consider Module Federation for a **large organization with independently owned and independently deployed frontend domains**.

I wouldn't introduce it into a small application simply because micro-frontends sound architecturally advanced.

Q368. What are the main challenges of Module Federation?

Answer:

The biggest challenge is that we introduce **runtime coupling between independently deployed applications**.

1. Dependency versions

Suppose:

Shell → React 18

Product App → React 19

We need to carefully manage compatibility.

2. Shared state

Different micro-frontends may need:

User

Cart

Authentication

Feature flags

Sharing this state across independently deployed applications can become complicated.

3. CSS isolation

Styles from one micro-frontend can accidentally affect another.

Solutions include:

- CSS Modules
- Naming conventions
- Shadow DOM
- Scoped styling approaches

4. Runtime failures

A remote application may be unavailable:

Shell

↓

Remote unavailable ❌

The shell needs graceful fallbacks.

5. Debugging

A user interaction can cross:

Shell

↓

Remote A

↓

Shared library

↓

Remote B

Debugging becomes harder than debugging a single application.

Interview answer

Module Federation provides deployment independence, but it moves some complexity from build time to runtime. The biggest concerns are dependency compatibility, shared state, styling, remote failures, and cross-application debugging.

Q369. What's the difference between npm, Yarn, and pnpm? What are phantom dependencies?

Answer:

All three are JavaScript package managers.

They handle:

- Installing packages
- Dependency resolution
- Lockfiles
- Scripts
- Workspaces

The main difference is how they manage dependencies on disk.

npm

npm is the default package manager distributed with Node.js.

It has broad ecosystem compatibility and is usually the easiest choice for beginners.

pnpm

pnpm uses a content-addressable store.

Instead of keeping a completely separate physical copy of every dependency for every project, packages can be stored centrally and linked into projects.

This can save significant disk space.

pnpm also uses a stricter dependency structure.

Yarn

Yarn has several generations. Modern Yarn, especially Yarn Berry, supports Plug'n'Play (PnP), where dependencies can be resolved without the traditional `node_modules` layout.

What is a phantom dependency?

Suppose:

my-app

└─ react

└─ scheduler

Your package.json contains:

```
{
  "dependencies": {
    "react": "..."
  }
}
```

But your application does:

```
import scheduler from "scheduler";
```

You never directly declared scheduler.

If your package manager's layout makes scheduler accidentally resolvable, your application has a **phantom dependency**.

The problem is:

React changes its dependencies

↓

scheduler is no longer installed

↓

Your application breaks

The correct solution is to explicitly declare:

```
{
  "dependencies": {
    "scheduler": "..."
  }
}
```

if your application directly uses it.

Interview answer

A phantom dependency is a package that my application uses without declaring it as a direct dependency. Strict dependency resolution helps expose these problems early.

Q370. Why are lockfiles important?

Answer:

A lockfile records the exact dependency versions and resolved package information used to install the project.

For example:

```
// package.json
{
  "react": "^18.0.0"
}
```

The range allows multiple compatible versions.

The lockfile can resolve that range to:

react → 18.3.1

Without a lockfile:

Developer A → React 18.3.1

Developer B → React 18.3.2

CI → React 18.3.3

This can create:

- Different builds
- Unexpected bugs
- Difficult-to-reproduce failures

With a committed lockfile:

Developer A 

Developer B  → same resolved dependency versions

CI 

Interview answer

A lockfile makes dependency installation reproducible by recording exact resolved versions. I commit it to the repository and use CI commands that respect the lockfile.

For example:

npm ci

is intended for clean, reproducible CI installations.

12B. Tooling & Environment — Continued

Q371. What is a monorepo? What are its advantages?

Answer:

A **monorepo** is a single repository containing multiple applications and/or packages.

For example:

company/

├─ apps/

| └─ web/

| └─ admin/

| └─ mobile/

|

└─ packages/

└─ ui/

└─ utils/

└─ config/

The web and admin applications can share:

packages/ui

packages/utils

packages/config

Advantages

1. Code sharing

Teams can directly consume shared packages.

2. Atomic changes

A shared library and its consumers can be updated in one commit.

3. Consistent tooling

Common:

- ESLint
- TypeScript
- Testing

- CI/CD

can be shared.

4. Dependency management

Workspace tooling can manage dependencies across applications.

5. Easier refactoring

A breaking API change can be updated across consumers in the same repository.

Disadvantages

- Repository complexity
- Larger CI workloads
- Dependency management complexity
- Potentially slower repository operations at very large scale
- Need for good task caching and affected-project analysis

Interview answer

A monorepo centralizes multiple applications and shared packages in one repository. It improves code sharing, atomic changes, and tooling consistency, but requires good dependency management and build orchestration.

Q372. What is Turborepo? How is it different from Nx?

Answer:

Both Turborepo and Nx help manage large repositories containing multiple projects.

Turborepo

Turborepo focuses primarily on **task orchestration and caching**.

For example:

turbo run build

turbo run test

turbo run lint

It can determine dependencies between tasks and cache their results.

Example:

packages/ui

↓

apps/web

If ui hasn't changed and its build result is cached:

Build ui → Cache hit

Build web → Build only what is necessary

Nx

Nx provides a broader monorepo platform.

It includes:

- Dependency graph
- Affected project analysis
- Generators
- Plugins
- Task orchestration
- Caching

For example:

`nx graph`

visualizes project relationships.

And:

`nx affected --target=test`

can run tests only for affected projects.

Comparison

Feature	Turborepo	Nx
Task orchestration	Yes	Yes
Caching	Yes	Yes
Remote caching	Yes	Yes
Dependency graph	Good	Strong
Generators	Limited/not core	Strong
Plugins	Smaller ecosystem	Large
Philosophy	Lightweight	Full-featured

Interview answer

I'd choose Turborepo when I want a lightweight task runner and caching layer, especially for an existing workspace. I'd consider Nx when I need advanced project graph analysis, generators, plugins, and affected-project workflows.

Q373. What are source maps? Why are they important in production debugging?

Answer:

A source map connects generated production code back to the original source code.

Suppose production contains:

app.a8f93d.js

A stack trace might initially say:

app.a8f93d.js:1:42892

That's difficult to debug.

A source map allows tooling to translate this into something like:

Checkout.tsx:87

Typical output

dist/

└─ app.a8f93d.js

└─ app.a8f93d.js.map

The JavaScript can contain:

```
//# sourceMappingURL=app.a8f93d.js.map
```

Error monitoring

Tools such as Sentry can use uploaded source maps to translate:

Minified stack trace

↓

Source map

↓

Original source location

This makes production errors much easier to investigate.

Security concern

Source maps can contain original source code.

If publicly exposed, users may be able to inspect:

- Business logic
- Internal file names
- Source code

- Implementation details

Therefore, for production applications, a common approach is:

Generate source maps



Upload to Sentry



Do NOT publicly serve them

Interview answer

Source maps improve production debugging by mapping minified code back to the original source. However, because they can expose source code, I prefer generating them for error monitoring while preventing public access.

Q374. What are frontend environment variables? Are they secret?

Answer:

Frontend environment variables are **not secrets**.

This is one of the most important frontend security concepts.

Consider:

VITE_API_URL=https://api.example.com

Vite can replace:

```
import.meta.env.VITE_API_URL
```

with the actual value during the build.

The final JavaScript sent to the browser contains that value.

Therefore:

Frontend environment variable



Build



JavaScript bundle



Browser



User can inspect it

Safe values

It's generally fine to expose:

API base URL

Public analytics ID

Public feature flags

Public publishable keys

Never expose

Never put these in client-exposed variables:

Database passwords

JWT signing secrets

Private API keys

Private tokens

Cloud credentials

For example:

`NEXT_PUBLIC_STRIPE_SECRET_KEY=sk_live_...`

would be a serious security mistake.

Framework conventions

Examples:

Vite → `VITE_`

Next.js → `NEXT_PUBLIC_`

CRA → `REACT_APP_`

These prefixes indicate values intended to be exposed to browser code.

Interview answer

Frontend environment variables are configuration, not secrets. Anything exposed to client-side code should be considered public because users can inspect the resulting JavaScript bundle.

Q375. What is the difference between build-time and runtime configuration in frontend applications?

Answer:

Build-time configuration

The value is injected during the build.

For example:

`VITE_API_URL=https://api.example.com`

Then:

`npm run build`

The value becomes part of the generated application.

If we change:

`https://api.example.com`

to:

`https://api-staging.example.com`

we generally need another build.

Runtime configuration

The application fetches configuration when it starts.

For example:

```
const config = await fetch("/config.json")
  .then(res => res.json());
```

`config.json`:

```
{
  "apiUrl": "https://api.example.com"
}
```

Now the server can change the configuration without rebuilding the JavaScript bundle.

Why runtime configuration can be useful

Imagine the same frontend image is deployed to:

Development

Staging

Production

Instead of creating three different frontend builds, the same artifact can load different runtime configuration.

Interview answer

Build-time configuration is embedded during compilation, so changing it generally requires a rebuild. Runtime configuration is loaded when the application starts, allowing configuration changes without rebuilding the frontend artifact.

Q376. What DevTools features should a senior frontend engineer know?

Answer:

I would focus on using DevTools to **diagnose problems**, rather than simply knowing the names of the panels.

1. Network

I use it to investigate:

- API failures
- Slow requests
- Request waterfalls
- Caching
- Headers
- Response payloads
- Authentication problems

Useful features include:

Throttling

Disable cache

Copy as fetch

Copy as cURL

Request blocking

2. Performance

The Performance panel helps identify:

- Long JavaScript tasks
- Expensive rendering
- Layout work
- Paint operations
- Frame drops

A task longer than roughly 50ms can contribute to responsiveness problems.

3. Memory

Heap snapshots help investigate memory leaks.

For example:

Take snapshot

↓

Open/close modal repeatedly

↓

Take snapshot



Compare

If detached DOM nodes or retained objects continue growing, we investigate references and event listeners.

4. React DevTools

For React applications, the Profiler helps identify:

- Expensive renders
- Frequently rendering components
- Prop/state changes
- Components contributing to slow interactions

5. Lighthouse

Lighthouse provides audits for:

- Performance
- Accessibility
- SEO
- Best practices

6. Console

Useful features include:

```
console.table(data);
```

```
console.time("operation");
```

```
// ...
```

```
console.timeEnd("operation");
```

Chrome also provides useful debugging helpers such as:

```
monitorEvents(element);
```

Interview answer

I use DevTools systematically: Network for request problems, Performance for runtime bottlenecks, Memory for leaks, React Profiler for component rendering issues, and Lighthouse for broader performance and accessibility audits. I measure first and optimize the actual bottleneck rather than guessing.

13. Code Management — Interview Questions & Answers

🔴 **Q377. What are the trade-offs between a monorepo and a polyrepo? How do you decide?**

Answer:

A **monorepo** keeps multiple applications and shared packages in a single repository, while a **polyrepo** keeps each application or package in a separate repository.

Monorepo

company/

├─ apps/

| └─ web/

| └─ admin/

| └─ mobile/

|

└─ packages/

└─ ui/

└─ utils/

└─ api-client/

Polyrepo

web-app/

admin-app/

design-system/

api-server/

shared-utils/

Monorepo advantages

- Easy code sharing
- Atomic changes across applications
- Consistent tooling and dependencies
- Easier refactoring across projects
- One PR can update shared code and its consumers
- Better visibility into dependencies

For example, if I change a shared Button component, I can update the component and all consuming applications in the same PR.

Monorepo disadvantages

- Repository can become large
- CI can become expensive

- Dependency management becomes more complex
- IDE performance can suffer
- Requires tooling such as Turborepo, Nx, or similar solutions
- Teams need clear ownership boundaries

Polyrepo advantages

- Strong team ownership
- Independent releases
- Smaller repositories
- Independent technology choices
- Easier permission boundaries

Polyrepo disadvantages

- Shared code requires package publishing
- Dependency/version coordination becomes harder
- Cross-repository changes require multiple PRs
- More duplicated configuration
- Keeping multiple repositories consistent is difficult

How I decide

My main decision factor is **how tightly coupled the projects are**.

I prefer a **monorepo** when:

- Multiple applications share significant code.
- Teams frequently make cross-project changes.
- A shared design system is heavily used.
- Atomic changes are important.

I prefer **polyrepos** when:

- Teams are highly independent.
- Projects have different release cycles.
- Technology stacks are significantly different.
- There are strong organizational or security boundaries.

Interview-ready answer

"I choose based on coupling and release independence. If applications share code frequently and need coordinated changes, I'd prefer a monorepo, typically with Turborepo or Nx. If teams are organizationally independent and services

have independent lifecycles, polyrepos can be better. The important thing is not choosing one universally, but minimizing coordination cost."

● Q378. What is the difference between trunk-based development and Gitflow?

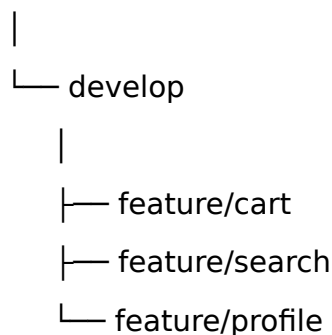
Answer:

Both are Git branching strategies, but they optimize for different development models.

Gitflow

Gitflow typically has long-lived branches such as:

main



There may also be:

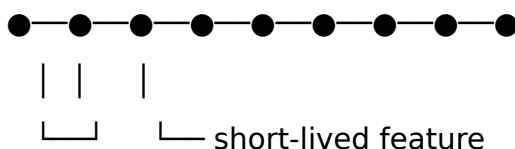
- release/*
- hotfix/*
- main
- develop

Features are developed separately and merged into develop, followed by a release process.

Trunk-based development

Developers integrate their work into the main branch frequently.

main



Feature branches are usually very short-lived.

Comparison

Area	Trunk-based	Gitflow
Main branch	Frequently integrated	Less frequent integration
Feature branches	Short-lived	Can be long-lived
Releases	Frequent	Structured releases
Merge conflicts	Generally lower	Can become significant
CI/CD	Excellent fit	More complex
Feature flags	Common	Less essential
Release branches	Usually unnecessary	Common
Best for	Continuous delivery	Formal release cycles

Why trunk-based development reduces merge problems

Suppose a developer keeps a branch open for three weeks.

During those three weeks:

main



↑

old branch

The main branch has changed significantly.

When the developer finally merges, they may encounter:

- Merge conflicts
- Changed APIs
- Changed components
- Changed dependencies
- Behavioral incompatibilities

With trunk-based development, changes are integrated frequently, so the divergence is much smaller.

Where feature flags help

A feature can be merged into main even when it isn't ready for everyone.

```
const enabled = useFeatureFlag("new-checkout");
```

return enabled

? <NewCheckout />

: <LegacyCheckout />;

The code is deployed, but the feature can initially be enabled only for:

Internal users

↓

1%

↓

10%

↓

50%

↓

100%

This separates **deployment** from **feature release**.

Interview-ready answer

"I generally prefer trunk-based development for modern frontend teams because it encourages small changes, frequent integration, and continuous delivery. Feature flags allow incomplete features to be merged safely without exposing them to all users. Gitflow can still make sense when an organization has formal release branches or needs to maintain multiple release versions."

● **Q379. How does CI/CD work for a frontend application? What would you put in the pipeline?**

Answer:

For frontend applications, I would divide the pipeline into **PR validation**, **production deployment**, and **post-deployment verification**.

A typical pipeline is:

Developer

↓

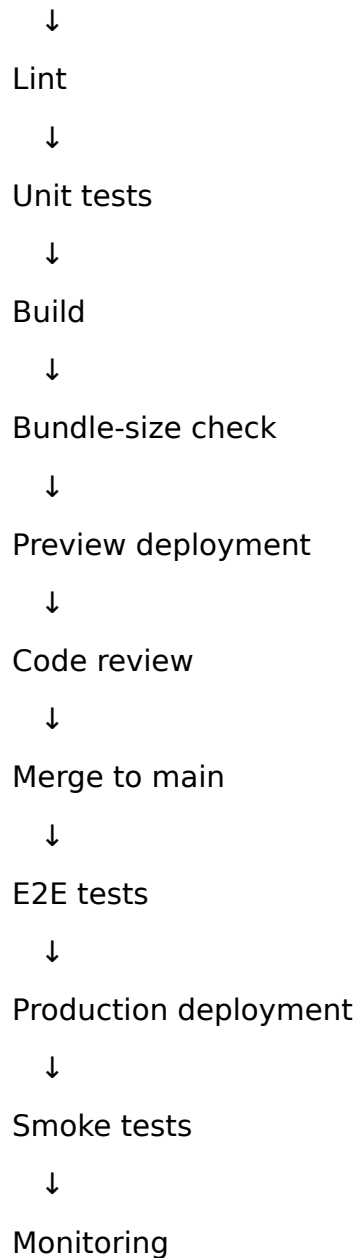
Pull Request

↓

Install dependencies

↓

Type check



1. Install dependencies

Use the lockfile to ensure reproducible installations.

`npm ci`

I prefer `npm ci` in CI rather than `npm install`.

2. Type checking

`npx tsc --noEmit`

This catches TypeScript errors without generating JavaScript.

3. Linting

`npm run lint`

This catches things such as:

- Invalid patterns
- Hook violations
- Unused variables
- Accessibility problems
- Import issues

4. Tests

npm test

Depending on the application:

- Unit tests
- Integration tests
- Component tests

5. Build

npm run build

This ensures the production build actually succeeds.

6. Bundle-size checks

I would monitor JavaScript bundle size because a seemingly small dependency can significantly increase the shipped JavaScript.

For example:

main.js

Before: 180 KB

After: 420 KB

The CI pipeline should flag unexpected increases.

7. Preview deployment

Every PR can receive a unique deployment:

PR #377

↓

preview.example.com/pr-377

This allows developers, QA, designers, and product teams to test the actual built application before merging.

8. E2E testing

After merging to main, critical flows can be tested with tools such as Playwright:

Login

Search

Add to cart

Checkout

Logout

9. Production deployment

After all checks pass:

main

↓

Build

↓

CDN / Hosting

10. Post-deployment verification

I would run smoke tests such as:

HTTP 200?

Application loads?

Critical API available?

JavaScript errors?

Error rate normal?

Monitoring tools can then detect problems.

Important principle

CI/CD should **fail fast**.

For example:

Type check

↓

Lint

↓

Tests

↓

Build

↓

E2E

↓

Deploy

There's no point running expensive E2E tests if TypeScript compilation already fails.

Interview-ready answer

"For frontend CI/CD, I typically run dependency installation, TypeScript checks, linting, unit tests, production build, and bundle-size checks on every PR. I also like preview deployments so reviewers can test the real application. After merging to main, I run critical E2E tests, deploy to the CDN or hosting platform, run smoke tests, and monitor errors. The exact pipeline depends on the application's risk and release frequency."

● Q380. What code-quality tools would you use in a production frontend project?

Answer:

I use multiple layers of automated quality checks rather than depending entirely on code review.

A typical stack is:

TypeScript

↓

ESLint

↓

Prettier

↓

Husky + lint-staged

↓

Commitlint

↓

CI/CD

↓

Tests + Build + Performance checks

1. TypeScript

TypeScript catches problems such as:

```
type User = {  
  id: number;  
};
```

```
const user: User = {  
  id: "123", // Type error  
};
```

It provides compile-time safety.

2. ESLint

ESLint enforces coding rules and detects potential problems.

For example:

```
const user = getUser();
```

```
console.log(user);
```

A project may configure:

```
"no-console": "warn"
```

It can also enforce:

- React rules
 - Hooks rules
 - Accessibility
 - Import restrictions
 - TypeScript best practices
-

3. Prettier

Prettier handles formatting.

Instead of developers debating:

```
const user={name:"John",age:30}
```

versus:

```
const user = {  
  name: 'John',  
  age: 30,  
};
```

the formatter decides automatically.

This keeps code reviews focused on logic rather than formatting.

4. Husky

Husky allows Git hooks to run automatically.

For example:

git commit

↓

Husky

↓

lint-staged

↓

ESLint + Prettier

5. lint-staged

Instead of linting the entire project, it can check only modified files.

Example:

```
{
  "lint-staged": {
    "*.ts,tsx": [
      "eslint --fix",
      "prettier --write"
    ]
  }
}
```

This keeps pre-commit checks fast.

6. Commitlint

Commitlint can enforce Conventional Commits.

feat(cart): add quantity selector

fix(auth): handle expired token

chore(deps): update React

This makes Git history more consistent and enables automated changelogs and semantic versioning.

7. CI

Local checks aren't enough because developers can bypass Git hooks.

Therefore CI remains the final gate:

PR

↓

TypeScript

↓

ESLint

↓

Tests

↓

Build

↓

Bundle checks

↓

Merge

Why multiple layers?

Each layer catches problems at a different stage:

Layer	Purpose
Editor	Immediate developer feedback
TypeScript	Type safety
ESLint	Code correctness/patterns
Prettier	Formatting
Pre-commit	Local quality gate
CI	Repository-level quality gate
Tests	Behavioral correctness
Build	Production validity
E2E	User-flow validation

Interview-ready answer

"I don't rely on one code-quality tool. I use TypeScript for type safety, ESLint for code and React rules, Prettier for formatting, Husky and lint-staged for fast local checks, Commitlint for consistent commits, and CI as the final enforcement layer. The goal is to automate repetitive checks so code reviews can focus on architecture, business logic, performance, and UX."

★ Quick Revision

Q377 — Monorepo vs Polyrepo

Remember:

Shared code + coordinated changes → Monorepo.

Independent teams + independent releases → Polyrepo.

Q378 — Trunk-based vs Gitflow

Remember:

Frequent integration + short-lived branches → Trunk-based.

Formal release branches + multiple supported versions → Gitflow.

Feature flags make trunk-based development practical for incomplete features.

Q379 — Frontend CI/CD

Remember:

Install

↓

Type check

↓

Lint

↓

Test

↓

Build

↓

Bundle check

↓

Preview

↓

E2E

↓

Deploy

↓

Smoke test

↓

Monitor

Q380 — Code Quality

Remember:

TypeScript → type safety

ESLint → code quality

Prettier → formatting

Husky → Git hooks

lint-staged → changed files

Commitlint → commit convention

CI → final quality gate

14A. App Router & Rendering

Q381. How does the Next.js App Router work?

Answer:

The Next.js App Router is a **file-system-based routing system** introduced in Next.js 13.

The folder structure inside the app directory determines the URL structure, and special files provide layouts, loading states, errors, and pages.

For example:

app/

├─ layout.tsx

├─ page.tsx

├─ loading.tsx

├─ error.tsx

├─ not-found.tsx

|

├─ products/

```

|   ├── layout.tsx
|   ├── page.tsx
|   └── [slug]/
|       └── page.tsx
|
└── (auth)/
    ├── login/
    |   └── page.tsx
    └── signup/
        └── page.tsx

```

This creates routes such as:

```

/                → app/page.tsx
/products        → app/products/page.tsx
/products/shoes  → app/products/[slug]/page.tsx
/login           → app/(auth)/login/page.tsx

```

Important conventions are:

- page.tsx → defines a route
- layout.tsx → shared UI around routes
- loading.tsx → loading UI using Suspense
- error.tsx → error boundary
- not-found.tsx → 404 UI
- [slug] → dynamic route
- (auth) → route group that doesn't appear in the URL

Interview point

One major advantage of the App Router is that **layouts persist across navigation**.

For example, navigating from:

/products

to:

/products/shoes

doesn't require the entire application layout to be recreated. The shared layouts remain mounted while the inner page changes.

Q382. What is the purpose of layout.tsx in the Next.js App Router?

Answer:

layout.tsx defines UI that is shared across multiple routes.

For example:

```
export default function ProductsLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="flex">
      <aside>Filters</aside>

      <main>{children}</main>
    </div>
  );
}
```

If this exists at:

app/products/layout.tsx

it wraps:

/products

/products/shoes

/products/laptops

The important feature is that **layouts persist across navigation**.

This makes layouts useful for:

- Navigation bars
 - Sidebars
 - Headers
 - Authentication wrappers
 - Dashboards
 - Shared filters
-

Q383. What is loading.tsx in Next.js?

Answer:

loading.tsx provides the loading UI for a route segment.

Next.js automatically uses it as a **Suspense fallback** while the route's content is loading.

Example:

```
// app/products/loading.tsx
```

```
export default function Loading() {  
  return <div>Loading products...</div>;  
}
```

When the user navigates to /products, Next.js can immediately display the loading UI while the actual page is being rendered.

This works particularly well with **streaming**, where parts of the UI can be sent to the browser as they become ready.

Interview answer

"loading.tsx is a route-level loading boundary. Next.js automatically wraps the relevant content in Suspense and displays the loading UI while that segment is being rendered."

Q384. What is error.tsx and how does error handling work in the App Router?

Answer:

error.tsx defines an **error boundary** for a route segment.

Example:

```
'use client';
```

```
export default function Error({  
  error,  
  reset,  
}): {  
  error: Error;  
  reset: () => void;  
}) {
```

```

return (
  <div>
    <h2>Something went wrong</h2>

    <button onClick={() => reset()}>
      Try again
    </button>
  </div>
);
}

```

The component must be a **Client Component** because error boundaries require client-side behavior.

If a child page throws an error, the nearest error.tsx can display the fallback UI instead of crashing the entire application.

The parent layout can remain available.

Key idea

Root Layout

↓

Products Layout

↓

Error Boundary

↓

Product Page ❌

Only the failed segment needs to show the error state.

Q385. What are dynamic routes in Next.js?

Answer:

Dynamic routes allow part of the URL to be captured as a parameter.

For example:

app/products/[slug]/page.tsx

matches:

/products/iphone

/products/laptop

/products/shoes

The slug can then be accessed from params.

```
export default async function ProductPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;

  return <h1>Product: {slug}</h1>;
}
```

Dynamic routes are useful for:

- Product pages
 - User profiles
 - Blog posts
 - Documentation
 - Orders
-

Q386. What are route groups in Next.js?

Answer:

Route groups allow us to organize routes without affecting the URL.

They are created using parentheses:

```
app/
└─ (auth)/
    └─ login/
        └─ signup/
```

The resulting URLs are:

/login

/signup

not:

/auth/login

/auth/signup

Route groups are useful when we want to share layouts or organize routes by functionality.

Q387. What are parallel routes in Next.js?

Answer:

Parallel routes allow multiple route segments to render simultaneously within the same layout.

They use the @ convention.

Example:

```
app/  
├─ layout.tsx  
├─ @sidebar/  
│   └─ page.tsx  
└─ @main/  
    └─ page.tsx
```

They are useful for UI structures such as:

- Dashboard panels
- Sidebars
- Modals
- Independent sections
- Multi-pane applications

For example, a dashboard could independently render:

Dashboard

```
├─ Sidebar  
├─ Main Content  
└─ Notifications
```

Q388. What are intercepting routes?

Answer:

Intercepting routes allow Next.js to render a route in a different UI context depending on how the user navigates to it.

A common example is a photo application.

If the user clicks a photo from a gallery:

/gallery

↓ click

/photo/123

the application can intercept that navigation and show:

Gallery + Photo Modal

But if the user directly visits:

/photo/123

they can see:

Full Photo Page

This is particularly useful for:

- Photo modals
 - Product quick views
 - Login dialogs
 - Nested modal navigation
-

Server vs Client Components

Q389. What is the difference between Server Components and Client Components in Next.js?

Answer:

In the App Router, components are **Server Components by default**.

A component becomes a Client Component when we add:

'use client';

Server Components

Can:

- Fetch data directly
- Access databases
- Access server-side resources
- Use async/await
- Keep large dependencies out of the client bundle

Cannot:

- Use useState
- Use useEffect

- Use browser APIs
- Use event handlers such as onClick

Client Components

Can:

- Use React hooks
- Handle user interactions
- Use browser APIs
- Maintain client-side state
- Add event handlers

Example:

```
'use client';
```

```
import { useState } from 'react';
```

```
export default function Counter() {
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```
    <button onClick={() => setCount(count + 1)}>
```

```
      {count}
```

```
    </button>
```

```
  );
```

```
}
```

Interview rule

"I keep components as Server Components by default and push the Client Component boundary as deep as possible."

This reduces the amount of JavaScript sent to the browser.

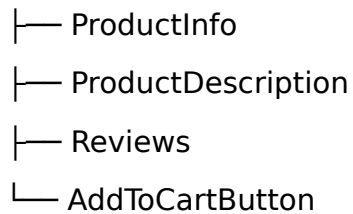
Q390. Why should we push the "use client" boundary as deep as possible?

Answer:

Because making a high-level component a Client Component can cause a much larger portion of the component tree to participate in the client-side JavaScript bundle.

Suppose a product page contains:

ProductPage



Only AddToCartButton requires interactivity.

Instead of:

```
'use client';
```

```
export default function ProductPage() {
  // entire page becomes client-oriented
}
```

we can keep:

```
export default async function ProductPage() {
  const product = await getProduct();

  return (
    <>
      <ProductInfo product={product} />
      <ProductDescription product={product} />
      <AddToCartButton id={product.id} />
    </>
  );
}
```

where only:

```
'use client';
```

```
function AddToCartButton() {
  // interactive logic
}
```

```
}
```

is client-side.

Benefit

- Smaller client bundle
 - Less JavaScript
 - Better performance
 - Better server-side data access
 - Faster initial rendering
-

Q391. Can a Client Component directly import a Server Component?

Answer:

No, a Client Component cannot directly import a Server Component as a normal child import.

However, a Server Component can pass a Server Component into a Client Component through children or another supported prop.

Example:

```
<ClientLayout>  
  <ServerContent />  
</ClientLayout>
```

Here:

- ServerContent remains a Server Component.
- ClientLayout is interactive.
- The server-rendered content is passed into the client component composition.

This allows us to combine server-rendered content with client-side interactive shells.

Data Fetching & Caching

Q392. How does data fetching work in the Next.js App Router?

Answer:

In the App Router, data can be fetched directly inside Server Components.

For example:

```
export default async function ProductsPage() {
```

```
const response = await fetch(
  'https://api.example.com/products'
);

const products = await response.json();

return <ProductList products={products} />;
}
```

We don't need the Pages Router APIs:

getServerSideProps

getStaticProps

Instead, the App Router uses:

- Server Components
 - fetch
 - Cache configuration
 - Revalidation
 - Server Actions
 - Route Handlers
-

Q393. What are the different caching strategies for fetch in Next.js?

Answer:

The main strategies are:

1. Cached request

```
fetch(url, {
  cache: 'force-cache',
});
```

The response can be cached and reused.

2. Dynamic request

```
fetch(url, {
  cache: 'no-store',
});
```

The request is performed without using the persistent Data Cache.

Useful for frequently changing data.

3. Time-based revalidation

```
fetch(url, {  
  next: {  
    revalidate: 60,  
  },  
});
```

The cached data can be revalidated after 60 seconds.

This is useful when we want a balance between freshness and performance.

Interview warning

Be careful with statements such as "**fetch is cached by default.**"

Next.js caching behavior has changed across versions, so in a real interview I would mention the specific Next.js version when discussing defaults.

Q394. What is ISR in Next.js?

Answer:

ISR stands for **Incremental Static Regeneration**.

It allows statically generated content to be revalidated after a specific period instead of rebuilding the entire application.

Example:

```
const products = await fetch(  
  'https://api.example.com/products',  
  {  
    next: {  
      revalidate: 60,  
    },  
  }  
);
```

Conceptually:

Request
↓
Cached page/data
↓

60 seconds



Revalidation



Updated content

ISR is useful for:

- E-commerce catalogs
 - Blogs
 - News pages
 - Product pages
 - Content that changes periodically
-

Q395. What is the difference between `revalidatePath()` and `revalidateTag()`?

Answer:

`revalidatePath()` invalidates cached data associated with a specific route.

```
revalidatePath('/products');
```

`revalidateTag()` invalidates cached data associated with a specific cache tag.

```
revalidateTag('products');
```

For example:

```
fetch('/api/products', {  
  next: {  
    tags: ['products'],  
  },  
});
```

Then after updating a product:

```
revalidateTag('products');
```

This is useful when multiple pages depend on the same underlying data.

Easy way to remember

`revalidatePath` → "Refresh this route"

`revalidateTag` → "Refresh everything using this data tag"

Q396. How do you cache database queries in Next.js?

Answer:

For non-fetch operations such as database queries, Next.js provides caching APIs such as `unstable_cache` in supported versions.

For example:

```
const getProducts = unstable_cache(
  async () => {
    return db.product.findMany();
  },
  ['products'],
  {
    revalidate: 3600,
  }
);
```

This allows expensive database operations to be cached and revalidated.

The exact caching API can vary with the Next.js version, so in production I would follow the current Next.js caching documentation for the installed version.

Q397. Does Next.js automatically deduplicate identical fetch requests?

Answer:

Next.js and React provide request deduplication/memoization mechanisms for appropriate server-side fetch patterns.

If multiple Server Components request the same resource during a render, we should avoid assuming that each component necessarily causes a separate network request.

For example:

Header

↓

`fetch(/api/user)`

Sidebar

↓

`fetch(/api/user)`

Page

↓

```
fetch(/api/user)
```

The framework can deduplicate appropriate identical requests during rendering.

Interview-safe answer

"Next.js provides request memoization for server-side fetches, so components can request the same resource without necessarily causing redundant network requests."

Server Actions

Q398. What are Server Actions in Next.js?

Answer:

Server Actions are asynchronous functions that execute on the server and are designed primarily for **mutations and server-side operations**.

They are defined using:

```
'use server';
```

Example:

```
'use server';
```

```
export async function createProduct(formData: FormData) {
```

```
  const name = formData.get('name');
```

```
  await db.product.create({
```

```
    data: {
```

```
      name: String(name),
```

```
    },
```

```
  });
```

```
}
```

They are especially useful for:

- Form submissions
- Create operations
- Update operations
- Delete operations

- Database mutations
 - Revalidating cached data
-

Q399. How do you use Server Actions with forms?

Answer:

A Server Action can be passed directly to a form's action.

```
<form action={createProduct}>  
  <input name="name" />  
  <button type="submit">  
    Create  
  </button>  
</form>
```

The browser submits the form to the server action.

This provides a convenient way to build forms without manually creating an API endpoint for every simple mutation.

Q400. What is progressive enhancement with Server Actions?

Answer:

Progressive enhancement means the form can still function even if client-side JavaScript hasn't loaded.

For example:

```
<form action={submitReview}>  
  <textarea name="comment" />  
  <button type="submit">  
    Submit  
  </button>  
</form>
```

The server action handles the submission.

With JavaScript available, Next.js can enhance the experience without requiring a traditional full-page navigation.

Interview answer

"Server Actions integrate closely with HTML forms and support progressive enhancement, allowing important form functionality to work without depending entirely on client-side JavaScript."

Q401. How do you handle loading and error states for Server Actions?

Answer:

In modern React/Next.js applications, we can use APIs such as `useActionState` to manage action state.

Example:

```
'use client';
```

```
const [state, formAction, isPending] =  
  useActionState(submitReview, null);
```

Then:

```
<form action={formAction}>  
  <textarea name="comment" />  
  
  <button disabled={isPending}>  
    {isPending ? 'Submitting...' : 'Submit'}  
  </button>  
  
  {state?.error && (  
    <p>{state.error}</p>  
  )}  
</form>
```

This gives us:

Form

↓

Server Action

↓

Pending state

↓

Success / Error state

Q402. When would you use Server Actions vs API Routes?

Answer:

I would use **Server Actions** for application-internal mutations such as:

- Creating a record
- Updating a profile
- Deleting an item
- Form submissions
- Updating application state

I would use **Route Handlers/API endpoints** when I need a conventional HTTP API.

For example:

Requirement	Choice
Internal form mutation	Server Action
CRUD mutation from Next.js UI	Server Action
Mobile app consuming API	API Route
Third-party integration	API Route
Webhook	API Route
Public REST API	API Route
Custom HTTP semantics	API Route

Interview answer

"Server Actions are great for application-internal mutations, while Route Handlers are better when I need a public or reusable HTTP API."

Q403. Are Server Actions secure?**Answer:**

Server Actions execute on the server, but that doesn't automatically make the business operation secure.

I still need to:

1. Authenticate the user.
2. Authorize the requested operation.
3. Validate all input.
4. Prevent unauthorized resource access.

5. Handle errors safely.
6. Avoid trusting client-provided IDs or roles.

For example:

```
'use server';
```

```
export async function deleteProduct(productId: string) {  
  const user = await getCurrentUser();
```

```
  if (!user) {  
    throw new Error('Unauthorized');  
  }
```

```
  const parsedId = productIdSchema.parse(productId);
```

```
  // Check authorization before deletion
```

```
  await db.product.delete({  
    where: {  
      id: parsedId,  
    },  
  });  
}
```

Key interview point

"Server Actions are server-side execution mechanisms, not an authorization system. Authentication, authorization, and validation are still my responsibility."

14B. Features & Deployment

Q404. How does Next.js Middleware work?

Answer:

Middleware intercepts requests before they reach the relevant application route and can perform operations such as:

- Redirects
- Rewrites

- Request/response header changes
- Authentication checks
- Routing decisions

For example:

```
import { NextRequest, NextResponse } from 'next/server';
```

```
export function middleware(request: NextRequest) {
  const token = request.cookies.get('session');
```

```
  if (
    request.nextUrl.pathname.startsWith('/dashboard') &&
    !token
  ) {
    return NextResponse.redirect(
      new URL('/login', request.url)
    );
  }
```

```
  return NextResponse.next();
}
```

A common use case is protecting routes before rendering begins.

Q405. What are common use cases for Next.js Middleware?

Answer:

Common use cases include:

Authentication

/dashboard

↓

Middleware

↓

Authenticated?

↙ ↘

Yes No

↓ ↓

Page /login

Other use cases

- Authentication redirects
- Authorization checks
- A/B testing
- Localization
- Geo-based routing
- URL rewriting
- Security headers
- Maintenance mode
- Bot detection

Middleware should remain lightweight because runtime capabilities are more restricted than a normal Node.js environment.

Q406. Can Next.js Middleware use Node.js APIs such as fs?

Answer:

Not when the middleware is running in an Edge-style runtime where Node.js APIs such as fs are unavailable.

For example, this is not appropriate:

```
import fs from 'fs';
```

inside middleware.

If the logic requires:

- File-system access
- Heavy computation
- Native Node modules
- Complex backend processing

I would move that logic to an appropriate server-side location such as:

- Route Handler
- Server Component
- Server-side service
- Backend service

Interview answer

"I keep middleware lightweight and use it mainly for request-level decisions such as redirects, rewrites, authentication checks, and headers."

Images & Fonts

Q407. How does next/image optimize images?

Answer:

next/image provides image optimization features such as:

- Responsive image sizing
- Lazy loading
- Modern image formats where supported
- Image resizing
- CDN/cache integration depending on deployment
- Prevention of layout shift when dimensions/aspect ratio are known

Example:

```
import Image from 'next/image';
```

```
<Image  
  src="/product.jpg"  
  alt="Running shoes"  
  width={400}  
  height={300}  
  sizes="(max-width: 768px) 100vw, 400px"  
>
```

For an important above-the-fold image, modern Next.js applications may use the appropriate priority/preload mechanism for the version being used.

Q408. Why does next/image need width and height?

Answer:

The dimensions allow the browser to reserve the appropriate space before the image finishes loading.

Without reserved space:

Page loads

↓

Image loads

↓

Image takes space

↓

Content moves

This creates **Cumulative Layout Shift (CLS)**.

With dimensions:

Reserved space

↓

Image loads

↓

No major layout movement

Therefore, dimensions help improve visual stability and Core Web Vitals.

Q409. Can next/image load images from an external domain?

Answer:

Yes.

The external source must be configured in next.config.js using appropriate remotePatterns.

Example:

```
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'images.example.com',
      },
    ],
  },
};
```



```
export default nextConfig;
```

This prevents arbitrary external image sources from being optimized through the application.

Q410. What is next/font and why is it useful?

Answer:

next/font provides optimized font loading for Next.js applications.

Example:

```
import { Inter } from 'next/font/google';
```

```
const inter = Inter({  
  subsets: ['latin'],  
});
```

```
export default function RootLayout({  
  children,  
}: {  
  children: React.ReactNode;  
}) {  
  return (  
    <html className={inter.className}>  
      <body>{children}</body>  
    </html>  
  );  
}
```

Benefits include:

- Font optimization
 - Self-hosting of supported Google fonts
 - Reduced dependency on external font requests
 - Better loading behavior
 - Reduced layout shift
-

Deployment

Q411. What are the major deployment options for Next.js?

Answer:

The main options are:

1. **Vercel**
2. **Self-hosted Node.js**
3. **Docker**
4. **Serverless platforms**
5. **Static export**, when the application doesn't require server features

Vercel

Best when we want:

- Simple deployment
- Preview environments
- Automatic CI/CD
- Strong Next.js integration

Node.js

Run:

next build

next start

Useful when we need infrastructure control.

Docker

We can package the Next.js application into a container and deploy it to platforms such as AWS ECS or other container platforms.

Static export

Useful for applications that can be completely pre-rendered.

Q412. What is the difference between Node.js, Serverless, Edge, and Static deployment?

Answer:

Node.js

A persistent Node.js server runs the application.

Good for:

- Full server capabilities

- Long-running applications
- Infrastructure control

Serverless

Functions execute on demand.

Good for:

- Variable traffic
- Automatic scaling
- Pay-per-use architectures

Potential drawback:

- Cold starts
- Execution limits depending on provider

Edge

Code runs closer to users geographically.

Good for:

- Redirects
- Authentication checks
- Lightweight request processing
- Personalization

But runtime APIs can be more restricted.

Static

The application is generated into static assets.

Good for:

- Documentation
- Marketing websites
- Blogs
- Static content

No runtime server is required for the generated output.

Q413. What happens when you use output: 'export' in Next.js?

Answer:

It generates a **static export** of the application.

For example:

```
const nextConfig = {  
  output: 'export',  
};
```

```
export default nextConfig;
```

The build produces static files that can be hosted on services such as:

S3

CloudFront

Netlify

GitHub Pages

However, features that require a Next.js server runtime aren't available in the static export model.

For example, you shouldn't expect normal server-side capabilities such as:

- SSR
- Server Actions
- Runtime API routes
- Server-side middleware
- ISR

to work as they would in a full Next.js server deployment.

Framework Comparison

Q414. When would you choose Next.js over Remix or Astro?

Answer:

I would choose based on the application's requirements rather than simply choosing the most popular framework.

Next.js

I'd choose Next.js for:

- SaaS applications
- E-commerce
- Dashboards
- Full-stack React applications
- Applications needing different rendering strategies

Astro

I'd choose Astro for:

- Documentation
- Blogs
- Marketing sites
- Content-heavy websites

especially when most of the UI doesn't need JavaScript.

Remix / React Router ecosystem

I'd consider it for applications where:

- Web standards are important
- Forms and mutations are central
- Server-side data loading is important
- The team's architecture fits its routing/data model

Interview answer

"For a SaaS dashboard, I'd generally choose Next.js because I may need Server Components, SSR, caching, authentication, and interactive React components. For a mostly static documentation site, I'd consider Astro because minimizing client-side JavaScript is a major advantage."

High-Value Interview Follow-ups

Q415. Why is the App Router considered an improvement over the Pages Router?

Answer:

The App Router provides a more modern architecture around:

- Server Components
- Nested layouts
- Streaming
- Suspense
- Server Actions
- Route-level loading states
- Route-level error boundaries
- More granular rendering

For example, instead of treating every page as one large rendering unit, the App Router lets us structure an application as:

Root Layout

↓

Products Layout

↓

Product Page

↓

Interactive Client Component

This provides more granular control over rendering and client-side JavaScript.

Q416. Explain the rendering flow of a typical Next.js App Router page.

Answer:

A simplified flow is:

Browser Request

↓

Middleware / Request handling

↓

Route matching

↓

Server Components

↓

Data fetching

↓

Rendering

↓

HTML + React Server Component payload

↓

Browser

↓

Client Components hydrate

↓

Interactive UI

The important distinction is that **not everything needs to become client-side JavaScript.**

Server Components can render on the server, while only interactive Client Components need hydration.

Q417. How would you optimize a large Next.js application?

Answer:

I would optimize it at multiple levels.

1. Rendering

Use the appropriate strategy:

Static → stable content

ISR → periodically changing content

SSR → request-specific content

CSR → highly interactive client UI

2. Client JavaScript

Keep:

'use client'

boundaries as deep as possible.

3. Data fetching

Use:

- Appropriate caching
- Revalidation
- Request deduplication
- Parallel data fetching

4. Images

Use:

`next/image`

with appropriate sizing and responsive sizes.

5. Fonts

Use:

`next/font`

6. Code splitting

Use dynamic imports when appropriate:

```
const Chart = dynamic()
```

```
() => import('./Chart')  
);
```

7. Monitoring

Use production monitoring for:

- Core Web Vitals
- Errors
- API latency
- Server performance
- Bundle size

Senior-level answer

"I wouldn't optimize only the React component tree. I'd look at rendering strategy, server/client boundaries, data caching, network requests, JavaScript bundle size, images, fonts, and backend latency as one system."

Q418. How do you build an AI-powered chat interface? What are the key frontend challenges?

Answer:

An AI chat interface is more than a normal chat UI because AI responses are **slow, streamed incrementally, non-deterministic, and often contain Markdown, code, tables, and tool calls.**

A typical architecture is:

User

↓

Chat Input

↓

Frontend

↓

Backend API

↓

AI Model

↓

Streaming Response

↓

Frontend

↓

Incremental UI Update

The frontend generally maintains:

```
const [messages, setMessages] = useState([]);
```

```
const [input, setInput] = useState('');
```

```
const [isStreaming, setIsStreaming] = useState(false);
```

When the user submits a prompt:

1. Add the user message immediately.
2. Create an empty assistant message.
3. Send the conversation to the backend.
4. Receive the AI response as a stream.
5. Append each chunk to the assistant message.
6. Render Markdown/code progressively.
7. Allow the user to stop generation.
8. Handle partial responses and errors.
9. Auto-scroll while the user remains at the bottom.

Important frontend challenges

1. Streaming

Instead of waiting 10 seconds for the complete answer, show tokens as they arrive.

2. Auto-scrolling

The UI should scroll as new content arrives, but if the user manually scrolls upward, automatic scrolling should stop.

3. Markdown rendering

AI responses commonly contain:

- headings
- lists
- tables
- links
- code blocks
- bold/italic text

The frontend needs safe incremental Markdown rendering.

4. Stop generation

Use `AbortController` to cancel an in-progress request.

5. Long conversations

Sending the entire conversation on every request becomes expensive and can eventually exceed the model's context window.

Common strategies are:

- Keep the system prompt + recent messages.
- Summarize older messages.
- Use a sliding window.
- Retrieve only relevant historical messages.

6. Error handling

If streaming fails halfway through, don't throw away the partial response. Show:

Response interrupted — Retry

Interview summary

"I would design the chat UI around streaming rather than treating AI like a normal request-response API. The frontend needs incremental message updates, Markdown rendering, auto-scroll with user override, cancellation using AbortController, robust error handling, and context management for long conversations."

Q419. How does streaming work for AI responses? Explain ReadableStream and EventSource.

Answer:

AI models generate responses incrementally, so streaming allows the frontend to display the answer while the model is still generating it.

There are two common approaches.

1. Server-Sent Events with EventSource

```
const eventSource = new EventSource('/api/chat');
```

```
eventSource.onmessage = (event) => {  
  const chunk = JSON.parse(event.data);
```

```
  updateUI(chunk.text);
```

```
};
```

EventSource is simple because it is designed for server-to-client event streaming.

However, it has important limitations:

- Primarily uses GET.
- Cannot send a request body.
- Limited support for custom request headers.
- Less convenient when sending large conversation histories.

Therefore, it isn't always ideal for an AI chat API.

2. **fetch() + ReadableStream**

For chat applications, `fetch()` with a readable response stream is often more flexible.

```
const controller = new AbortController();
```

```
const response = await fetch('/api/chat', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json',  
  },  
  body: JSON.stringify({  
    messages,  
  }),  
  signal: controller.signal,  
});
```

```
const reader = response.body.getReader();  
const decoder = new TextDecoder();
```

```
while (true) {  
  const { done, value } = await reader.read();
```

```
  if (done) break;
```

```
  const text = decoder.decode(value, {  
    stream: true,
```

```
});
```

```
  updateUI(text);
```

```
}
```

The important point is that a network response doesn't necessarily arrive as one complete string.

Instead:

AI response

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

...

The frontend consumes those chunks and updates the UI incrementally.

AbortController

For a Stop button:

```
const controllerRef = useRef(null);
```

```
async function handleSubmit() {  
  const controller = new AbortController();
```

```
  controllerRef.current = controller;
```

```
  try {  
    await streamResponse(controller.signal);  
  } catch (error) {  
    if (error.name !== 'AbortError') {  
      console.error(error);  
    }  
  } finally {  
    controllerRef.current = null;
```

```
}  
}
```

```
function handleStop() {  
  controllerRef.current?.abort();  
}
```

A ref is useful because the Stop button needs immediate access to the current controller without causing another React render.

Interview summary

"EventSource is simple for server-sent events but restrictive for chat because it doesn't support POST bodies and custom headers conveniently. For AI chat, I generally prefer fetch with ReadableStream because I can send the full conversation, authentication headers, and an AbortSignal while consuming the response incrementally."

Q420. What is prompt engineering for UI? How do structured outputs and function calling work?

Answer:

When AI is integrated into a product UI, free-form text isn't always enough.

For example, suppose I want the AI to return:

```
{  
  "sentiment": "positive",  
  "rating": 5,  
  "pros": ["Fast", "Easy to use"],  
  "cons": [],  
  "summary": "Excellent product"  
}
```

The frontend needs predictable structure to render this reliably.

Structured output

A weak approach is:

Return only valid JSON.

Do not include Markdown.

Do not add explanations.

This relies on the model following the instruction.

A better production approach is **schema-constrained structured output**, where the API validates/constrains the response against a schema.

For example:

```
const schema = {
  type: 'object',
  properties: {
    sentiment: {
      type: 'string',
      enum: ['positive', 'negative', 'neutral'],
    },
    rating: {
      type: 'integer',
    },
    summary: {
      type: 'string',
    },
  },
  required: ['sentiment', 'rating', 'summary'],
};
```

The backend validates the response before sending it to the frontend.

Function calling / tool use

Function calling allows the model to request an operation rather than pretending that it performed the operation.

For example:

User:

What's the weather in Mumbai?

AI:

```
Call get_weather({
  city: "Mumbai"
})
```

The model **doesn't actually call the weather API itself**.

Instead:

User



AI Model



Tool request



Backend executes `get_weather()`



Weather API



Tool result



AI Model



Final answer



Frontend

This distinction is extremely important in interviews.

Frontend implications

Tools allow an AI assistant to interact with an application:

"Show my recent orders"



`get_orders()`

"Add this product to my cart"



`add_to_cart()`

"Schedule a meeting"



`create_calendar_event()`

The backend should execute these operations and enforce authorization.

Validating AI output

Never blindly trust AI output.

A production system should use:

AI response

↓

Parse

↓

Schema validation

↓

Sanitize

↓

Business validation

↓

Render

For example, Zod can validate structured responses:

```
const ReviewSchema = z.object({  
  sentiment: z.enum(['positive', 'negative', 'neutral']),  
  rating: z.number(),  
  summary: z.string(),  
});
```

```
const result = ReviewSchema.safeParse(aiResponse);
```

```
if (!result.success) {  
  // fallback  
}
```

Interview summary

"For UI integration, I prefer structured outputs over relying purely on prompting. I validate the response before rendering it. For actions, I use function calling where the model requests a tool, but my backend actually executes it and enforces authorization."

Q421. What are AI SDKs like Vercel AI SDK? How do they simplify integration?

Answer:

AI SDKs provide abstractions around common AI application concerns such as:

- streaming
- message state
- model integration
- tool calls
- cancellation
- loading states
- error handling

Without an SDK, I may need to manually manage:

fetch()

↓

ReadableStream

↓

Chunk parsing

↓

Message state

↓

AbortController

↓

Loading state

↓

Error handling

An AI SDK can abstract much of this.

A typical architecture is:

React UI

↓

AI SDK hook

↓

Next.js API route

↓

AI provider

For example, conceptually:

```
const {  
  messages,  
  input,  
  handleInputChange,  
  handleSubmit,  
  isLoading,  
  stop,  
} = useChat();
```

The UI can then focus mainly on rendering:

```
{messages.map(message => (  
  <Message  
    key={message.id}  
    role={message.role}  
    content={message.content}  
  />  
))}
```

Benefits

Less boilerplate

The SDK handles much of the streaming and state-management logic.

Provider abstraction

It can make switching between supported model providers easier.

Tool integration

Modern AI SDKs can also simplify tool/function-call workflows.

React integration

Hooks provide a natural interface for React applications.

When I would use an SDK

I'd use an SDK for:

- standard chat
- text generation

- streaming responses
- tool calling
- common AI UI patterns

I'd consider custom implementation when I need:

- unusual streaming protocols
- custom transport
- multiple parallel streams
- specialized chunk processing
- non-React clients

Interview summary

"An AI SDK reduces the amount of infrastructure code I have to write for streaming, state management, cancellation and tool calls. I would use it when my requirements match the abstraction, but I wouldn't introduce it blindly if I need highly customized streaming or transport behavior."

Q422. What UX patterns work for AI features? How do you handle loading, errors, and trust?

Answer:

AI UX is different from traditional application UX because:

- responses can take several seconds
- results aren't deterministic
- responses can be partially generated
- the AI can be incorrect
- users need ways to verify important information

1. Thinking state

Instead of:

Loading...

use something meaningful:

Thinking...

Then transition to:

Generating...

and finally:

Complete

This makes the AI's latency feel intentional.

2. Streaming cursor

Once tokens begin arriving, show the answer progressively:

React is a JavaScript library...



The user doesn't have to stare at a spinner for 10 seconds.

3. Grounding and citations

Don't invent a fake confidence score such as:

AI confidence: 97%

LLM-generated confidence isn't necessarily a calibrated probability.

Instead, show real evidence:

Sources

└─ Documentation

└─ Internal knowledge base

└─ Product specification

This gives the user something they can actually verify.

4. Regenerate

🔄 Regenerate

Useful because AI responses can vary.

5. Edit prompt

Allow the user to modify their original request rather than starting from scratch.

6. Feedback

Provide:

👍 Helpful

👎 Not helpful

This can help measure quality.

7. Error states

Different errors should have different UX.

Problem	UX
Rate limit	Try again in 30 seconds
Timeout	Response taking longer than expected
Network failure	Retry
Streaming interruption	Show partial answer + Retry
Content policy	Explain that the request can't be completed

Important security rule

Never render untrusted AI-generated HTML directly.

If HTML must be rendered:

AI output

↓

Sanitize

↓

Render

For example, DOMPurify can be used where appropriate.

Interview summary

"I design AI UX around progressive disclosure: thinking state, streaming output, clear errors, regeneration, feedback, and source grounding. I avoid fake confidence scores and never trust or render AI-generated HTML without appropriate validation and sanitization."

Q423. How should you talk about AI-assisted development tools in interviews?

Answer:

I would answer honestly and emphasize engineering judgment.

A strong answer would be:

"I use AI tools such as Copilot or Cursor for boilerplate, repetitive code, test scaffolding, debugging ideas, and exploring unfamiliar APIs. I don't blindly accept generated code. I review it, verify important APIs against official documentation, write tests, and pay particular attention to security and edge cases."

Good use cases

Test boilerplate

AI can generate:

describe(...)

it(...)

expect(...)

Then I add meaningful edge cases myself.

Exploring unfamiliar APIs

I can ask for an initial example and then verify it against official documentation.

Repetitive code

For example:

- TypeScript interfaces
- CRUD boilerplate
- React component skeletons
- test setup
- regex patterns

What I wouldn't delegate blindly

I wouldn't blindly ask AI to decide:

- application architecture
- security boundaries
- authentication design
- authorization rules
- database architecture
- critical business logic

Those decisions require understanding the application's requirements and trade-offs.

Risks of AI-generated code

1. Security vulnerabilities
2. Incorrect APIs
3. Outdated APIs
4. Missing edge cases
5. Incorrect assumptions

6. License/IP concerns

7. Over-reliance on AI

Interview summary

"I treat AI as a productivity multiplier, not a substitute for engineering knowledge. It gives me a fast first draft, but I remain responsible for understanding, testing, securing and maintaining the code."

Q424. What is a token, and how is an AI feature priced?

Answer:

AI APIs are generally priced based on **tokens**, rather than simply the number of requests.

A token is a unit of text processed by the model.

There are usually two major categories:

Input tokens

+

Output tokens

=

Total token usage

Input tokens

Everything sent to the model:

System prompt

+

Conversation history

+

User message

+

Tool information

Output tokens

Everything generated by the model.

For example:

Input: 2,500 tokens

Output: 400 tokens

If a hypothetical model costs:

Input: \$3 / 1M tokens

Output: \$15 / 1M tokens

Then:

Input:

$2500 \times \$3 / 1,000,000$

$= \$0.0075$

Output:

$400 \times \$15 / 1,000,000$

$= \$0.006$

Total:

$\$0.0135$

The actual pricing depends on the provider and model, so I would always check the current provider pricing rather than hard-code interview numbers.

Important production insight

A chat can become expensive because every new request may resend the entire conversation.

For example:

Turn 1 → 500 tokens

Turn 2 → 1,000 tokens

Turn 3 → 1,500 tokens

Turn 4 → 2,000 tokens

...

Therefore, context management is both a **performance and cost problem**.

Interview summary

"I estimate AI cost from input and output token usage, with output often being more expensive. In production I monitor actual token usage rather than estimating only from word counts, and I control context size because conversation history can become the dominant input cost."

Q425. How do you reduce the cost of an AI chat application?

Answer:

I would use several layers of cost optimization rather than relying on one technique.

1. Prompt caching

If a large prefix is repeated across requests, provider-supported prompt caching can reduce the cost of processing that repeated content.

For example:

System instructions

+

Stable context

+

Conversation history

If the prefix is cacheable, subsequent requests can reuse it.

The important concept is that caching usually depends on a **stable prefix**.

If I accidentally put this at the beginning:

Current timestamp: 2026-08-15 10:30:01

and it changes every request, the prefix may no longer match the cache.

2. Limit output

Set an appropriate maximum output token limit.

Don't allow a simple question to generate thousands of tokens.

3. Trim conversation history

Instead of:

Entire conversation

use:

System prompt

+

Summary of old conversation

+

Recent messages

4. Model routing

Use inexpensive models for simple tasks.

Classification → small model

Normal chat → mid-tier model

Complex reasoning → expensive model

5. Batch processing

For non-interactive workloads such as:

Nightly summarization

Bulk classification

Document processing

use batch processing where the provider offers lower-cost asynchronous execution.

6. Rate limiting

Prevent one user or buggy client from generating unlimited requests.

User

↓

Authentication

↓

Quota

↓

Rate limiter

↓

AI API

7. Monitoring

Track:

- tokens per request
- cost per user
- cost per feature
- cache hit rate
- model usage

- latency
- retry rate

Interview summary

"I control AI cost through prompt caching, context trimming, model routing, output limits, batching, rate limiting and monitoring. I also keep model credentials on the backend so users can't bypass quotas and directly consume the provider account."

Q426. How do you choose which AI model to use?

Answer:

I don't automatically choose the most powerful model.

I evaluate the task against:

Quality

Latency

Cost

Context requirements

Tool capabilities

Reliability

A common production strategy is tiered model routing.

Small/fast model

Good for:

- classification
- autocomplete
- tagging
- routing
- extraction
- simple transformations

Mid-tier model

Good for:

- general chat
- coding assistance
- content generation
- normal tool-use workflows

Advanced reasoning model

Good for:

- complex debugging
- multi-step reasoning
- difficult planning
- complex code generation
- correctness-critical analysis

Example:

Customer message

↓

Fast model

↓

Classify:

billing / refund / technical

↓

Mid model

↓

Answer

↓

If complex → advanced model

How do I decide when to escalate?

I use production data.

For example:

Thumbs down

Human escalation

Incorrect result

Repeated retries

If a certain class of requests frequently fails with the cheaper model, I can route those requests to a more capable model.

Interview summary

"Model selection is a quality-versus-latency-versus-cost decision. I would start with the cheapest model that meets the quality requirement, measure failures in

production, and selectively escalate difficult requests rather than using the most expensive model everywhere."

Q427. What is tool use or function calling, and how does the tool-use loop work?

Answer:

Tool use allows an AI model to request functions that are implemented by our application.

The most important point is:

The model does not execute our function. It requests the function, and our backend executes it.

For example:

User:

Where is order A-1024?

↓

AI Model

↓

Tool request:

```
get_order_status({  
  orderId: "A-1024"  
})
```

↓

Backend

↓

Order service

↓

```
{  
  status: "shipped",  
  eta: "tomorrow"  
}
```

↓

AI Model

↓

"Your order has shipped..."

Tool-use loop

User request

↓

Model

↓

Tool call?

/ \

No Yes

↓

↓

Answer Execute tool

↓

Tool result

↓

Model

↓

Continue

The model can potentially make several tool calls before finishing.

Why return the tool result to the model?

Because the model needs the actual result to formulate the answer.

For example:

```
{  
  "status": "shipped",  
  "eta": "tomorrow"  
}
```

allows the model to generate:

"Your order has shipped and is expected tomorrow."

Without the result, the model would have to guess.

Interview summary

"Function calling is a structured protocol where the model requests a tool, but my backend executes it. The result is sent back to the model, which can then continue the conversation or request another tool."

Q428. How do you render AI tool calls in the UI?

Answer:

Tool calls introduce an intermediate state that normal chat doesn't have.

Instead of:

User message

↓

[blank 5 seconds]

↓

AI answer

I would show what the assistant is doing:

User:

Where is my order?

AI:

 Looking up order A-1024...

✓ Order found

AI:

Your order has shipped...

Tool states

I would model the tool call explicitly:

type ToolStatus =

| 'pending'

| 'running'

| 'done'

| 'error';

Then render different states.

```
function ToolCall({ call }) {  
  if (call.status === 'running') {  
    return <div>🔧 Looking up order...</div>;  
  }  
  
  if (call.status === 'done') {  
    return <div>✓ Order found</div>;  
  }  
  
  if (call.status === 'error') {  
    return <div>⚠ Failed to retrieve order</div>;  
  }  
  
  return <div>Preparing...</div>;  
}
```

Three important UX principles

1. Make actions visible

The user should understand why the application is waiting.

2. Keep actions in chronological order

If the agent does:

Search

↓

Fetch

↓

Calculate

↓

Update

the UI should reflect that sequence.

3. Don't show raw JSON

Instead of:

```
{  
  "status": "shipped",  
  "eta": "tomorrow"  
}
```

render:

 Order A-1024

Status: Shipped

ETA: Tomorrow

Interview summary

"I treat tool calls as first-class UI states. I show pending, running, success and error states so the user understands what the agent is doing. I render meaningful cards or components instead of exposing raw tool JSON."

Q429. How do you handle side-effectful AI tools safely? What is human-in-the-loop approval?

Answer:

I distinguish between **read-only** and **side-effectful** tools.

Read-only tools

Examples:

get_order()

search_products()

check_inventory()

search_documents()

These can often run automatically.

Side-effectful tools

Examples:

send_email()

delete_account()

charge_card()

submit_order()

transfer_money()

These can change the real world.

For important actions, I introduce a **human approval step**.

AI requests action

↓

Approval UI

↓

User approves

↓

Backend validates permission

↓

Tool executes

↓

Result returned

For example:

```
function ApprovalGate({ call, onApprove, onCancel }) {
```

```
  return (
```

```
    <div>
```

```
      <p>
```

```
        The assistant wants to send this email.
```

```
      </p>
```

```
      <button onClick={onApprove}>
```

```
        Approve
```

```
      </button>
```

```
      <button onClick={onCancel}>
```

```
        Cancel
```

```
    </button>
  </div>
);
}
```

Critical security rule

The frontend approval is **not the security boundary**.

A malicious user can modify frontend JavaScript or directly call the backend API.

Therefore the backend must independently verify:

Authenticated user

↓

Authorization

↓

Tool permissions

↓

Business rules

↓

Execute

Least privilege

Don't give the AI:

`delete_any_user()`

when it only needs:

`delete_current_user_draft()`

Give tools the smallest possible capability.

Interview summary

"I automatically allow safe read-only tools where appropriate, but I gate irreversible or high-impact actions behind explicit user approval. The approval UI is only a UX mechanism; the backend must independently authorize every action."

Q430. What's the difference between a chatbot and an AI agent?

Answer:

A **chatbot** primarily follows:

Input

↓

Model

↓

Response

An **agent** works toward a goal across multiple steps and can use tools.

For example:

Chatbot

User:

Summarize this document.

AI:

Here is the summary...

Agent

User:

Reschedule my Tuesday meetings.

Agent:

↓

Check calendar

↓

Find Tuesday meetings

↓

Check availability

↓

Identify conflicts

↓

Propose changes

↓

Ask approval

↓

Update calendar

↓

Confirm

The agent is effectively running a loop:

Goal

↓

Think/decide

↓

Tool

↓

Observe result

↓

Decide next action

↓

Tool

↓

...

↓

Final result

Should everything be an agent?

No.

Agents introduce:

- more latency
- more model calls
- higher cost
- more complex state
- more failure modes
- more security concerns

For:

Summarization

Translation

Simple Q&A

Classification

a single model call is usually better.

Interview summary

"A chatbot primarily responds to messages, while an agent pursues a goal across multiple steps using tools and the results of those tools. I wouldn't use an agent unless the task genuinely requires dynamic multi-step execution."

Q431. What is model reasoning or extended thinking, and how does it affect frontend UX?

Answer:

Reasoning models can spend additional computation working through a difficult problem before producing the final answer.

For example:

User asks difficult question

↓

Thinking/reasoning phase

↓

Final answer generation

↓

Streaming response

The frontend implication is important:

There may be a significant period where no final answer tokens are visible.

A normal spinner can make the application look stuck.

Instead, I would expose an explicit state:

Thinking...

Then:

Generating answer...

Adaptive reasoning

Modern model APIs can allow the model to dynamically decide how much reasoning effort to use.

Conceptually:

Easy question

↓

Low reasoning effort

Hard question



Higher reasoning effort

This can help balance:

Quality

Latency

Cost

When should reasoning be enabled?

Good candidates:

- complex debugging
- difficult mathematics
- multi-step planning
- complex code generation
- difficult analysis

Poor candidates:

- autocomplete
- classification
- simple extraction
- short factual queries
- high-volume low-latency interactions

Interview summary

"Reasoning introduces a meaningful pre-answer latency, so I represent it explicitly in the UI as a thinking phase rather than making users stare at a generic spinner. I would reserve deeper reasoning for tasks where the quality improvement justifies the additional latency and cost."

Q432. How should you display model reasoning in the UI?

Answer:

I would distinguish between **internal chain-of-thought** and a user-facing **reasoning summary**.

I would not expose private internal chain-of-thought as if it were a normal application log.

Instead, if the model/API supports a user-facing reasoning summary, I can display that summary.

A good UI pattern is:

Thinking...



Reasoning summary



Final answer

For example:

| Thinking... |

| Analyzing the API flow... |

| |

| ▼ Show reasoning summary |

| |

Final answer:

■ ■ ■

Why collapse it?

Most users don't need additional reasoning detail.

A collapsed section gives:

- clean default UI
- transparency when needed
- less visual clutter

Important interview distinction

Don't say:

"I'll display the model's chain of thought."

A better answer is:

"If the provider exposes a safe user-facing reasoning summary, I can render that summary. I wouldn't expose private internal chain-of-thought."

Interview summary

"I treat reasoning as a separate UI phase and, where supported, show a concise reasoning summary rather than exposing private internal chain-of-thought. I would normally keep it collapsed so the final answer remains the primary experience."

Q433. When should you use a reasoning model versus a fast model?

Answer:

I choose based on the task's:

Complexity

+

Accuracy requirements

+

Latency requirements

+

Cost

Fast model

Use for:

Autocomplete

Classification

Simple Q&A

Search suggestions

Extraction

High-volume requests

The priority is:

Low latency + low cost

Reasoning model

Use for:

Complex debugging

Multi-step planning

Difficult mathematics

Complex code generation

High-value analysis

The priority becomes:

Higher accuracy

even if it means more:

Latency

Cost

Example

Suppose I build a coding assistant.

I could use:

Autocomplete

↓

Fast model

Explain an error

↓

Fast/mid model

Debug a complex distributed-system issue

↓

Reasoning model

This is better than using the reasoning model for every interaction.

Interview summary

"I wouldn't apply a reasoning model everywhere. I'd use fast models for latency-sensitive, repetitive tasks and reserve reasoning models for genuinely difficult tasks where additional quality justifies the cost and latency. This is essentially task-based model routing."

Q434. What are the important gotchas with useState?

Answer:

The main useState gotchas are:

1. **Lazy initialization**
2. **Functional updates**
3. **Automatic batching**
4. **State immutability**
5. **Stale closures**

1. Lazy initialization

If the initial state requires an expensive calculation, pass a function:

```
const [state, setState] = useState(() => computeExpensiveState());
```

Instead of:

```
const [state, setState] = useState(computeExpensiveState());
```

The second expression calls `computeExpensiveState()` during every render, although React only uses the result for the initial state.

2. Functional updates

When the new state depends on the previous state, use:

```
setCount(prev => prev + 1);
```

For example:

```
setCount(count + 1);
```

```
setCount(count + 1);
```

may result in only one increment because both updates use the same count from the current render.

Whereas:

```
setCount(prev => prev + 1);
```

```
setCount(prev => prev + 1);
```

correctly increments by 2.

3. Batching

React batches multiple state updates into a single render:

```
setName("Priyanka");
```

```
setAge(28);
```

```
setLoading(false);
```

React generally processes these together rather than rendering three times.

4. State is immutable

Don't mutate objects directly:

```
user.age = 29;
```

```
setUser(user); // ❌
```

Create a new object:

```
setUser(prev => ({  
  ...prev,  
  age: 29  
}));
```

Interview line:

useState updates are asynchronous from the component's point of view, React batches updates, and functional updates should be used whenever the next state depends on the previous state.

Q435. What are the common pitfalls of useEffect?

Answer:

The major problems are:

- Missing dependencies
- Stale closures
- Incorrect cleanup
- Race conditions
- Using useEffect when it isn't necessary

Example of a stale closure:

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log(count);  
  }, 1000);  
  
  return () => clearInterval(id);  
}, []);
```

Because count isn't in the dependency array, the callback captures the value from the render that created the effect.

A better approach when updating state is:

```
useEffect(() => {  
  const id = setInterval(() => {  
    setCount(prev => prev + 1);  
  }, 1000);  
  
  return () => clearInterval(id);  
}, []);
```

Interview line:

The dependency array determines when an effect is synchronized again, and cleanup is essential for subscriptions, timers, event listeners, and cancellable asynchronous operations.

Q436. How do you handle race conditions in useEffect?

Answer:

Suppose the user quickly changes:

userId = 1

↓

userId = 2

Two requests may be running simultaneously.

If request 2 finishes first and request 1 finishes later, request 1 can overwrite the newer data.

Solution 1 — cancellation flag

```
useEffect(() => {  
  let cancelled = false;  
  
  fetchUser(userId).then(data => {  
    if (!cancelled) {  
      setUser(data);  
    }  
  });  
  
  return () => {  
    cancelled = true;  
  };  
}, [userId]);
```

Solution 2 — AbortController

```
useEffect(() => {  
  const controller = new AbortController();  
  
  fetch(`/api/users/${userId}`, {  
    signal: controller.signal  
  })  
    .then(res => res.json())
```

```

    .then(setUser)
    .catch(error => {
      if (error.name !== "AbortError") {
        setError(error);
      }
    });

    return () => controller.abort();
  }, [userId]);

```

Interview line:

Cleanup prevents an obsolete effect from updating state after a newer effect has become active.

Q437. What are the performance problems with useContext?

Answer:

When a Context provider's value changes, consumers of that context can re-render.

For example:

```

const value = {
  user,
  theme
};

```

```

<UserContext.Provider value={value}>
  {children}
</UserContext.Provider>

```

If value changes, consumers may re-render even when they only care about one part of the context.

Solutions

1. Split contexts

UserContext

ThemeContext

AuthContext

Instead of one huge context.

2. Memoize provider values

```
const value = useMemo(  
  () => ({ user, setUser }),  
  [user]  
);
```

3. Use a state-management solution

For complex shared state, libraries such as Zustand or Redux can provide more targeted subscriptions.

Interview line:

Context is excellent for dependency injection and relatively low-frequency shared state, but it isn't automatically an optimized global state-management solution.

Q438. When should you use useReducer instead of useState?

Answer:

Use useReducer when state transitions become complex or multiple pieces of state change together.

For simple state:

```
const [isOpen, setIsOpen] = useState(false);
```

is preferable.

For complex state:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Example:

```
const initialState = {  
  values: {},  
  errors: {},  
  touched: {},  
  isSubmitting: false  
};
```

Then:

```
dispatch({  
  type: "SUBMIT_START"  
});
```

The reducer handles the complete transition.

Advantages

- Centralizes state transitions
- Makes complex logic easier to understand
- Actions are explicit
- Reducers are pure and testable
- Easier to debug

Interview line:

I use useState for simple independent state and useReducer when multiple related state changes form a state machine or the update logic becomes difficult to manage with multiple setters.

Q439. When does useCallback actually help?

Answer:

useCallback memoizes a function reference.

It is most useful when the function is passed to a memoized child:

```
const handleClick = useCallback((id) => {  
  setSelectedId(id);  
}, []);
```

```
return (  
  <ExpensiveList  
    items={items}  
    onItemClick={handleClick}  
  />  
);  
If:
```

```
const ExpensiveList = React.memo(...)
```

then keeping onItemClick stable can prevent unnecessary child renders.

Common misuse

```
const handleClick = useCallback(() => {  
  setCount(c => c + 1);  
}, []);
```



```
return <button onClick={handleClick}>Click</button>;
```

This usually provides no meaningful benefit because a normal DOM button isn't going to skip its parent's rendering because of `useCallback`.

Interview line:

I don't use `useCallback` everywhere. I use it when referential equality matters, typically with `React.memo`, hook dependencies, or APIs that depend on stable function references.

Q440. When should you use `useMemo`?**Answer:**

`useMemo` caches the result of an expensive calculation.

```
const filteredProducts = useMemo(() => {  
  return products  
    .filter(p => p.category === category)  
    .sort((a, b) => a.price - b.price);  
}, [products, category]);
```

This can avoid repeating expensive work when unrelated state changes.

Good use case

```
const result = useMemo(() => {  
  return expensiveCalculation(data);  
}, [data]);
```

Bad use case

```
const fullName = useMemo(  
  () => `${firstName} ${lastName}`,  
  [firstName, lastName]  
);
```

The calculation is so cheap that memoization may add unnecessary complexity and overhead.

Interview line:

`useMemo` is a performance optimization, not something I use by default. I first identify an expensive computation or a referential-equality requirement.

Q441. What are the use cases for `useRef` beyond DOM access?

Answer:

useRef stores a mutable value that survives renders without causing a render when changed.

1. DOM access

```
const inputRef = useRef(null);
```

```
inputRef.current?.focus();
```

2. Previous value

```
const previous = useRef();
```

```
useEffect(() => {  
  previous.current = value;  
}, []);
```

3. Timer IDs

```
const timerRef = useRef(null);
```

```
timerRef.current = setInterval(() => {  
  // ...  
}, 1000);
```

4. Mutable instance variables

```
const requestId = useRef(0);
```

Changing:

```
requestId.current++;  
doesn't cause a render.
```

Key difference:

useState

↓

persistent + reactive

useRef

↓

persistent + non-reactive

Q442. What is the difference between `useEffect` and `useLayoutEffect`?

Answer:

The important difference is **when they execute relative to browser painting**.

Simplified lifecycle:

React renders

↓

DOM mutations

↓

`useLayoutEffect`

↓

Browser paints

↓

`useEffect`

Use `useLayoutEffect` when you need to measure or synchronously modify layout before the user sees the result.

Example:

```
useLayoutEffect(() => {  
  const rect = elementRef.current.getBoundingClientRect();  
  
  elementRef.current.style.left = `${rect.width}px`;  
}, []);
```

Typical use cases:

- Measuring DOM dimensions
- Positioning tooltips
- Restoring scroll position
- Preventing visual flicker

Use `useEffect` for:

- API calls
- Subscriptions
- Analytics
- Logging

Interview line:

useLayoutEffect runs synchronously after DOM updates but before paint, so I use it only for layout-sensitive work because it can block painting.

Q443. What does useImperativeHandle do?**Answer:**

useImperativeHandle lets a child control what a parent receives through a ref.

Instead of exposing the entire DOM element:

```
useImperativeHandle(ref, () => ({  
  focus() {  
    inputRef.current.focus();  
  },  
  
  clear() {  
    inputRef.current.value = "";  
  }  
}));
```

The parent can do:

```
inputRef.current.focus();
```

```
inputRef.current.clear();
```

This is useful for reusable component APIs such as:

- Custom inputs
- Modals
- Video players
- Rich text editors
- Component libraries

Interview line:

useImperativeHandle lets a child expose a controlled imperative API instead of exposing its complete internal DOM implementation.

Q444. What is useId and why was it introduced?**Answer:**

useId generates stable unique IDs that work correctly with server rendering and hydration.

```
function FormField({ label }) {  
  const id = useId();  
  
  return (  
    <>  
      <label htmlFor={id}>{label}</label>  
      <input id={id} />  
    </>  
  );  
}
```

This is particularly useful for accessibility:

label

↓ htmlFor

input

↓ id

Before useId, developers might use:

Math.random()

which can generate different values between server and client.

Interview line:

useId provides stable IDs across server and client rendering, making it especially useful for accessible form controls and avoiding hydration mismatches.

Q445. How does useTransition work?

Answer:

useTransition allows us to mark an update as **non-urgent**.

```
const [isPending, startTransition] = useTransition();
```

Then:

```
setQuery(value);
```

```
startTransition(() => {  
  setResults(expensiveResults(value));  
});
```

```
});
```

Here:

Typing/input update

↓

urgent

↓

updates immediately

Filtering results

↓

non-urgent

↓

can be interrupted

isPending tells us whether the transition is still processing.

Example

```
function Search() {  
  const [query, setQuery] = useState("");  
  const [results, setResults] = useState([]);  
  
  const [isPending, startTransition] = useTransition();  
  
  function handleChange(e) {  
    const value = e.target.value;  
  
    setQuery(value);  
  
    startTransition(() => {  
      setResults(filterLargeDataset(value));  
    });  
  }  
  
  return (  

```

```

<>
  <input value={query} onChange={handleChange} />

  {isPending && <p>Updating results...</p> }

  <Results results={results} />
</>
);
}

```

Interview line:

useTransition improves responsiveness by allowing urgent updates such as typing to take priority over expensive non-urgent UI updates.

Q446. What is useDeferredValue?

Answer:

useDeferredValue lets us defer the consumption of a value.

```
const deferredQuery = useDeferredValue(query);
```

The original value remains current:

```
query
```

while expensive components can temporarily use:

```
deferredQuery
```

Example:

```

function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);

  return (
    <ExpensiveList query={deferredQuery} />
  );
}

```

useTransition vs useDeferredValue

Hook	What it defers
useTransition	State update

Hook	What it defers
------	----------------

useDeferredValue	Value/ consumer
------------------	--------------------

Use:

useTransition

→ I control the state update

useDeferredValue

→ I receive the value and want to defer its consumer

Q447. What is useSyncExternalStore?

Answer:

useSyncExternalStore allows React components to safely subscribe to data that exists outside React's state system.

Examples:

- Browser online status
- Media queries
- External stores
- Third-party state-management libraries

Example:

```
function useOnlineStatus() {  
  return useSyncExternalStore(  
    subscribe,  
    getSnapshot,  
    getServerSnapshot  
  );  
}
```

The important pieces are:

subscribe

↓

tells React when the external store changes

getSnapshot



gets current value

getServerSnapshot



gets value during SSR

Interview line:

useSyncExternalStore provides a React-safe way to subscribe to external mutable stores, particularly important with concurrent rendering and SSR.

Q448. What is useInsertionEffect?

Answer:

useInsertionEffect is designed primarily for CSS-in-JS libraries.

Its purpose is to allow styles to be inserted before layout effects read the DOM.

Conceptually:

useInsertionEffect



DOM mutations



useLayoutEffect



paint



useEffect

You generally shouldn't use it for normal application logic.

It exists mainly for libraries such as CSS-in-JS systems that need to inject styles at the correct point in React's rendering lifecycle.

Interview line:

useInsertionEffect is a specialized hook intended primarily for CSS-in-JS libraries, not general application effects.

Q449. What is useOptimistic in React 19?

Answer:

useOptimistic is used for optimistic UI updates.

Instead of waiting for the server:

User action

↓

Show expected result immediately

↓

Send request

↓

Server confirms

Example:

```
const [optimisticTodos, addOptimisticTodo] = useOptimistic(
  todos,
  (currentTodos, newTodo) => [
    ...currentTodos,
    {
      ...newTodo,
      pending: true
    }
  ]
);
```

Then:

```
addOptimisticTodo(newTodo);
```

```
await addTodoAction(newTodo);
```

The user immediately sees:

New Todo (saving...)

When the real server state arrives, React reconciles the optimistic state with the underlying state.

Interview line:

useOptimistic simplifies optimistic UI by allowing the UI to temporarily represent the expected result while the actual operation is still pending.

Q450. What is useFormStatus?

Answer:

useFormStatus gives a child component information about the nearest parent form's submission status.

Example:

```
function SubmitButton() {  
  const { pending } = useFormStatus();  
  
  return (  
    <button disabled={pending}>  
      {pending ? "Submitting..." : "Submit"}  
    </button>  
  );  
}
```

Usage:

```
<form action={submitForm}>  
  <input name="email" />  
  <SubmitButton />  
</form>
```

Important point:

useFormStatus must be used in a component rendered **inside** the form.

It reads the status of its parent form.

Q451. What is useActionState?

Answer:

useActionState connects an action with state and pending status.

```
const [state, formAction, isPending] =  
  useActionState(action, initialState);
```

The action can return:

```
{  
  errors: {  
    email: "Invalid email"  }  
}
```

```
},  
  success: false  
}
```

The component can then render the result:

```
<form action={formAction}>  
  <input name="email" />  
  
  {state?.errors?.email && (  
    <p>{state.errors.email}</p>  
  )}  
  
  <button disabled={isPending}>  
    {isPending ? "Submitting..." : "Submit"}  
  </button>  
</form>
```

Why is it useful?

It avoids manually coordinating:

loading state

+

error state

+

success state

+

form action

Interview line:

useActionState is useful when a form action needs to return structured state such as validation errors, success information, and pending status.

Q452. What is the use() API in React?

Answer:

use() can read a Promise or Context during rendering.

For a Promise:

```
const product = use(productPromise);
```

If the Promise hasn't resolved, React suspends the component.

Promise pending

↓

Suspense fallback

↓

Promise resolves

↓

Component renders

It can also read Context:

```
const theme = use(ThemeContext);
```

One important difference is that use() can be called conditionally.

```
if (showAdmin) {  
  const settings = use(AdminContext);  
}
```

Regular hooks cannot generally be called conditionally.

Interview line:

use() integrates Promise and Context consumption with React's rendering and Suspense model, and unlike traditional hooks it can be called conditionally.

Q453. How do you build a custom hook?

Answer:

A custom hook is a reusable JavaScript function that uses React hooks internally.

Example:

```
function useLocalStorage(key, initialValue) {  
  const [value, setValue] = useState(() => {  
    const stored = localStorage.getItem(key);  
  
    return stored  
      ? JSON.parse(stored)  
      : initialValue;  
  });
```

```
useEffect(() => {
  localStorage.setItem(
    key,
    JSON.stringify(value)
  );
}, [key, value]);

return [value, setValue];
}
```

Usage:

```
const [theme, setTheme] =
  useLocalStorage("theme", "light");
```

Good custom-hook principles

- Start the name with use
 - Keep the API small
 - Accept configuration through parameters
 - Handle cleanup
 - Make reusable logic independent of UI
 - Type the API with TypeScript
 - Test state transitions
-

Q454. How do you test custom hooks?

Answer:

React Testing Library provides `renderHook` for testing custom hooks.

For example, conceptually:

```
const { result } = renderHook(() =>
  useCounter()
);
```

Then trigger updates inside `act()`:

```
act(() => {
  result.current.increment();
});
```

```
});
```

For asynchronous hooks, use:

```
await waitFor(() => {  
  expect(result.current.loading).toBe(false);  
});
```

The important idea is to test the **observable behavior of the hook**, rather than its internal implementation.

Q455. What are the Rules of Hooks?

Answer:

There are two fundamental rules:

Rule 1 — Call hooks at the top level

Don't call hooks inside:

- if
- for
- while
- nested functions
- event handlers

Incorrect:

```
if (isLoggedIn) {  
  useEffect(() => {  
    // ✗  
  }, []);  
}
```

Correct:

```
useEffect(() => {  
  if (isLoggedIn) {  
    // logic  
  }  
}, [isLoggedIn]);
```

Rule 2 — Call hooks only from React functions

Hooks should be called from:

React components

Custom hooks

Not ordinary JavaScript functions.

Q456. Why can't hooks be called conditionally?

Answer:

React associates hooks with their **call order**.

Consider:

```
const [name, setName] = useState("");  
const [age, setAge] = useState(0);  
const theme = useContext(ThemeContext);
```

React internally relies on this consistent order.

If we do:

```
if (showAge) {  
  const [age, setAge] = useState(0);  
}
```

then the hook sequence can change:

First render

Hook 1 → name

Hook 2 → age

Hook 3 → theme

Second render

Hook 1 → name

Hook 2 → theme

Now React's hook positions no longer match.

Interview line:

Hooks must execute in the same order on every render because React associates hook state with their call order.

Q457. What happens if you call useState inside a normal JavaScript function?

Answer:

Hooks depend on React's internal rendering context and hook state associated with a component's fiber.

A normal JavaScript function doesn't have that context.

Incorrect:

```
function calculateSomething() {  
  const [value, setValue] = useState(0); // ❌  
}
```

Instead, either call it from a component:

```
function Component() {  
  const [value, setValue] = useState(0);  
}
```

or turn the reusable logic into a custom hook:

```
function useSomething() {  
  const [value, setValue] = useState(0);  
  
  return [value, setValue];  
}
```

Q458. What is the exception to the Rules of Hooks?

Answer:

The React use() API is the important exception.

Unlike traditional hooks, use() can be called conditionally:

```
if (showAdmin) {  
  const settings = use(AdminContext);  
}
```

It can also be used inside loops.

Regular hooks such as:

useState()

useEffect()

useMemo()

useCallback()

useContext()

must follow the normal Rules of Hooks.

Interview one-liner:

use() is specifically designed as an exception and can be called conditionally, while traditional hooks must preserve call order.

Q459. useCallback vs useMemo — what's the difference?

Answer:

The simplest distinction is:

useMemo

→ memoizes a VALUE

useCallback

→ memoizes a FUNCTION

Example:

```
const result = useMemo(  
  () => expensiveCalculation(data),  
  [data]  
);
```

versus:

```
const handleClick = useCallback(  
  () => doSomething(id),  
  [id]  
);
```

You can think of:

useCallback(fn, deps)

roughly as:

useMemo(() => fn, deps)

Interview line:

useMemo caches a computed result, while useCallback caches the function reference itself.

Q460. useTransition vs useDeferredValue?

Answer:

The difference is **where you control the deferral**.

useTransition

You control the state update:

```
startTransition(() => {  
  setResults(results);  
});
```

useDeferredValue

You already have the value and want to defer its consumers:

```
const deferredQuery = useDeferredValue(query);
```

Think:

useTransition

"I control the update."

useDeferredValue

"I receive the value."

Q461. useEffect vs useLayoutEffect — which should you prefer?

Answer:

Prefer useEffect by default.

Use:

```
useLayoutEffect(...)
```

only when you must perform DOM measurement or visual mutation before paint.

Example:

Need API call?

→ useEffect

Need analytics?

→ useEffect

Need subscription?

→ useEffect

Need measure element before user sees it?

→ `useLayoutEffect`

Need position tooltip before first paint?

→ `useLayoutEffect`

Senior-level answer:

`useLayoutEffect` can block painting, so I treat it as a specialized tool rather than a replacement for `useEffect`.

🔥 Q462. Explain the complete React Hooks mental model.

Answer:

I think about hooks in these categories:

STATE

|

├─ `useState`

└─ `useReducer`

SIDE EFFECTS

|

├─ `useEffect`

├─ `useLayoutEffect`

└─ `useInsertionEffect`

REFERENCES

|

└─ `useRef`

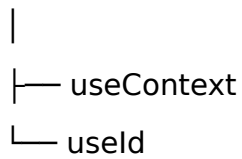
MEMOIZATION

|

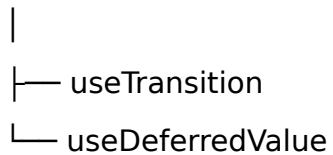
├─ `useMemo`

└─ `useCallback`

CONTEXT / IDs



CONCURRENCY



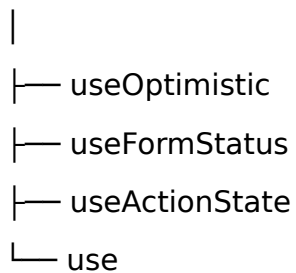
EXTERNAL STATE



IMPERATIVE APIs



REACT 19 / ACTIONS



CUSTOMIZATION



The most important interview principle is:

Don't use a hook because it exists. Use it because the problem requires the behavior that hook provides.

